From Silicon to Software · 从硅片到软件

一本把电子工程(EE)与计算机科学(CS)知识串联起来的「大地图」之书

— pans —

目录 · Table of Contents

1. 前言 · Preface
2. 第 1 章 · 总览:The Big Map
3. 第 2 章 · 半导体物理:为什么是硅?
4. 第 3 章 · PN 结与二极管
5. 第 4 章 · MOSFET 与 CMOS:数字世界的开关
6. 第 5 章 · 布尔代数与逻辑门
7. 第 6 章 · 组合逻辑:加法器、译码器、多路选择器
8. 第 7 章 · 时序逻辑:锁存器、触发器、有限状态机
9. 第 8 章 · Verilog 入门:用代码描述硬件
10. 第 9 章 · EDA 流程:综合、布局布线、签核
11. 第 10 章 · 网表 (Netlist):门级世界的「汇编语言」
12. 第 11 章 · 数据通路与控制器:一颗 CPU 的雏形
13. 第 12 章 · 流水线、冒险、分支预测
14. 第 13 章 · 存储层次:寄存器、Cache、内存、磁盘
15. 第 14 章 · 指令集架构 (ISA):软硬件契约
16. 第 15 章 · 机器码与汇编语言
17. 第 16 章 · 链接、加载、ELF 文件结构
18. 第 17 章 · C 语言:可移植的汇编

19. 第 18 章 · 内存模型:栈、堆、指针、地址空间
20. 第 19 章 · 从 C 到汇编:每一行代码的代价
21. 第 20 章 · 变量、表达式、类型系统
22. 第 21 章 · 控制流:条件、循环、函数、递归
23. 第 22 章 · 抽象机制:作用域、闭包、对象、泛型
24. 第 23 章 · 编译原理总览:词法 → 语法 → 语义 → IR → 目标代码
25. 第 24 章 · 编译器实战:写一个迷你编译器
26. 第 25 章 · 解释器、虚拟机与 JIT
27. 第 26 章 · 操作系统:管理硬件资源的软件
28. 第 27 章 · 一行 `print("Hello")` 的完整旅程
29. 第 28 章 · 全栈视角:抽象层的代价与意义
30. 附录 A · 推荐阅读与课程
31. 附录 B · 术语表(中英对照)
32. 附录 C · 实验环境搭建

前言 · Preface

写这本书的动机

我学过半导体物理,知道 PN 结里的载流子是怎么扩散的。 我学过数字电路,会画与或非门、能用真值表化简卡诺图。 我写过 Verilog,跑过 EDA 综合,看过门级网表。 我学过计算机组成原理,能背出五级流水线和 Cache 的命中率公式。 我写过汇编,调过 C 程序的段错误。 我用 Python 写过爬虫,用 JavaScript 写过网页。

但有一天,我突然意识到:**这些知识在我脑子里是一座座孤岛。**

当我在 Python 里写下:

```
a = 1 + 2
print(a)
```

我能不能,从硅原子里电子的运动,一直讲到屏幕上那个 `3` 的字符是怎么被点亮的? 中间不允许跳过任何一层?

试了一下,发现我讲不全。

- 我知道 CMOS 是个开关,但我说不清它怎么变成 NAND 门。
- 我知道 NAND 门能搭加法器,但我说不清加法器怎么变成 ALU。
- 我知道 ALU 是 CPU 的一部分,但我说不清 CPU 怎么"读懂"一条 `add` 指令。
- 我知道 C 编译成汇编,但我说不清 Python 解释器怎么变成 C 函数调用。

这本书,就是我把这些断点一个一个补上,顺便整理成一张完整地图的过程。

这本书写给谁?

- **学过 EE 的工程师**:你熟悉硅,但对软件世界的"上层建筑"心里没底。
- **学过 CS 的程序员**:你写过无数代码,但对"代码最终怎么变成电子的运动"始终好奇。
- **跨学科的学生**:你想要一张能挂在墙上、随时定位自己位置的知识地图。
- **喜欢"打通任督二脉"那种顿悟感的人**:这本书每一章都在做一件事 —— 把两层之间的"魔法"拆穿。

这本书不是什么?

- ❌ **不是教科书的替代品。**每一个领域都有它的圣经(《Computer Organization and Design》、《编译原理》龙书、《半导体物理》……),本书不重复造轮子。
- ❌ **不追求每一层都讲到博士深度。**我们追求的是**广度上的连续性**,而不是深度上的极致。
- ❌ **不是"21 天精通 XX"。**读完这本书,你不会变成 IC 设计专家,也不会变成编译器专家。但你会拥有一张地图,知道自己想深入哪里。

这本书的特色

1. 一根主线:抽象的阶梯

每一章都在回答两个问题:

> **它是什么?(WHAT) 它如何"长成"上一层?(HOW does it become the next layer?)**

例如:

- 学 CMOS 时,我们不止讲 PMOS/NMOS 的物理特性,还要讲**它如何拼成一个 NAND 门。**
- 学 NAND 门时,我们不止讲真值表,还要讲**它如何拼成一个 D 触发器。**
- 学 Verilog 时,我们不止讲语法,还要讲**EDA 综合工具如何把它变回一堆门级网表。**
- 学汇编时,我们不止讲指令格式,还要讲**CPU 控制器如何"看懂"这串 0/1。**

2. 跨学科类比

EE 和 CS 是同一棵树的两根枝。本书不断在两边之间架桥:

EE 概念	CS 类比
网表 (Netlist)	门级电路的"汇编语言"
EDA 综合	Verilog → 网表的"编译器"
时钟 (Clock)	同步系统的"心跳"(类似事件循环)
触发器 (Flip-Flop)	一比特的"变量"(状态存储)
状态机 (FSM)	程序的 `switch-case` 控制流
流水线 (Pipeline)	软件中的生产者-消费者并发模型
Cache	程序中的 LRU 缓存 / Memoization

3. 最小可运行例子 (MWE)

每个关键概念都配一个**能跑起来**的最小例子:

- CMOS NAND:LTspice / Falstad 仿真链接
- 4 位加法器:Verilog + Icarus Verilog 仿真
- 五级流水线 CPU:简化版 RISC-V 实现
- 迷你编译器:Python 实现的算术表达式 → 类汇编

读者拿过来就能改、能跑、能看到波形/输出。

4. 承认边界

每一章末尾都有一个固定栏目:

> **🔍 本章没讲什么 📚 想深入读什么**

例如,讲完 PN 结,我们会说:"本章没讲量子隧穿、热载流子注入、短沟道效应。如果你想深入,推荐 Sedra/Smith 或者 Razavi。"

这避免了"读完了感觉啥都懂,但实际啥都不会"的错觉。

怎么读这本书?

推荐方式:从前到后顺读。 因为本书的核心价值就在于"连续性"。

但如果你时间有限,也可以:

- 🚀 **快速通读**:只读每章的"What"和"How it bridges to next layer"两节,用 1-2 个晚上把地图过一遍。
- 🎯 **精读某层**:你已经懂硅,只想补 CS,可以从 Part IV(体系结构)开始。
- 🌉 **专挑"桥梁章节"**:Ch 04(CMOS→门)、Ch 10(Verilog→网表)、Ch 14(硬件→ISA)、Ch 23(语言→编译器)是四座关键的桥。

致谢与声明

这本书的大部分内容,我并不是它们的"原创发现者"——所有的物理、电路、计算机原理,都是几十年来无数前辈的成果。我只是把它们重新串成一条线。

所以这本书的真正贡献,如果有的话,只在于**叙述的顺序与连接的方式。**

最后,这本书是开源的、活的。它不是一次写成的,而是会随着我自己的理解加深不断重写。

让我们从一粒沙开始。

→ 进入下一章:总览 The Big Map

第 1 章 · 总览:The Big Map

> 在深入每一层之前,先把这张地图挂在墙上。

1.1 一个引子: `print("Hello, World!")` 的旅程

让我们从一个看似最简单的问题出发:

```
print("Hello, World!")
```

当你按下回车,屏幕上出现 `Hello, World!`。**这中间究竟发生了什么?** 试着倒着看:

1. 屏幕上的字符是由像素点亮组成的 → 显卡发出信号
2. 显卡的信号来自 CPU 的 I/O 操作 → CPU 执行了一连串指令
3. 这些指令是 Python 解释器(本身是一个 C 程序)在执行 → 解释器把 `print(...)` 翻译成了对系统调用 `write()` 的请求
4. `write()` 是 C 标准库的函数 → 它最终触发软中断,陷入操作系统内核
5. 内核把数据交给终端驱动 → 驱动通过总线把数据送到显示控制器
6. 显示控制器逐个像素扫描 → 液晶/OLED 像素被通电
7. 每一次「通电」背后,是无数 CMOS 晶体管在导通和截止 → PMOS/NMOS 的栅极电压让沟道形成
8. 沟道里的载流子(电子/空穴)是半导体掺杂出来的 → 硅原子的能带结构允许这一切

这就是从软件到硅的一条完整链路。 而本书要做的,就是反过来 —— **从硅开始,一层一层往上爬**,直到我们能讲清楚为什么 `print("Hello")` 会成为现在这个样子。

1.2 抽象的阶梯 (The Ladder of Abstraction)

整个计算机系统,本质上是一座**抽象的阶梯**。每一层都向上隐藏自己的实现细节,只暴露一组「接口」。

```
第 9 层 │ 应用程序 (Web App / Game / AI 模型)
第 8 层 │ 高级语言 (Python / Java / JS)
第 7 层 │ 运行时 / 虚拟机 / 解释器
第 6 层 │ 系统编程语言 (C / C++ / Rust)
第 5 层 │ 操作系统 (Linux / Windows)
第 4 层 │ 汇编 / 机器码 (ISA: x86 / RISC-V)
────────┼──────────────  ← 软硬件分界线
第 3 层 │ 微架构 (Pipeline / Cache / Branch Predictor)
第 2 层 │ RTL / 网表 (Verilog → Gates)
第 1 层 │ 数字逻辑 (NAND / FlipFlop / FSM)
第 0 层 │ 器件 (CMOS / MOSFET / PN Junction)
第-1 层 │ 物理 (硅原子 / 能带 / 电子)
```

每一层之间,都有一个「翻译者」或「桥」:

从	到	桥
硅 → 晶体管	物理 → 器件	**掺杂 + 光刻**
晶体管 → 逻辑门	器件 → 数字逻辑	**CMOS 电路拓扑**
逻辑门 → 模块	门 → RTL	**组合 + 时序设计**
Verilog → 网表	RTL → 门	**EDA 综合工具**
网表 → CPU	门 → 微架构	**数据通路 + 控制器**
CPU → ISA	微架构 → 指令集	**指令解码 + 执行**
汇编 → 机器码	助记符 → 二进制	**汇编器 (Assembler)**
C → 汇编	高层 → 低层	**编译器 (Compiler)**
Python → C 调用	解释 → 执行	**解释器 / VM / JIT**
程序 → 进程	静态 → 动态	**操作系统加载器**

> **本书的章节,就是按这张表组织的。每一座「桥」都是一章的核心。**

1.3 两个世界,同一种思想

EE 和 CS 看似分裂,但它们共享同一种核心思想:**用简单元素的组合,搭建出极其复杂的系统。**

维度	EE 世界	CS 世界
最小元素	晶体管 (CMOS)	比特 (0/1)
基本组合	逻辑门 (NAND/NOR)	函数 / 语句
模块化	IP 核 (Adder, MUX)	函数 / 类 / 模块
描述语言	Verilog / VHDL	C / Python
编译产物	网表 (Gate-level)	汇编 / 字节码
时序模型	时钟驱动 (Clock)	事件循环 / 调度
状态	触发器 (FlipFlop)	变量 (Variable)
控制流	有限状态机 (FSM)	switch / if / loop
通信	总线 (Bus)	API / 消息
复用	流水线 (Pipeline)	异步 / 并发

注意最后两行:流水线和并发,本质上是同一件事 —— 让多个任务在时间上重叠。EE 工程师用触发器分级,CS 工程师用线程分阶段,但思想是同源的。

1.4 这本书的「四座桥」

如果整本书你只读 4 章,我推荐这四章 —— 它们是连接各层最关键的「桥梁」。

🌉 桥 1:CMOS → NAND 门 (Ch 04 → Ch 05)

> *「两个开关怎么就能算 0 和 1 了?」*

这是从**物理世界**进入**逻辑世界**的第一座桥。

🌉 桥 2:Verilog → 网表 (Ch 08 → Ch 10)

> *「我写一段 `always @(posedge clk)`,工具是怎么把它变成一堆门的?」*

这是从**人类描述**进入**机器实现**的桥。EDA 综合工具的本质,是一个「硬件编译器」。

🌉 桥 3:机器码 → 微架构 (Ch 14 → Ch 11)

> *「CPU 凭什么能读懂 `0x83C501` 这串数字,并且执行加法?」*

ISA 是软硬件之间的「合同」,一面是程序员,一面是芯片设计师。

🌉 桥 4:源码 → 可执行 (Ch 23 → Ch 27)

> *「我写的 `a + b`,经过几次翻译才变成 CPU 上跑的电流?」*

编译器、链接器、加载器、解释器 —— 它们共同构成了从人类思想到电子运动的流水线。

1.5 阅读地图

🔧 EE 出身,想补 CS

重点读:**Part IV** (体系结构)、**Part V** (ISA & 汇编)、**Part VIII** (编译器)。

💻 CS 出身,想补 EE

重点读:**Part I** (器件层)、**Part II-III** (数字逻辑 + Verilog)、**Part IV** (体系结构)。

🎓 学生 / 入门读者

按章节顺序读。每读完一部分,问自己:*「如果让我向小学生解释这一层是什么,我会怎么说?」*

1.6 一些约定

- 📦 **代码块**:所有可运行代码都标注语言,并尽量给出运行命令。
- 🔬 **MWE**: `Minimum Working Example`,最小可运行示例。
- ⚠️ **常见误解**:标出新手容易踩的坑。
- 🔍 **本章没讲什么**:每章末尾,坦诚地告诉你边界在哪里。
- 📚 **想深入读什么**:每章末尾,给出 1-3 本经典书籍。

1.7 准备好了吗?

合上这张地图,我们要从最底层 —— **一粒沙子(硅原子)** —— 开始,一步步爬上去。

> 沙子是怎么变成 CPU 的?一个开关是怎么变成 ChatGPT 的?

翻到下一页,我们出发。

← 返回前言 · → 进入 Part I:从沙子到晶体管

第 2 章 · 半导体物理:为什么是硅?

> 我们的计算机文明,建立在一种很普通的元素上:硅。 整片沙漠都是硅。但为什么是它?

2.1 从导体、绝缘体到半导体

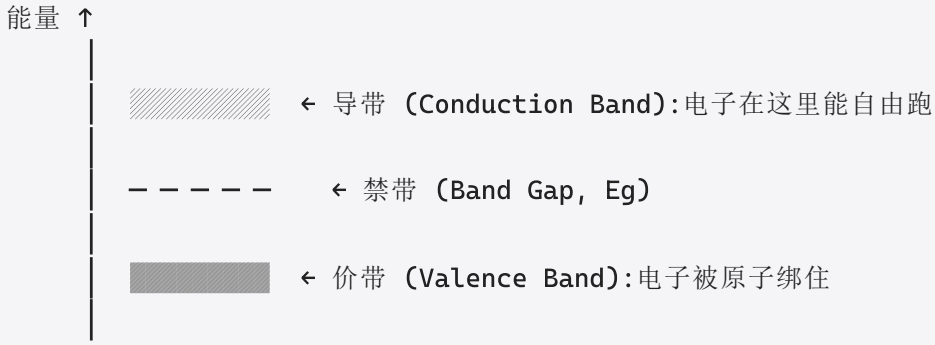
世界上的材料,按导电性可以粗分三类:

类别	例子	电阻率 (Ω·m)	电子状态
导体	铜、银	10^{-8} ~ 10^{-6}	自由电子很多
绝缘体	玻璃、橡胶	10^{8} 以上	电子被牢牢束缚
半导体	硅、锗	10^{-4} ~ 10^{7}	**可控**

「可控」三个字,是整本书的起点。 计算机本质上是一群"可控开关"的合奏。如果一种材料天生就导电(像铜),你没法关掉它;如果它天生就绝缘(像橡胶),你也没法打开它。只有半导体——它的导电能力**可以被外加电压调节**——才能成为开关。

2.2 能带模型:为什么硅可以「半」导?

固体物理告诉我们,原子结合成晶体后,电子的能量不再是一个个分立值,而是形成一段段**能带**(Band):



- **导体**:导带和价带重叠,电子随时能跑 → 永远导电。
- **绝缘体**:禁带很宽(>5 eV),电子跳不过去 → 永远不导电。
- **半导体**:禁带不宽不窄(硅约 1.12 eV)→ **常温下少量电子能跳上去,温度/电压/光照会显著改变这个数量。**

> 💡 **这就是「可控」的物理来源**:能隙刚好让我们能用毫伏到伏特级别的电压,改变多少电子在导带里跑。

2.3 掺杂:把「半导」变成「定向导」

纯硅(本征半导体)虽然可控,但导电能力太弱,而且对电子和空穴一视同仁。我们要做的下一步,是**掺杂(Doping)**:

- **N 型**:在硅里掺入磷(P)等五价元素,多出一个自由电子 → 多数载流子是**电子**。
- **P 型**:在硅里掺入硼(B)等三价元素,缺一个电子(留下「空穴」)→ 多数载流子是**空穴**。

```
本征硅:          Si – Si – Si        N 型:          Si – P  – Si
                 |    |    |                         |    |    |
                 Si – Si – Si                        Si – Si – Si
                                                            ↑
                                                       多一个自由电子
```

掺杂带来的关键变化:**我们得到了两种"性格"相反的材料**——N 型擅长输送电子,P 型擅长输送空穴。

> ⚠️ **常见误解**:不是 P 型材料「带正电」、N 型「带负电」。它们整体都是**电中性的**,只是「主要由谁来导电」不同。

2.4 为什么偏偏是「硅」?

历史上,锗(Ge)曾经是半导体之王(第一只晶体管就是锗的),但很快被硅取代。原因:

1. **储量丰富**:沙子的主要成分就是 SiO_2,地球上几乎取之不尽。

2. **氧化物稳定**:硅的氧化物 SiO_2 是天然绝缘体,在硅片表面长一层就成了完美的「栅极氧化层」(MOS 中的 M-O-S 的 O)。锗的氧化物则会溶于水。
3. **能隙合适**:1.12 eV 既不会导致室温下漏电太大,也不会让器件无法工作。
4. **温度耐受**:硅器件能在 100°C 以上正常工作,锗大约 70°C 就开始出问题。

> 📌 这第 2 点至关重要 —— **没有 SiO_2,就没有 MOS,就没有 CMOS,就没有现代芯片**。这是大自然送给我们的礼物。

2.5 串联到下一章

我们已经有了**两种相反性格的材料**:N 型(电子主导)和 P 型(空穴主导)。

下一个自然的问题是:**如果我们把它们贴在一起,会发生什么?**

答案是:一个能"单向导电"的器件——**PN 结**,也就是二极管。再进一步把多个 PN 结串起来,就有了三极管、MOSFET——开关。

而 MOSFET,正是我们一直追求的那个**可控开关**。

🔍 本章没讲什么

- 量子力学下的能带推导(Bloch 定理、布里渊区)。
- 载流子的迁移率、漂移与扩散方程。
- 直接带隙 vs 间接带隙(为什么硅不擅长发光,而 GaAs 擅长)。
- 极端工艺下的硅:绝缘体上硅(SOI)、应变硅、FinFET 沟道。

📚 想深入读什么

- Neamen, *Semiconductor Physics and Devices* — 经典本科教材,推导清晰。
- Sze, *Physics of Semiconductor Devices* — 学完上面再看,工业级深度。
- Feynman 物理学讲义第三卷 · 第 14 章 — 用最朴素的语言讲半导体。

← 上一章:总览 · → 下一章:PN 结

第 3 章 · PN 结与二极管

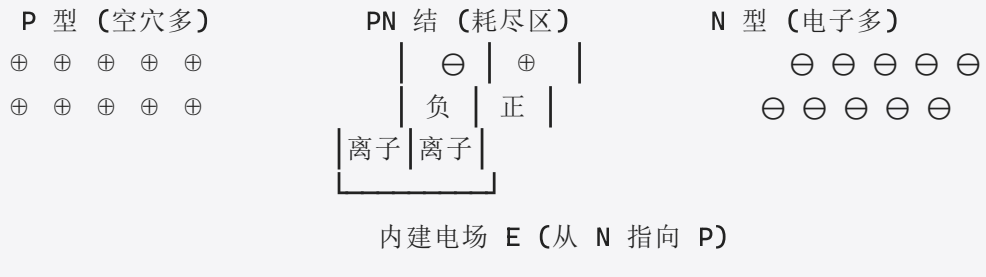
> 把 P 型硅和 N 型硅贴在一起,就得到了人类第一种电子开关。

3.1 接触瞬间发生了什么?

把 P 型和 N 型硅紧紧贴在一起的瞬间(实际工艺是在同一片硅上掺杂出两个区域):

- N 区电子很多,P 区电子很少 → 电子从 N 跑到 P
- P 区空穴很多,N 区空穴很少 → 空穴从 P 跑到 N

这叫**扩散运动**(类比:把一滴墨水滴进清水)。但请注意,**电子和空穴跑过去之后,留下的是带电的离子**:



扩散到一半,会发生一件神奇的事:**留下的离子产生了一个反方向的电场,把还想扩散的载流子推回去。**当扩散力 = 电场力时,系统达到平衡——形成一个没有自由载流子的「**耗尽区**」(Depletion Region),宽度大约几百纳米。

这个内建电场对应一个**内建电势差**,硅中约 0.7 V。这就是为什么二极管的"开启电压"是 0.7 V 的根本原因。

3.2 加电压会怎样?——单向导电的本质

正向偏置 (Forward Bias):P 接正,N 接负

外加电场方向 **与内建电场相反** → 把内建电场抵消 → 耗尽区变窄 → 载流子能轻松冲过去 → **电流大,导通。**

```
外电源 (+) ──> P [⊕ ⊕ ⊕→][←⊖⊖⊖] N <── (−)
                 耗尽区变窄,大量载流子冲过去
```

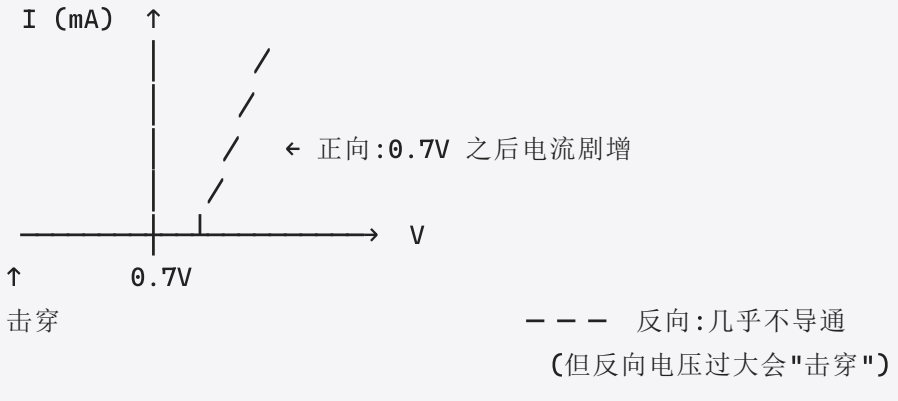
反向偏置 (Reverse Bias):P 接负,N 接正

外加电场方向 **与内建电场同向** → 加强内建电场 → 耗尽区变宽 → 载流子被往两边拉,过不去 → **电流几乎为零,截止。**

```
外电源 (−) ──> P [⊕   ←⊕][⊖→   ⊖] N <── (+)
                 耗尽区变宽,几乎没人过得去
```

这就是二极管「单向导电」的本质:不是因为材料偏心,而是因为内建电场的方向被外加电压**加强或削弱。**

3.3 二极管 I-V 曲线



数学上,二极管的 I-V 关系遵循 **Shockley 方程**:

$$ I = I_S \cdot \left(e^{V/V_T} - 1 \right) $$

其中 $V_T \approx 26\,\text{mV}$(室温下的热电压)。这是个指数关系,意味着电压稍微变化,电流变化非常剧烈——这正是它能当"开关"的原因。

3.4 二极管的应用:从整流到逻辑

二极管虽然简单,却是无数电路的基石:

应用	原理
整流(交流变直流)	只让电流朝一个方向走
检波	提取调制信号的包络
限幅 / 钳位	防止电压超限
二极管逻辑门	早期电脑用 D 和 R 搭出 AND/OR(DTL 逻辑)
LED 发光	正向偏置时电子和空穴复合,放出光子
太阳能电池	光照让 PN 结产生载流子,反过来用

3.5 串联到下一章:从二极管到三极管再到 MOSFET

二极管虽然是**自然界第一种电子开关**,但它有个致命缺陷:**没有"控制端"**。它的开关状态完全取决于流经它的两端电压,我们无法用第三只手去"指挥"它。

而做计算,需要的恰恰是**「我说开就开,我说关就关」**的开关。

于是工程师们发明了:

- **BJT(双极型晶体管)**:三只脚,基极电流控制集电极电流。但费电、不省地。
- **MOSFET(金属-氧化物-半导体场效应管)**:三只脚,栅极**电压**控制源漏导通。**几乎不费电,集成度极高。**

下一章,我们将看到 MOSFET 如何成为整个数字世界的「细胞」,以及 PMOS+NMOS 如何拼成 CMOS——一个完美的、几乎不耗静态功率的开关电路。

🔍 本章没讲什么

- PN 结的精确数学推导(连续性方程、准费米能级)。
- 雪崩击穿、隧穿击穿、肖特基势垒。
- PIN 结、变容二极管、齐纳二极管的特殊用途。

📚 想深入读什么

- Sedra & Smith, *Microelectronic Circuits* — 工程视角,直观清晰。
- Razavi, *Fundamentals of Microelectronics* — 推导严谨,例题极佳。

← 上一章:半导体物理 · → 下一章:MOSFET 与 CMOS

第 4 章 · MOSFET 与 CMOS:数字世界的开关

> 🌉 **桥 1**:这是从「物理世界」走进「逻辑世界」的第一座桥。 读完这一章,你将看到 0 和 1 是怎么从硅原子里"长"出来的。

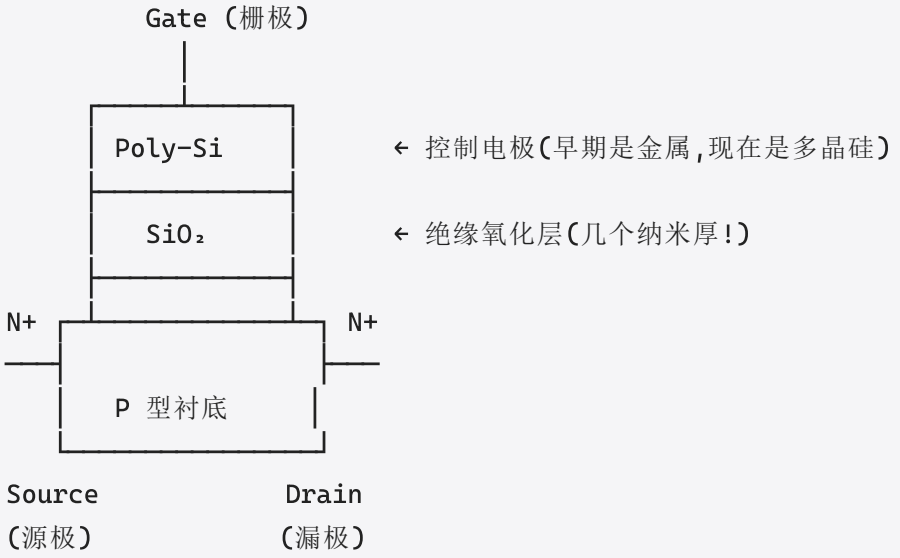
4.1 我们想要什么样的开关?

理想中的数字开关应该是这样的:

1. **有第三只手(控制端)**:用一个独立的输入信号控制开/关。
2. **控制端不耗电**:不能控制一下就花一份电。
3. **开和关分得很开**:开的时候像导线,关的时候像绝缘体。
4. **开关速度极快**:每秒几十亿次。
5. **小**:一颗芯片上要塞几十亿个。
6. **可以大量、便宜地制造。**

PN 结二极管是「单向阀门」,但没有控制端。BJT 三极管有控制端,但**控制端要耗电流。**直到 MOSFET 出现,以上六条全部满足。

4.2 NMOS:基本结构



关键观察:**栅极和沟道之间隔着一层绝缘的 SiO_2。**这意味着:**没有任何电流能从栅极流入沟道**——这正是我们想要的"控制端不耗电"。

工作原理(三句话)

1. 给栅极加一个**正电压**(超过阈值 V_{th},通常 0.3~0.7V)。
2. 正电压把 P 衬底里的电子吸引到栅极下方,形成一条 N 型沟道。
3. 沟道把源极和漏极连通 → **晶体管导通。**

电压撤掉,沟道消失,源漏断开 → **截止。**

> 🔑 **核心精髓**:用电场(而非电流)来调节沟道的导电能力。这就是 **FET = Field Effect Transistor(场效应管)** 的由来。

真值表(数字抽象)

栅极电压 V_G	沟道	NMOS 状态
低 (≈0V, 视作"0")	没有	截止(开路)
高 (≈Vdd, 视作"1")	形成	导通(短路)

4.3 PMOS:NMOS 的镜像孪生

PMOS 把所有"P"和"N"互换:N 型衬底,P+ 源漏。它的工作恰好相反:

栅极电压 V_G	PMOS 状态
高 ("1")	截止
低 ("0")	导通

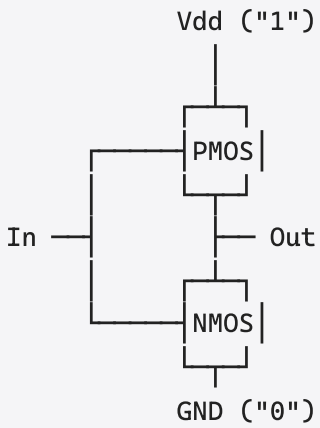
> 💡 把 NMOS 想成「看到 1 就开门」,PMOS 想成「看到 0 就开门」。

4.4 CMOS = Complementary MOS:互补的魔法

如果只用 NMOS 来搭门电路(NMOS 逻辑),会有个问题:**"输出为 0"时,有一条从 Vdd 通过电阻流到地的常态电流——非常费电。**

聪明人想到一个办法:**让 PMOS 在「上面」拉高,NMOS 在「下面」拉低,两人永远只有一个开。**这就是 **CMOS** 的核心思想——**互补(Complementary)**。

最简例:CMOS 反相器(NOT 门)



分析:

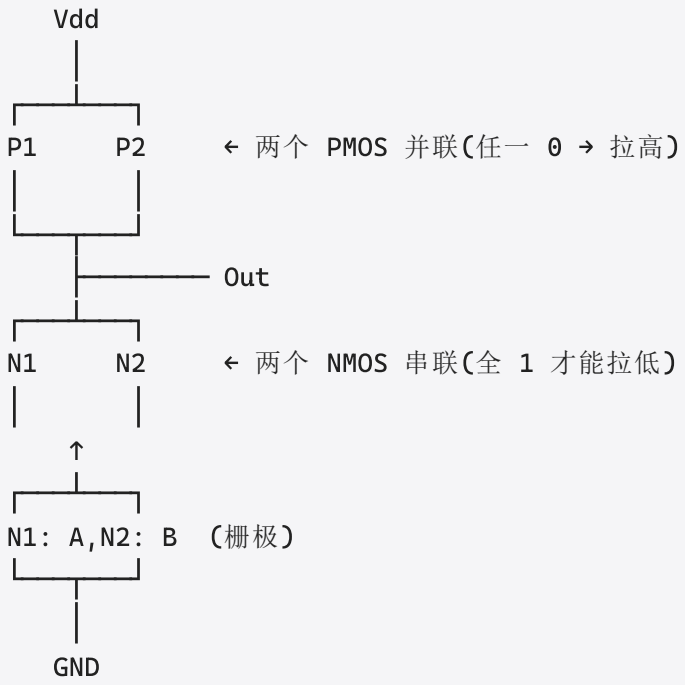
In	PMOS	NMOS	Out
0	导通	截止	**拉到 Vdd = 1**
1	截止	导通	**拉到 GND = 0**

—— **这就是一个 NOT 门!** 而且最关键的是:**稳态时永远是一个 ON 一个 OFF,没有从 Vdd 直流到 GND 的通路 → 静态功耗几乎为零!**

> 🌟 这是 CMOS 统治世界的根本原因。一颗 CPU 上几百亿个晶体管,如果每个都常态耗电,根本没法散热。CMOS 让数字电路只在**翻转的瞬间**耗能(动态功耗),而保持状态时几乎不耗能。

4.5 CMOS NAND 门:两个晶体管不够,我们用四个

CMOS 反相器只有 1 个输入。要实现 2 输入的 NAND,我们需要 4 个 MOS:



设计套路(可以套用到所有 CMOS 门):

- **上拉网络 (PMOS)**:决定什么时候输出 = 1。
- **下拉网络 (NMOS)**:决定什么时候输出 = 0。

- **互补关系**:PMOS 网络是 NMOS 网络的"对偶"——串联 ↔ 并联。

真值表验证:

A	B	N1·N2 串联	P1 ∥ P2 并联	Out
0	0	不通(任意一个截止)	都通	1
0	1	不通	P1 通	1
1	0	不通	P2 通	1
1	1	**通**	都不通	**0**

—— 完美的 NAND!

4.6 NAND 是「万能门」

数学上可以证明:**只用 NAND 门,可以搭出任何逻辑函数**(NAND 是「函数完备」的)。

```
NOT(A)        = NAND(A, A)
AND(A, B)     = NAND(NAND(A,B), NAND(A,B))
OR(A, B)      = NAND(NAND(A,A), NAND(B,B))
XOR(A, B)     = NAND(NAND(A,NAND(A,B)), NAND(B,NAND(A,B)))
```

也就是说,**只要我们能造 CMOS NAND 门,我们就能造出整个数字宇宙。**

4.7 串联到下一章:CMOS → 逻辑门 → ?

我们现在站在了"桥的对岸":

- **物理侧**:硅、掺杂、MOSFET、CMOS 反相器、CMOS NAND。
- **逻辑侧**:NOT、NAND、AND、OR、XOR...(下一章详细展开)

从这一章往后,我们将彻底**告别物理,进入逻辑世界。**我们会忘记电压、电流、沟道,只把它们抽象成 0 和 1。所有后续章节,只要还在硬件层,本质都在用 NAND 门搭东西——只是规模越来越大,从加法器到 CPU,从 KB 到 TB。

> 📌 **本书的第一道分界线**:从这一章开始,**「电压」变成「逻辑值」**。这就是抽象的力量。

🔍 本章没讲什么

- MOSFET 的精确电流方程(平方律、亚阈值导通、短沟道效应)。
- 工艺节点演进(180nm → 7nm → 3nm)与 FinFET、GAA。
- 漏电流、热载流子注入、栅氧击穿等可靠性问题。
- 模拟电路中 MOSFET 作为放大器/电流源的用法。

📚 想深入读什么

- Razavi, *Design of Analog CMOS Integrated Circuits* — 模拟视角的 MOSFET 圣经。
- Weste & Harris, *CMOS VLSI Design* — 数字 VLSI 的标准教材。
- Hodges 等, *Analysis and Design of Digital Integrated Circuits* — 经典本科教材。

← 上一章:PN 结 · → 下一章:布尔代数与逻辑门

第 5 章 · 布尔代数与逻辑门

> 上一章我们造出了 NAND。这一章,我们要把 NAND 当作"原子",忘掉电压。 整个数字世界,从这一章开始,只有 0 和 1。

5.1 抽象的飞跃

请做一个心理上的转变:

从这一章起,我们不再讨论电流、电压、晶体管。

我们只讨论 0 和 1,以及处理它们的「函数」。这种函数就是**逻辑函数**,描述它们的代数就是**布尔代数**。

> 💡 这种"忘掉下层细节,只关心接口"的思维,叫做**抽象(Abstraction)**。整本书的结构,就是一层一层往上建立抽象。**写软件的人之所以不用懂半导体物理,正是因为这条抽象链路足够稳固。**

5.2 七个基本逻辑门

门	符号	真值表(双输入)	公式
NOT	▷○	0→1, 1→0	$\bar{A}$
AND	&	全 1 才 1	$A \cdot B$
OR	≥1	任 1 即 1	$A + B$
NAND	&○	AND 取反	$\overline{A \cdot B}$
NOR	≥1○	OR 取反	$\overline{A + B}$
XOR	=1	不同为 1	$A \oplus B$
XNOR	=1○	相同为 1	$\overline{A \oplus B}$

记忆口诀: - AND = "都";OR = "或";XOR = "异"。 - 加 N(NAND/NOR/XNOR)= 上面取反 = CMOS 实现起来更省。

5.3 布尔代数:计算 0/1 的"加减乘除"

布尔代数有几个最重要的恒等式,它们是化简电路的工具:

基本律

名称	表达
交换律	$A + B = B + A$,$AB = BA$
结合律	$(A+B)+C = A+(B+C)$
分配律	$A(B+C) = AB + AC$
吸收律	$A + AB = A$
互补律	$A + \bar{A} = 1$,$A \cdot \bar{A} = 0$
德摩根律	$\overline{A + B} = \bar{A} \cdot \bar{B}$,$\overline{AB} = \bar{A} + \bar{B}$

德摩根律为什么重要?

它让我们能在「与」和「或」之间自由转换。这意味着:**任何电路都能转换成只用 NAND(或只用 NOR)实现** —— 这是 CMOS 工艺为什么偏爱 NAND/NOR 的代数根源。

例如: $$ A + B = \overline{\overline{A + B}} = \overline{\bar{A} \cdot \bar{B}} = \text{NAND}(\bar{A}, \bar{B}) $$

5.4 真值表 ↔ 布尔表达式 ↔ 电路

任何逻辑函数都可以用三种等价形式表示:

```
   真值表        ←──→       布尔表达式        ←──→       门电路
(枚举所有                        (代数                        (物理
  可能性)                        化简)                        实现)
```

例:多数表决器(3 个输入,任 2 个为 1 时输出 1)

真值表:

A	B	C	Y
0	0	0	0
0	0	1	0
0	1	0	0
0	1	1	1
1	0	0	0
1	0	1	1
1	1	0	1
1	1	1	1

布尔表达式(从真值表的 1 行直接构造,SOP 形式):

$$ Y = \bar{A}BC + A\bar{B}C + AB\bar{C} + ABC $$

化简(配对相邻项):

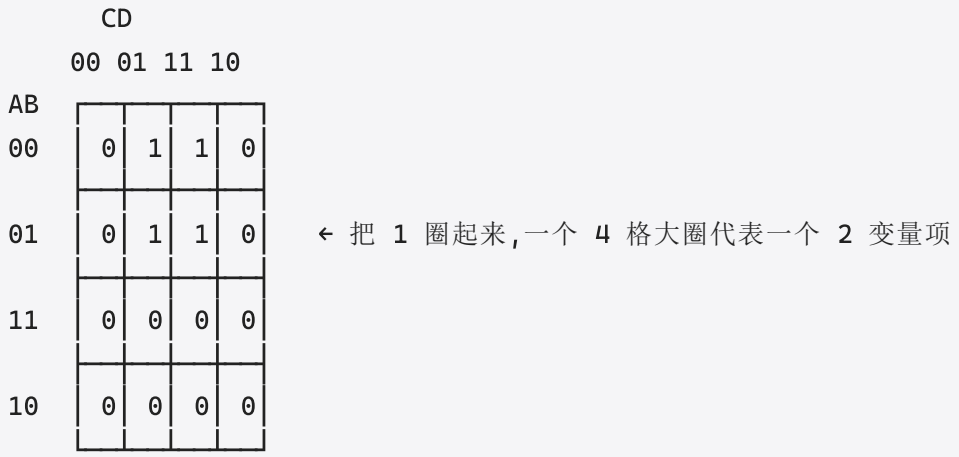
$$ Y = AB + BC + AC $$

电路实现:三个 AND 门 + 一个三输入 OR 门。

> 🔧 这个「真值表 → SOP → 化简 → 电路」的流程,本质就是**手工综合**。EDA 工具做的事情,就是把这个流程自动化和规模化。

5.5 卡诺图(K-Map):化简电路的图形方法

对于 2-4 个变量的小函数,**卡诺图**是一种非常直观的化简工具。它把真值表排成一个特殊的二维网格,**相邻格子只有一个变量不同**,然后我们用"圈相邻 1 的方块"的方法找出最简表达式。



实际工程中,卡诺图**只是教学工具**——大于 5 个变量,人就化不动了。工业界用的是 **Quine-McCluskey 算法**或**Espresso** 启发式算法,被集成在 EDA 工具中。

5.6 一个深刻的事实:NAND 是函数完备的

我们已经在上一章说过:**只用 NAND,就能搭出任何逻辑函数。**

这件事的伟大之处在于:**它把硬件设计的难题,从"造很多种门"变成了"造很多个 NAND"。**

工艺工程师只需把一种器件(CMOS NAND)做到极致——成本下降、密度上升、功耗下降——而所有其他逻辑功能,都由数字工程师在更高层用 NAND 拼出来。

> 这是工程史上最重要的"分工"之一,几乎可以类比成软件中的「图灵机」——一台简单到极点的机器,被证明能计算任何可计算的东西。

5.7 串联到下一章

我们现在拥有了**任意复杂的"瞬间运算"能力**:给定输入 0/1,立即产生输出 0/1。这种"输出只取决于当前输入"的电路,叫**组合逻辑电路**(Combinational Logic)。

下一章,我们将看到组合逻辑如何变出:

- **加法器**(让 CPU 能算数)
- **译码器**(让指令能选地址)
- **多路选择器**(让数据流能被控制)

但组合逻辑有个根本局限:**它没有"记忆"**。要让一个电路能存住"上次发生了什么"——也就是有**状态**——我们还需要时序电路(Ch 7)。

🔍 本章没讲什么

- 多输出函数的化简(Quine-McCluskey)。
- 关心项 / 不关心项(Don't Care)的处理。
- 实际门电路的延迟、扇入扇出、毛刺(glitch)。

📚 想深入读什么

- Mano & Ciletti, *Digital Design* — 经典本科教材。
- Patterson & Hennessy, *Computer Organization and Design* — Appendix A 复习数字逻辑。

← 上一章:CMOS · → 下一章:组合逻辑

第 6 章 · 组合逻辑:加法器、译码器、多路选择器

> 用 NAND 门,我们造一个能算 1+1=2 的电路。 然后用同样的方法,造译码器、MUX —— CPU 内部所有"瞬时运算"模块都从这一章开始。

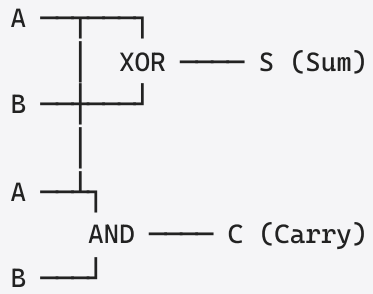
6.1 半加器(Half Adder):从最简单的加法说起

二进制 1 位加法的真值表:

A	B	Sum (S)	Carry (C)
0	0	0	0
0	1	1	0
1	0	1	0
1	1	0	1

观察:

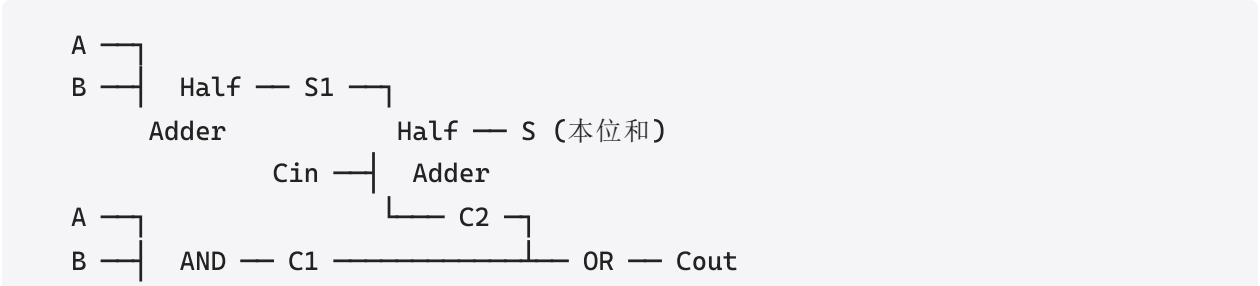
- $S = A \oplus B$(XOR!)
- $C = A \cdot B$(AND!)



这就是「半加器」。 它能算 1 位加法,但缺一个东西:**接收前一位送来的进位。**

6.2 全加器(Full Adder):有了进位输入

把"加来自上一位的进位 C_{in}"也算进来:

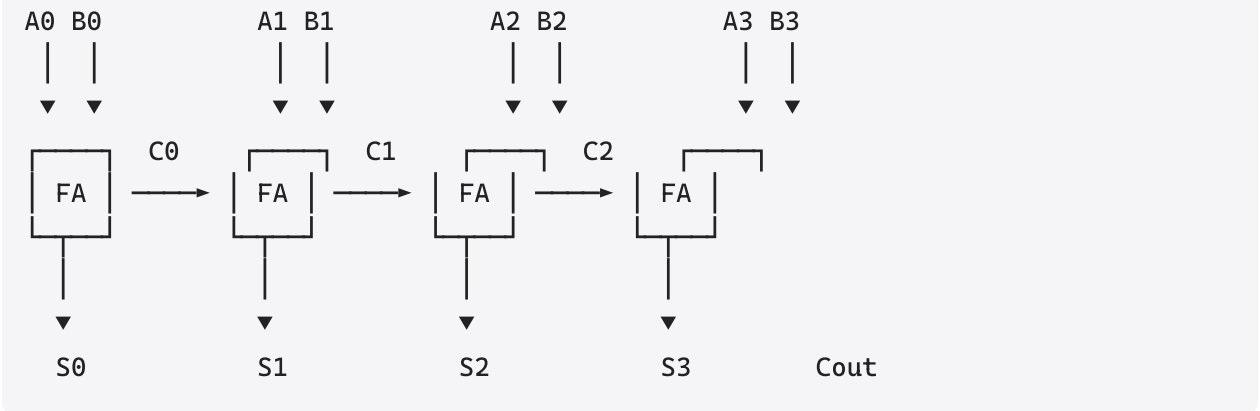


公式: $$ S = A \oplus B \oplus C_{in} $$ $$ C_{out} = AB + (A \oplus B) \cdot C_{in} $$

每个全加器:**输入 3 比特,输出 2 比特。**

6.3 4 位行波进位加法器(Ripple Carry Adder)

把 4 个全加器串起来,**每一位的进位输出接到下一位的进位输入**:



> ⚠️ **缺陷**:进位是「一位一位往后传」的(像水波),所以延迟随位宽线性增长。32 位加法器要等 32 个门延迟。
>
> 改进方案:**超前进位加法器(Carry-Lookahead Adder)**——通过并行计算所有进位,把延迟降到 $O(\log n)$。这是 CPU 实际使用的加法器结构。

6.4 译码器(Decoder):n 个输入选 1 个输出

2-to-4 译码器:输入 2 位选择信号,4 个输出中只有一个为 1。

A1	A0	Y3	Y2	Y1	Y0
0	0	0	0	0	1
0	1	0	0	1	0
1	0	0	1	0	0
1	1	1	0	0	0

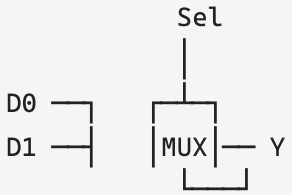
公式: - $Y_0 = \bar{A_1} \bar{A_0}$ - $Y_1 = \bar{A_1} A_0$ - $Y_2 = A_1 \bar{A_0}$ - $Y_3 = A_1 A_0$

译码器的核心用途:地址 → 选片信号。

例如内存有 1024 字节,需要 10 位地址。一个 10-to-1024 译码器把这 10 位地址变成 1024 根选通线,**正好选中那一字节的存储单元。**CPU 寻址、寄存器读写、指令译码,全部依赖译码器。

6.5 多路选择器(MUX):n 个输入选 1 个

2-to-1 MUX:有两个数据输入 D0、D1,一个选择信号 S,一个输出 Y。




```
Y = (S = 0) ? D0 : D1
公式:Y = S̄·D0 + S·D1
```

4-to-1 MUX:用 2 位选择信号 S1,S0 在 4 个输入中挑一个。

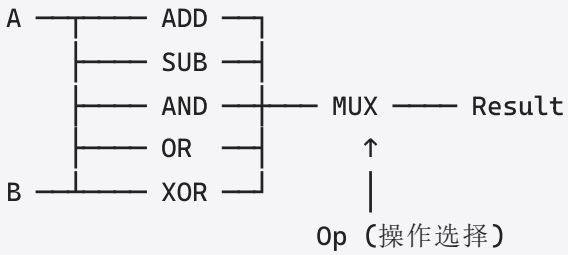
> 💡 **MUX 是 if-else 的硬件版本。**CPU 里所有"分支选择数据"的地方,本质上都是一个或多个 MUX。下一章我们做 ALU 时会大量用到它。

MUX 的伟大事实:它也是函数完备的

任何逻辑函数都能只用 MUX 实现(查找真值表的任意一行,等价于在 MUX 的输入端写常数)。这个性质让现代 FPGA 用**查找表(LUT)**作为基本单元——本质上就是一个超大的 MUX。

6.6 ALU 雏形:把加法器、AND、OR、MUX 拼起来

CPU 中负责算术和逻辑运算的核心模块叫 **ALU(Arithmetic Logic Unit)**。最简单的 ALU 这么搭:



- 4 个并行的运算单元同时计算结果。
- MUX 根据控制信号 `Op` 挑出我们想要的那个。

> 🔑 **关键观察**:数字硬件不像软件,**它总是把所有可能的结果都算出来**(因为门电路一直在跑),然后用 MUX 选一个。这是软硬件思维方式的根本差异之一。

6.7 串联到下一章

我们现在能造任何**组合逻辑模块**——加法、比较、译码、选择……一切"瞬时计算"。

但有个根本问题:**这些电路没有记忆。**

- 给它一组输入,输出立刻出现;输入一变,输出马上变。

- 我们没法存住"上一秒的结果"。
- 没有记忆,就无法做迭代、循环、状态机——也就无法做出真正的"计算机"。

下一章,我们将引入**时序逻辑**:用反馈环把电路变成有记忆的——锁存器、触发器、寄存器,以及由它们构成的**有限状态机(FSM)**。一旦有了状态,我们就有了**程序计数器、寄存器堆**,CPU 的雏形就要出现了。

🔍 本章没讲什么

- 超前进位、Wallace 树、Booth 乘法器等高速算术电路。
- 编码器、优先编码器、桶形移位器(barrel shifter)。
- 组合逻辑的延迟分析与时序裕度。

📚 想深入读什么

- Mano, *Digital Design* — 第 4 章组合逻辑。
- Harris & Harris, *Digital Design and Computer Architecture* — 把组合逻辑直接连到 CPU 设计。

← 上一章:逻辑门 · → 下一章:时序逻辑

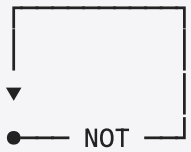
第 7 章 · 时序逻辑:锁存器、触发器、有限状态机

> 给电路加上"记忆",它就从计算器变成了计算机。

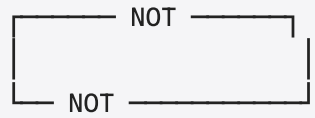
7.1 什么是「记忆」?——反馈的魔法

组合电路输出只取决于当前输入。要让电路记住"过去发生了什么",我们需要把**输出的一部分,接回输入**——这叫**反馈(Feedback)**。

最简单的例子:



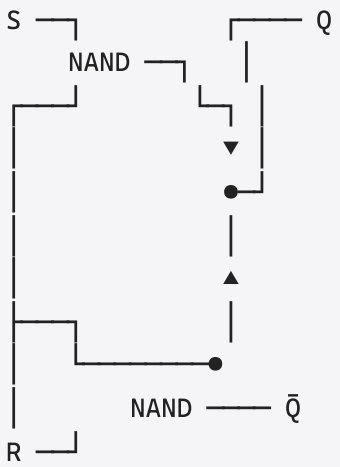
一个非门的输出接回它自己的输入——这是个不稳定振荡器(永远在 0、1 之间跳)。但如果我们让两个非门首尾相接:



—— 它就会**稳定**!而且根据它当前的状态,**有两种稳定模式**:左边 0、右边 1,或者左边 1、右边 0。**这就是 1 比特记忆的物理原型。**

7.2 SR 锁存器(SR Latch):最早的记忆单元

用两个 NAND 门交叉耦合:

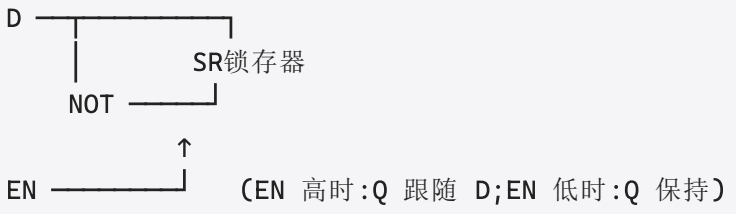


S	R	Q (新)	Q̄ (新)	含义
0	0	**不变**	**不变**	**保持** ← 这就是"记忆"!
0	1	0	1	复位(Reset)
1	0	1	0	置位(Set)
1	1	(禁用)	—	不允许

> 🔑 SR 锁存器是**最简单的"1 比特存储器"**。给它发一个 Set,它记住 1;发一个 Reset,它记住 0;不发就保持。**整个计算机的所有寄存器、Cache、SRAM,本质都是它的变体。**

7.3 D 锁存器:更友好的接口

SR 锁存器有两个输入,容易用错。D 锁存器只有一个数据输入 D 和一个使能信号 EN:



行为: - EN = 1: `Q = D` (透明,数据"流过") - EN = 0: `Q` 保持上一次的值

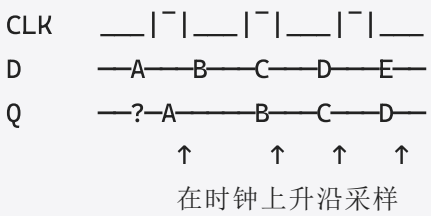
> ⚠️ **锁存器的问题:电平触发。**只要 EN 是高电平,Q 就一直跟着 D 变。这在实际系统中会引起信号竞争和不确定性。

7.4 D 触发器(D Flip-Flop):时钟边沿触发

把两个 D 锁存器**主-从串联**,前一个用 CLK,后一个用 ¬CLK:

```
D ──► [Master Latch] ──► [Slave Latch] ──► Q
         EN=CLK             EN=¬CLK
```

只有在时钟从 0 跳到 1 的瞬间(上升沿),数据才会进来一次。其余时间,Q 保持不变。



> 🌟 **D 触发器是整个数字系统的"时间原子"。**所有同步电路 —— CPU、GPU、内存控制器 —— 都靠它在时钟边沿"咔哒"地推进状态。这就是「同步设计」的灵魂。

7.5 寄存器:多个 D 触发器并排

把 32 个 D 触发器并联,共享同一个时钟,就得到一个 **32 位寄存器**:

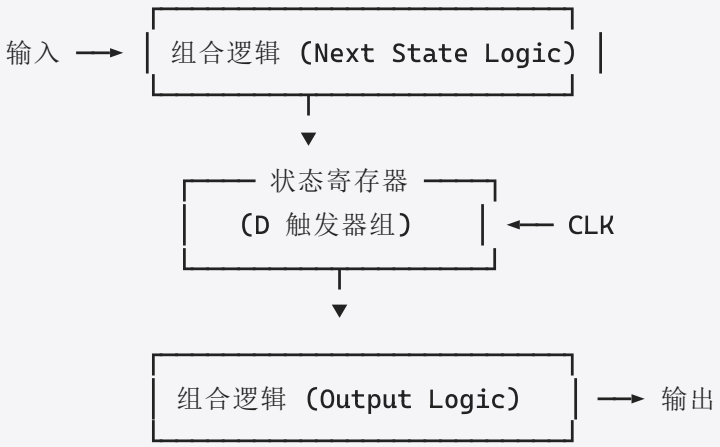


CPU 的 PC、x0~x31 通用寄存器、状态寄存器,**全是这个东西**——只是数量、连接关系不同。

寄存器堆(Register File):把多个寄存器集成到一起,加上读端口和写端口,就是 CPU 内部那块「最快的内存」。

7.6 有限状态机(FSM):时序电路的"程序"

有了状态(寄存器),再加上组合逻辑(决定下一个状态),我们得到了一个能"按步骤工作"的电路——**有限状态机(Finite State Machine, FSM)**。



每一个时钟边沿,FSM: 1. 看当前状态 + 输入, 2. 通过组合逻辑算出"下一个状态", 3. 把新状态存进寄存器。

例子:红绿灯控制器

状态:`{RED, GREEN, YELLOW}`。

当前状态	输入(定时器到)	下一状态
RED	是	GREEN
GREEN	是	YELLOW
YELLOW	是	RED

3 个状态用 2 比特编码,加一些组合逻辑选下一状态,加一些组合逻辑输出灯的颜色——FSM 就完成了。

> 💡 **CPU 的控制器,本质上就是一个巨大的 FSM。**当它处于"取指阶段",它会在下一个周期跳到"译码阶段";处在"译码阶段",会跳到"执行阶段"……这正是 5 级流水线的状态划分。

7.7 同步系统的根本契约:Setup / Hold

D 触发器有两个关键时序约束:

- **建立时间(Setup time, t_{su})**:数据必须在时钟边沿**之前** t_{su} 就稳定。
- **保持时间(Hold time, t_h)**:数据必须在时钟边沿**之后**再保持 t_h。

如果违反了这两个,触发器会进入「亚稳态」——既不是 0,也不是 1,而是一个慢慢恢复的中间值。亚稳态会传播,可能导致整个系统崩溃。

> 这是 **EDA 静态时序分析(Static Timing Analysis, STA)** 的全部目的:确保整个芯片上,每一对触发器之间的延迟,都满足 setup/hold。

7.8 串联到下一章

我们现在已经具备了搭建 CPU 的**全部硬件零件**:

- ✅ 组合逻辑 → 加法、比较、选择(ALU)
- ✅ 时序逻辑 → 寄存器、状态机(PC、控制器、寄存器堆)

- ✅ 时钟驱动 → 同步推进每一个周期

但是 —— **如果让我们用纸笔画门和触发器去画一个 32 位 CPU,大概需要画几万张图。**这显然不可行。

下一章我们将看到工程师怎么解决这个问题:**用一种「类似编程语言」的方式,描述电路。**这种语言叫 **Verilog**(以及它的兄弟 VHDL)。然后通过 **EDA 综合**,把代码自动翻译回门和触发器——也就是「网表」。

这正是 Part III 的主题:**HDL 与 EDA 流程。**

🔍 本章没讲什么

- JK、T 触发器(在工业界已被 D 触发器全面取代)。
- 异步 FSM、Mealy vs Moore 机的微妙区别。
- 时钟域跨越(CDC)与亚稳态恢复电路。
- 工业级 SRAM/DRAM 单元的物理实现。

📚 想深入读什么

- Mano, *Digital Design* — 第 5-6 章时序逻辑。
- Harris & Harris, *Digital Design and Computer Architecture* — 把 FSM 直接连到 CPU 控制器。

← 上一章:组合逻辑 · → 下一章:Verilog 入门

第 8 章 · Verilog 入门:用代码描述硬件

> 当门电路多到几亿个,我们需要一种"语言"来代替画图。 这种语言,长得像 C,但**它不是被执行的,而是被翻译成硬件的。**

8.1 Verilog 是什么?——它不是编程语言

最常见的误解之一:**「Verilog 是一种编程语言」**。

更准确的说法是:

> **Verilog 是一种硬件描述语言(Hardware Description Language, HDL)。它的每一条语句最终都会变成一些门电路 / 触发器 / 连线。**

我们写 Verilog 不是为了"运行"它,而是为了**描述一份硬件设计图纸。**EDA 工具(下一章详述)读这份图纸,把它综合成网表 → 布局布线 → 流片成芯片。

> ⚠️ **写 Verilog 时,脑子里要画电路,不要画流程图。**如果你以"软件思维"写 Verilog,综合工具会生成又大又慢的电路,甚至**根本综合不出来。**

8.2 第一个例子:2 选 1 多路选择器

```
module mux2to1 (
    input  wire       sel,
    input  wire [7:0] a,
    input  wire [7:0] b,
    output wire [7:0] y
);
    assign y = sel ? b : a;
endmodule
```

逐行解释:

- `module ... endmodule`:模块的边界,等价于电路的"封装"。
- `input wire`、`output wire`:端口声明。`wire` 表示"导线"。
- `[7:0]`:8 位宽的总线。
- `assign`:**连续赋值**,描述组合逻辑。`y` 永远等于 `sel ? b : a`。

综合后,这段代码会变成 **8 个 2-to-1 MUX**(每位一个)。

8.3 D 触发器:学会写时序逻辑

```
module dff (
    input  wire clk,
    input  wire rst_n,
    input  wire d,
    output reg  q
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            q ≤ 1'b0;
        else
            q ≤ d;
    end
endmodule
```

关键点:

- `always @(posedge clk ...)`:**告诉综合工具"在时钟上升沿采样"**。这就是它会被综合成 D 触发器的原因。
- `output reg q`:`reg` 不是"寄存器",而是"在 always 块里被赋值的变量"。它**可能**变成触发器,也**可能**只是组合逻辑变量,取决于敏感列表。
- `≤`:**非阻塞赋值**,用于时序逻辑。所有 `≤` 在一个时钟边沿同时发生,模拟硬件的并行性。
- `=`:**阻塞赋值**,用于组合逻辑。

> ⚠️ **Verilog 最容易踩的坑**:在 `always @(posedge clk)` 里用 `=`,或者在组合 `always @(*)` 里用 `≤`。两者都会导致仿真和综合不一致。

铁律:

> *时序逻辑用 `≤`,组合逻辑用 `=`。*

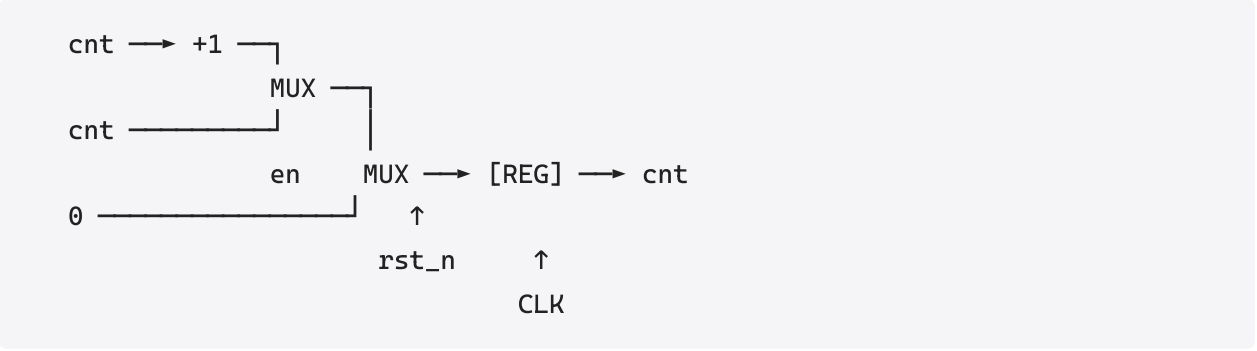
8.4 把组合 + 时序拼起来:4 位计数器

```
module counter4 (
    input  wire       clk,
    input  wire       rst_n,
    input  wire       en,
    output reg  [3:0] cnt
);
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n)
            cnt ≤ 4'd0;
        else if (en)
            cnt ≤ cnt + 4'd1;
    end
endmodule
```

这段代码综合后是什么?

- 一个 4 位寄存器(4 个 D 触发器)
- 一个 4 位加法器(产生 `cnt + 1`)

- 一个 MUX(根据 `en` 选择"加 1 后的值"还是"原值")
- 一个 MUX(根据 `rst_n` 选择"0"还是"上面的结果")



写代码 → 心里画电路 → 验证综合结果,这是 IC 设计的基本功。

8.5 Verilog 的"两个世界":可综合 vs 仿真

这是初学者最容易混乱的一点:

类型	特点	例子
可综合代码	综合工具能映射到真实硬件	`assign`、`always @(posedge clk)`、`if-else`
仅仿真代码	只用来在仿真器里测试	`$display`、`#10`、`initial`、`fork-join`

`testbench`(测试台)只用仿真特性,不会被综合。**综合工具会忽略所有仿真专用语句。**

一个简单的 Testbench

```
module tb_counter4;
    reg clk = 0, rst_n = 0, en = 0;
    wire [3:0] cnt;

    counter4 uut (.clk(clk), .rst_n(rst_n), .en(en), .cnt(cnt));

    always #5 clk = ~clk;          // 100 MHz 时钟

    initial begin
        $dumpfile("counter.vcd");
        $dumpvars(0, tb_counter4);
        #20 rst_n = 1;
        #10 en = 1;
        #200 $finish;
    end
endmodule
```

运行(用免费的 Icarus Verilog):

```
iverilog -o counter.vvp counter4.v tb_counter4.v
vvp counter.vvp                  # 生成 .vcd 波形文件
gtkwave counter.vcd              # 看波形
```

8.6 Verilog vs SystemVerilog vs VHDL

- **Verilog (1984)**:语法像 C,简洁但有不少陷阱。本书主要用 Verilog。
- **SystemVerilog (2005)**:Verilog 的"加强版",增加了 `logic`、`always_ff`、`always_comb`、接口、断言等。**强烈推荐**用于新设计。
- **VHDL (1987)**:语法像 Ada,更严格更啰嗦,欧洲、军工、航天用得多。

> 💡 在工业界,**Verilog/SystemVerilog 占主流(尤其北美和亚洲)**。

8.7 一个微妙但深刻的点:Verilog 是「时间空间」的

C 程序员看 Verilog,常感到困惑:

```
// C: 顺序执行
a = 1;
b = a + 1;      // 这一行 a 已经是 1
```

```
// Verilog (always @* 组合逻辑里):
a = 1;
b = a + 1;      // 这一行也认为 a = 1,因为是 = (阻塞)

// Verilog (always @(posedge clk) 时序逻辑里):
a ≤ 1;
b ≤ a + 1;     // ⚠️ 这一行 a 还是"上一周期的 a"!
```

原因:`≤` 描述的是"当前时刻拍下输入,下一时刻输出"——所有的 `≤` 右边的值都是"当前周期"的值,而左边的更新发生在"周期切换瞬间"。这是为了**模拟所有触发器并行更新的物理现实。**

> 🔑 这就是为什么 Verilog 的执行模型很难用"程序顺序"理解 —— 它描述的是**电路在每个时钟周期发生的事,而不是 CPU 上一行一行执行的指令。**

8.8 串联到下一章

我们能用 Verilog 描述电路了,但还有一个根本问题:

> **当我写下 `cnt ≤ cnt + 1`,工具是怎么把这一行"翻译"成具体的门和触发器的?**

这就是 **EDA 综合(Synthesis)** 要做的事情——它就像一个"硬件世界的编译器",把 RTL(寄存器传输级)代码翻译成门级网表。

下一章我们会看完整的 EDA 流程:**Synthesis → Floorplan → P&R → STA → DRC/LVS → Tapeout**,理解一颗芯片是怎么从代码一步步变成可以光刻的版图的。

🔍 本章没讲什么

- Verilog 完整语法(参数、generate、`function` / `task`)。
- SystemVerilog 的高级特性(class、覆盖率、UVM)。
- 形式化验证、SVA 断言。

📚 想深入读什么

- Samir Palnitkar, *Verilog HDL: A Guide to Digital Design and Synthesis* — 经典入门。
- Stuart Sutherland, *SystemVerilog for Design* — 新设计就用 SV。
- 在线练习:HDLBits。

← 上一章:时序逻辑 · → 下一章:EDA 流程

第 9 章 · EDA 流程:综合、布局布线、签核

> 「写 RTL 是设计;EDA 工具把它变成芯片。」 这一章看一颗芯片从代码到流片的完整生命周期。

9.1 EDA 是什么?

EDA = Electronic Design Automation(电子设计自动化)。

它是一组软件工具,把人写的 RTL 代码(几千几万行 Verilog)**自动翻译成可以送进光刻厂的 GDS 版图。**

工业界三巨头(俗称 "Big 3"):

- **Synopsys** —— Design Compiler、PrimeTime、IC Compiler、VCS……
- **Cadence** —— Genus、Innovus、Tempus、Xcelium……
- **Siemens EDA(原 Mentor)** —— Calibre、Questa、Modus……

> 💡 EDA 工具是芯片产业的"无形基础设施"——一颗几亿门的芯片,人类不可能手工设计每一个连线,**全部依赖 EDA 工具自动化。**

9.2 全流程鸟瞰

一颗 ASIC 从 RTL 到流片,要经历以下大步骤:

```
RTL (Verilog)
   ↓
① 仿真 (Simulation)          ← 验证功能正确
   ↓
② 综合 (Synthesis)           ← RTL → 门级网表
   ↓
③ 形式验证 (Equivalence Check)
   ↓
④ 布局规划 (Floorplan)       ← 决定模块在芯片上的位置
   ↓
⑤ 布局 (Placement)           ← 把每个标准单元放下来
   ↓
⑥ 时钟树综合 (CTS)           ← 让时钟到每个寄存器同步
   ↓
⑦ 布线 (Routing)             ← 走金属线把所有点连起来
   ↓
⑧ 静态时序分析 (STA)         ← 检查每条路径是否满足时序
   ↓
⑨ 物理验证 (DRC + LVS)       ← 检查版图是否符合工艺规则
   ↓
⑩ 流片 (Tapeout) → 光刻 → 封装 → 测试
```

我们把这条流水线分成两半:

- **前端(Front-End)**:① ~ ③,**逻辑层面的事。**设计工程师的领地。
- **后端(Back-End)**:④ ~ ⑨,**物理层面的事。**物理设计工程师的领地。

9.3 ① 仿真:验证功能,不变成硬件

仿真器(VCS、Xcelium、Verilator、Icarus)在 PC 上**「假装跑」一段 Verilog**,产生波形(VCD/FSDB),让工程师确认电路功能正确。

> ⚠️ **仿真通过 ≠ 硬件正确。**仿真只覆盖你写的测试用例,真实硬件可能在你没想到的输入下出错。**所以才有「验证工程师」这个职业,以及 UVM、形式化验证等手段。**

9.4 ② 综合(Synthesis):RTL 的「编译器」

综合工具(Synopsys DC、Cadence Genus、开源 Yosys)做的事情:

```
Verilog (RTL)  +  工艺库 (.lib)  +  约束文件 (.sdc)
                          ↓
                  [ 综合工具 ]
                          ↓
              门级网表 (.v) + 时序报告
```

核心步骤:

1. **解析(Parse)**:把 Verilog 转成内部数据结构(类似 AST)。
2. **HDL → 通用逻辑**:把 `if-else`、`case`、`+` 等翻译成与或非门、加法器等"通用单元"。
3. **优化**:布尔函数化简、共享子表达式、常量传播。
4. **工艺映射(Technology Mapping)**:**用工艺库里实际有的标准单元(NAND2_X1、DFF_X2 等)替换通用单元**,选最快/最小的那一种,满足约束。
5. **输出**:门级网表(本质是 Verilog,但只剩 `module` 和 `instance`,没有 `always` 这种行为代码)。

🔑 综合的本质

综合 = 编程语言中的「编译」。

软件世界	硬件世界
C 源码 → 汇编	Verilog → 网表
编译器 (GCC)	综合工具 (DC)
目标 ISA(如 x86)	工艺库(如 28nm tcbn28hpcplus)
优化(O2、O3)	时序优化、面积优化

EDA 综合就是「**硬件世界的 GCC**」。理解了这个类比,就理解了为什么这一步如此关键。

例:综合一段 Verilog

输入(RTL):

```
assign y = a & b | c;
```

综合后(门级网表,简化):

```
AND2_X1  U1 ( .A(a), .B(b), .Z(n1) );
OR2_X1   U2 ( .A(n1), .B(c), .Z(y) );
```

—— 你看到的 RTL 抽象的"&"和"|",变成了具体的门实例。

9.5 ③ 形式验证:综合后还和 RTL 等价吗?

综合工具优化时可能出 bug,所以工业界会跑**逻辑等价性检查(LEC)**,数学上证明"综合前 RTL"和"综合后网表"在所有输入下行为完全相同。常用工具:Synopsys Formality、Cadence Conformal。

9.6 ④-⑦ 物理设计:把网表"摆"到硅上

网表只是一堆"门和连接关系",还没有"在哪里"的概念。物理设计就是把它们映射到 2D 平面上。

- **Floorplan(布局规划)**:决定每个大模块(CPU 核、Cache、IO)的位置和形状。
- **Placement(布局)**:把每个标准单元(几千几百万个)放到具体行/列上。
- **CTS(时钟树综合)**:时钟必须同时到达所有寄存器(skew 越小越好),所以会插一棵平衡的时钟缓冲器树。
- **Routing(布线)**:在多层金属(M1~M10)中走线,连接所有引脚。**这是最耗时的一步,可能跑几天。**

9.7 ⑧ 静态时序分析(STA):时序的"体检"

STA 工具(PrimeTime、Tempus)检查芯片上**所有寄存器对之间**的路径,验证:

- **Setup**:数据是否在时钟边沿前 t_{su} 到达?
- **Hold**:数据是否在时钟边沿后还能保持 t_h?
- **时钟偏斜(Skew)**、**抖动(Jitter)**

如果发现违例(violation),要回头修改 floorplan、加缓冲器、甚至改 RTL。**这是后端最折磨人的环节。**

9.8 ⑨ DRC / LVS:版图的"体检"

- **DRC(Design Rule Check)**:检查版图是否符合工艺厂商的几何规则(线宽、间距、密度……)。
- **LVS(Layout vs. Schematic)**:检查最终版图的连接关系**和网表完全一致。**

通过这两关,版图(GDSII 文件)才能交给晶圆厂(TSMC、Samsung、Intel Foundry……)流片。

9.9 EDA = 硬件世界的"软件栈"

回头看这条流程,你会发现它有非常强的**编译器/工具链**类比:

软件工具链	EDA 工具链
词法/语法分析	RTL 解析
中间表示(IR)	通用逻辑(GTECH)
优化(passes)	综合优化
寄存器分配	工艺映射
链接(linker)	物理设计 / 全局布线
反汇编/调试	网表查看、波形

> 💡 这就是为什么很多前 EDA 工程师能转去做编译器(LLVM、MLIR),反之亦然 —— **它们解决的本质上是同一类问题:如何把高层描述自动转成低层实现。**

9.10 串联到下一章

EDA 工具的输出之一是 **网表(Netlist)**——一个"只有门和连线"的 Verilog 文件。

下一章,我们专门拆解网表:**它是什么,长什么样,为什么我们说它是"门级世界的汇编语言"**。这个类比会让你看到 EDA 与软件编译器之间惊人的对偶性。

🔍 本章没讲什么

- DFT(Design for Test)、扫描链、ATPG。
- 低功耗设计:UPF/CPF、电压域、Power Gating。
- 模拟/混合信号 EDA(SPICE、Virtuoso)。
- FPGA 流程(综合 + 布局布线 + 比特流生成)与 ASIC 的差异。

📚 想深入读什么

- Bhasker, *A Verilog HDL Primer* + *Static Timing Analysis for Nanometer Designs*。
- Synopsys 的 *Design Compiler User Guide*(注册可下载)。
- Naveed Sherwani, *Algorithms for VLSI Physical Design Automation* — 物理设计经典。

← 上一章:Verilog · → 下一章:网表

第 10 章 · 网表 (Netlist):门级世界的「汇编语言」

> 🌉 这是「桥 2」的另一端。 你写 Verilog,EDA 综合给你网表;就像你写 C,GCC 给你汇编。

10.1 什么是网表?

网表(Netlist)= 一份只描述"门 + 连线"的电路清单。

它通常仍然是合法的 Verilog 文件,但**不再有任何"行为级"语句**(`always`、`if`、`case` 全没了)。它只剩两种东西:

1. **实例(Instance)**:一个具体的标准单元,例如 `NAND2_X1 U123 (.A(...), .B(...), .Z(...));`
2. **连线(Wire)**:把这些实例的端口连起来。

10.2 一个最小例子:从 RTL 到网表

输入:RTL 代码

```
// my_logic.v
module my_logic (
    input  wire a, b, c,
    output wire y
);
    assign y = (a & b) | c;
endmodule
```

综合后:网表

```
// my_logic_synth.v -- 综合后的门级网表
module my_logic ( a, b, c, y );
    input  a, b, c;
    output y;

    wire   n1;

    AND2_X1  U1 ( .A1(a), .A2(b), .ZN(n1) );
    OR2_X1   U2 ( .A1(n1), .A2(c),  .ZN(y) );
endmodule
```

对比观察:

RTL	网表
`assign y = (a & b) | c;`	两个 instance + 一根中间线
抽象的运算符 `&`, `|`	具体的标准单元 `AND2_X1`、`OR2_X1`
无延迟、无面积概念	每个 cell 都有真实的延迟和面积

> 💡 **这正像 C 编译成汇编**——`a + b` 变成了具体的 `add eax, ebx`,操作的是真实的寄存器。

10.3 一个略大的例子:D 触发器

RTL:

```
always @(posedge clk) q ≤ d;
```

网表(简化):

```
DFF_X1 U_FF (
    .D(d),
    .CK(clk),
    .Q(q),
    .QN()       // 反相输出,这里没用,悬空
);
```

—— 综合工具直接选了一个标准单元 `DFF_X1`(工艺库里预先制造好的 D 触发器版图)。

10.4 标准单元库(Standard Cell Library)

工艺库(`.lib` / `.db` / `.lef` 文件)里包含上千种"乐高积木"。一些常见的:

单元名	功能
`INV_X1` 、 `INV_X2` 、 `INV_X4` 、 `INV_X8`	反相器,X 后的数字代表驱动能力
`NAND2_X1` 、 `NAND3_X1`	2 / 3 输入 NAND
`NOR2_X1` 、 `NOR3_X1`	NOR 系列
`AOI21_X1`	$\overline{A \cdot B + C}$,常用作复合逻辑
`OAI21_X1`	$\overline{(A + B) \cdot C}$
`MUX2_X1`	2 选 1 MUX
`XOR2_X1`	异或
`DFF_X1` 、 `DFFR_X1`	普通 D 触发器、带异步复位的 D 触发器
`LATCH_X1`	锁存器

每个单元在 `.lib` 中都标注了: - 输入引脚电容 - 输出驱动能力 - 输入到输出的延迟(随负载、温度变化的查找表) - 面积、漏电、动态功耗

> 🔑 **综合的本质 = 在这上千种乐高里挑一组,组合起来"既能实现你写的 RTL 功能,又满足时序、面积、功耗"。**

10.5 网表 = 硬件世界的「汇编语言」

下面这张对比表,是本章最值得记住的:

软件世界	硬件世界
C 源码	Verilog RTL
汇编代码	门级网表
机器码(0/1)	物理版图 GDSII
ISA(x86 指令集)	工艺库的标准单元集
编译器 (GCC/Clang)	综合工具 (DC/Genus)
寄存器分配	工艺映射
链接器	物理设计
加载器(把可执行文件放进内存)	流片(把版图刻到硅上)
`objdump -d` 看汇编	综合后看网表 / GUI 看 schematic

这是一个完美的对偶。理解它,你就能把"硬件设计"理解成"用一种特殊编译器,把 Verilog 编译到一个特殊的'指令集'(标准单元库)上"。

10.6 看一下 Yosys 实操(开源)

Yosys 是一个免费的开源综合工具。装上以后:

```
yosys -p "read_verilog my_logic.v; synth; write_verilog -noattr my_logic_synth.v
```

输出会是一份网表,虽然没有商业工艺库的标准单元名,但可以看到:

```
$_AND_  _0_ (.A(a), .B(b), .Y(_1_));
$_OR_   _2_ (.A(_1_), .B(c), .Y(y));
```

`$_AND_`、`$_OR_` 是 Yosys 的"通用门",真实流片时会被替换成工艺库里的具体单元。但**结构完全相同**——这就是网表。

10.7 网表上能做什么?(为什么它重要)

工业界很多关键任务**直接在网表上做**:

- **STA(静态时序分析)**:给每条门级路径算延迟。
- **功耗分析**:基于活动率(toggle rate)算每个 cell 的功耗。
- **DFT(可测性设计)**:在触发器上挂扫描链。
- **ECO(Engineering Change Order)**:发现 bug 后,直接在网表上手术式修改,而不是重综合。
- **形式验证**:对网表与 RTL 做等价性证明。

10.8 串联到下一章:网表如何变成 CPU?

到这一章为止,我们打通了:

```
CMOS → 门 → 组合/时序 → Verilog → 网表
```

整个芯片的硬件层至此完整闭环。

但我们还没有看过任何**完整的处理器**——CPU 是怎么从这些 NAND/MUX/DFF 里组装出来的?它怎么读"指令"?为什么它能"流水线"地工作?

下一部分(Part IV)我们就跨进这扇门:**计算机体系结构**——把硬件零件组装成一台真正的"会执行程序的机器"。

🔍 本章没讲什么

- 网表的物理表示(GDSII)和层次化平面化(flatten)。
- LEF/DEF 文件:版图层面的网表。
- DPI/DFT 插入对网表的修改。

📚 想深入读什么

- Synopsys 的 *Design Compiler Reference Manual*。
- 任何一本《CMOS VLSI Design》的标准单元章节。

← 上一章:EDA 流程 · → 下一章:数据通路与控制器

第 11 章 · 数据通路与控制器:一颗 CPU 的雏形

> 把前面所有零件——ALU、寄存器堆、MUX、PC——拼起来,我们就能造一颗会执行程序的 CPU。

11.1 CPU 的最小要素

一颗 CPU,无论多复杂,都必须有这几样东西:

部件	类型	作用
PC (Program Counter)	寄存器	记录"下一条要执行的指令在哪"
指令存储器 (Instr Mem)	ROM/RAM	存放程序
寄存器堆 (Register File)	32 个 32 位寄存器	CPU 内部最快的"变量"
ALU (Arithmetic Logic Unit)	组合电路	加减、与或、比较
数据存储器 (Data Mem)	RAM	存放变量、堆栈
控制器 (Control Unit)	FSM / 译码逻辑	决定每个周期"开关谁、连谁"

把它们用导线和 MUX 连起来,就得到了**数据通路(Datapath)**。

11.2 一条指令的"五个动作"

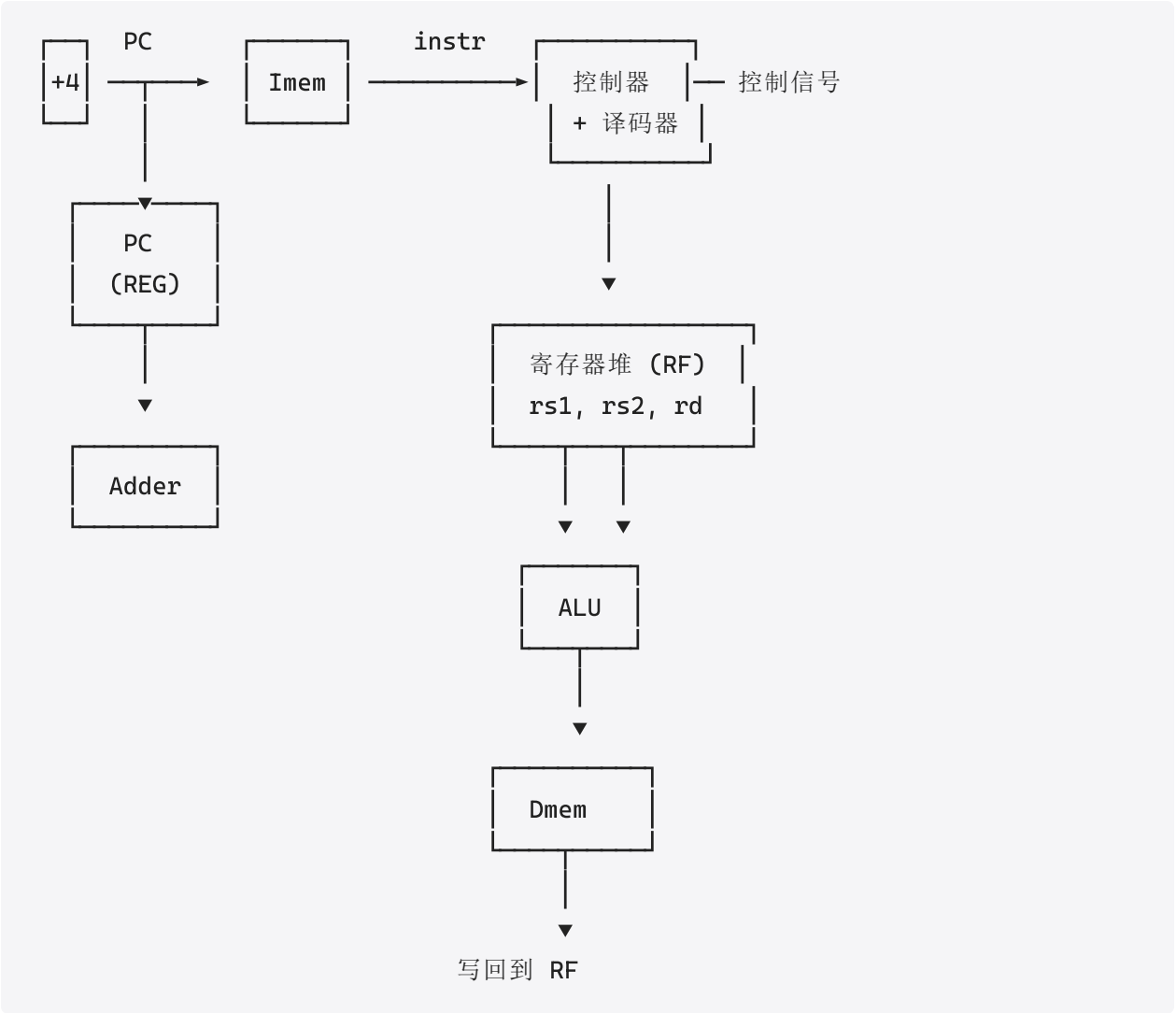
经典 RISC 的每条指令都被拆成 5 个动作:

```
IF   →   ID   →   EX   →   MEM   →   WB
取指     译码     执行     访存       写回
```

阶段	主要部件	在干什么
IF (Instruction Fetch)	PC + 指令存储器	用 PC 当地址,读出一条 32 位指令;PC ← PC + 4
ID (Instruction Decode)	控制器 + 寄存器堆	解析指令字段,读取源寄存器
EX (Execute)	ALU	执行算术/逻辑/地址计算
MEM (Memory Access)	数据存储器	load/store 才用;算术指令空转
WB (Write Back)	寄存器堆	把结果写回目标寄存器

> 💡 **这就是经典「五段式 RISC 数据通路」**,Patterson & Hennessy 教材里的灵魂图。本书用 RISC-V 的 32 位整数子集 (RV32I) 来举例。

11.3 单周期 CPU 的数据通路图



(为简洁省略了多个 MUX、立即数扩展、跳转加法器等细节)

11.4 控制器:数据通路的"指挥棒"

数据通路是固定的"硬件骨架",**指挥它每个周期做什么**——靠的是**控制器。**

控制器接收当前指令的操作码(opcode),输出一组**控制信号**,例如:

控制信号	含义
`RegWrite`	这个周期是否要写寄存器堆?
`MemRead` / `MemWrite`	是否访问数据存储器?
`ALUOp`	ALU 该做加、减、还是与?
`ALUSrcB`	ALU 第二个输入选寄存器还是立即数?
`Branch` / `Jump`	是否跳转?

它本质上就是一个**真值表 / FSM**:opcode → 一组控制位。可以用 ROM 实现(微程序),也可以用组合逻辑直接综合(硬连线)。

11.5 一个具体例子: `add x3, x1, x2`

这条 RISC-V 加法指令在数据通路上"走过一遍"的过程:

1. **IF**:从 Imem[PC] 读出 32 位指令字 `0x002080B3`;PC ← PC + 4。
2. **ID**:控制器看到 opcode = R-type、funct3=000、funct7=0 → 这是 `ADD`。同时寄存器堆读出 `rs1=x1` 和 `rs2=x2`。
3. **EX**:ALU 接到两个值, `ALUOp = ADD`,输出 `x1 + x2`。
4. **MEM**:这是算术指令,不访存,空转一拍。
5. **WB**: `RegWrite = 1`,把 ALU 输出写回 `rd = x3`。

—— **这就是「执行一条指令」**。CPU 不停地循环这五步,程序就一条接一条地跑起来了。

11.6 单周期 → 多周期 → 流水线

问题:单周期 CPU 中,每条指令的所有 5 步都在一个时钟周期里完成。但 **MEM 阶段可能要 100ps,而 EX 只要 30ps** —— 时钟周期被最慢的那一步绑架,整体性能很差。

改进方案:

1. **多周期**:把 5 步分到 5 个时钟周期。每周期短了,但每条指令要 5 个周期。**总时间没变,但更省硬件**(可以复用 ALU 等)。

2. **流水线:5 个阶段并行处理 5 条不同的指令** —— 这才是真正的性能突破。下一章详细讨论。

11.7 串联到下一章

我们现在有了一颗能"一条一条执行指令"的 CPU。但现代 CPU 之所以能跑 GHz 级的频率、每秒执行几十亿条指令,核心秘诀是 **流水线、超标量、乱序执行、分支预测。**

下一章我们就深入流水线 —— 看一颗 CPU 如何"同时处理 5 条指令",以及它必须为此付出的代价(冒险、气泡、转发)。

🔍 本章没讲什么

- RISC-V 完整指令集(本章只用了几条做示意)。
- CISC vs RISC 的历史与权衡。
- 微码(microcode)与 x86 的内部 RISC-fy。

📚 想深入读什么

- Patterson & Hennessy, *Computer Organization and Design (RISC-V Edition)* — 第 4 章是本章的"超长展开版"。
- Harris & Harris, *Digital Design and Computer Architecture* — 直接给 Verilog 实现。

← 上一章:网表 · → 下一章:流水线

第 12 章 · 流水线、冒险、分支预测

> 流水线让 CPU 频率上了 GHz,但也带来一连串"麻烦"——冒险与气泡。

12.1 流水线的直觉:洗衣店

想象一家洗衣店有 4 个工序:**洗** → **烘** → **叠** → **收**。每件衣服要 30 分钟,4 个工序各 30 分钟,合计 2 小时。

- **不流水**:洗一件,烘一件,叠一件,收一件 —— 8 小时洗 4 件。
- **流水**:衣服 1 在「烘」时,衣服 2 已经在「洗」 —— **每 30 分钟出一件**,4 件总共 2.5 小时。

CPU 的流水线一样的道理:**每条指令仍然走 5 个阶段,但 5 条指令各处不同阶段,同时在跑。**

12.2 五级流水线时序图

```
时钟  1     2     3     4     5     6     7     8
i1   IF    ID    EX    MEM   WB
i2         IF    ID    EX    MEM   WB
i3               IF    ID    EX    MEM   WB
i4                     IF    ID    EX    MEM   WB
i5                           IF    ID    EX    MEM   WB
```

理想情况下,每个时钟周期完成一条指令(IPC = 1),而时钟周期可以做得很短(因为每段只是 1/5 的工作量)。

12.3 为流水线服务:加流水线寄存器

物理实现上,**每两个阶段之间要插一组流水线寄存器(Pipeline Register)**,它们在每个时钟边沿"打一拍",把上一阶段的中间结果传给下一阶段。

```
   IF ──[IF/ID Reg]── ID ──[ID/EX Reg]── EX ──[EX/MEM Reg]── MEM ──[MEM/WB Reg]-
```

这些寄存器其实就是上一章的 D 触发器组,**它们让流水线"看上去并行,实际上靠时钟边沿同步推进"**。

12.4 三大流水线冒险(Hazards)

理想很美好,现实很残酷。流水线里**指令彼此不再独立**,会引发三类问题:

① 数据冒险(Data Hazard)

```
i1:  add  x1, x2, x3      ; 写回 x1 在 WB(周期 5)
i2:  sub  x4, x1, x5      ; 但 ID 阶段(周期 3)就要读 x1!
```

`i2` 在 i1 还没把 x1 写回时就想读 x1 —— 读到的是旧值。

解决方法: - **气泡(stall)**:让 i2 等几拍。简单但慢。 - **转发 / 旁路(Forwarding/Bypass)**:**直接从 EX/MEM 的输出"抢"出新值送回 EX 输入**,不等写回。这是现代 CPU 的常规手段。

```
i1:  IF  ID  EX ─┬─ MEM   WB
                 └──────► EX  (i2 的 EX 直接拿到 i1 的结果)
i2:  IF  ID  EX     ...
```

② 控制冒险(Control Hazard)

```
i1:  beq  x1, x2, target   ; 分支指令
i2:  ???                    ; 谁该跟在 i1 后面?
```

问题:i1 的分支结果(跳还是不跳)要到 EX 之后才知道,但流水线已经把 i2、i3 取进来了。如果分支真的要跳,这两条就**白做了**。

解决方法: - **冲刷流水线**:发现分支跳了,就把已经取的指令清空(气泡)。损失大。 - **延迟槽(Delay Slot)**:MIPS 早期搞过,让程序员/编译器把"分支后总会执行"的一条放到分支指令之后。复杂,现在大多放弃了。 - **分支预测(Branch Prediction)**:**猜!**

③ 结构冒险(Structural Hazard)

两个阶段同时想用同一个硬件单元(例如内存)。早期设计常见,现代 CPU 通过**Cache + 多端口存储器**来回避。

12.5 分支预测:让 CPU "赌"未来

现代 CPU 有一个小硬件:**分支预测器**,它根据历史信息**预测**这条分支会不会跳,以及跳到哪。然后流水线**乐观地按预测值继续取指令**。

- **预测对**:零损失,流水线满速运行。
- **预测错**:把已经取/译/执行了一半的"猜测指令"全冲掉,流水线损失几个周期。

预测器的演进史:

类型	思路	准确率
Static (Always Taken)	总猜跳	~60%
1-bit predictor	上次跳就猜跳	~80%
2-bit saturating counter	"强不跳/弱不跳/弱跳/强跳"	~90%
BHT (Branch History Table) + 2-bit	按 PC 索引记录	~93%
Two-level / GShare	全局历史 + 局部历史	~95%
TAGE / 神经网络预测	多个长度历史预测器投票	~98%+

> 💡 **现代 CPU 的分支预测准确率超过 95%**,这是流水线性能的关键。Intel/AMD 旗舰 CPU 的分支预测器复杂度堪比一个小型 ML 模型。

12.6 超标量与乱序执行(简介)

工业界 CPU 早就不止"5 级、每周期 1 条"了:

- **超标量(Superscalar)**:**每周期同时发射多条指令**(Intel 主流 CPU 是 4-6 发射)。

- **乱序执行(Out-of-Order)**:把指令打乱顺序,**只要数据准备好的就先执行**,最后再按顺序写回(Tomasulo 算法)。
- **重命名寄存器**:解决"假数据冒险"(WAR/WAW)。
- **超长流水线**:Pentium 4 一度做到 31 级。

但这些都建立在"5 级流水线 + 三大冒险"这个心智模型上,只是规模放大。

12.7 ⚠️ 流水线的副作用:Spectre / Meltdown

2018 年震动业界的 CPU 安全漏洞,**根源就是分支预测和乱序执行**:

- CPU 在分支预测错误时虽然冲掉了"逻辑结果",**但 Cache 中已经留下了访问痕迹**。
- 攻击者通过精心设计的代码 + Cache 侧信道测量,能读出本不应该看到的内核数据。

这告诉我们:**流水线不是单纯的性能优化,它对程序的"可观察行为"也有影响。**

12.8 串联到下一章

CPU 跑得再快,也得"喂得上数据"。

- **寄存器** 极快但只有几十个。
- **DRAM 主存** 大但很慢(一次访问 100ns 级别 = 几百个时钟周期)。

中间这"几百倍的差距",必须靠**Cache** 来填。下一章讨论 **存储层次结构**——CPU 性能优化里最重要、也最反直觉的话题之一。

🔍 本章没讲什么

- VLIW(超长指令字)与显式并行。
- SIMD / Vector 指令。
- GPU 的 SIMT 架构与 CPU 的差异。
- 多核、缓存一致性协议(MESI)。

📚 想深入读什么

- Hennessy & Patterson, *Computer Architecture: A Quantitative Approach* — 体系结构圣经。
- Shen & Lipasti, *Modern Processor Design* — 超标量/OoO 详解。

← 上一章:数据通路 · → 下一章:存储层次

第 13 章 · 存储层次:寄存器、Cache、内存、磁盘

> CPU 越来越快,内存却没跟上。整个体系结构的"伪装术",就是让你以为内存又快又大。

13.1 内存墙(Memory Wall)

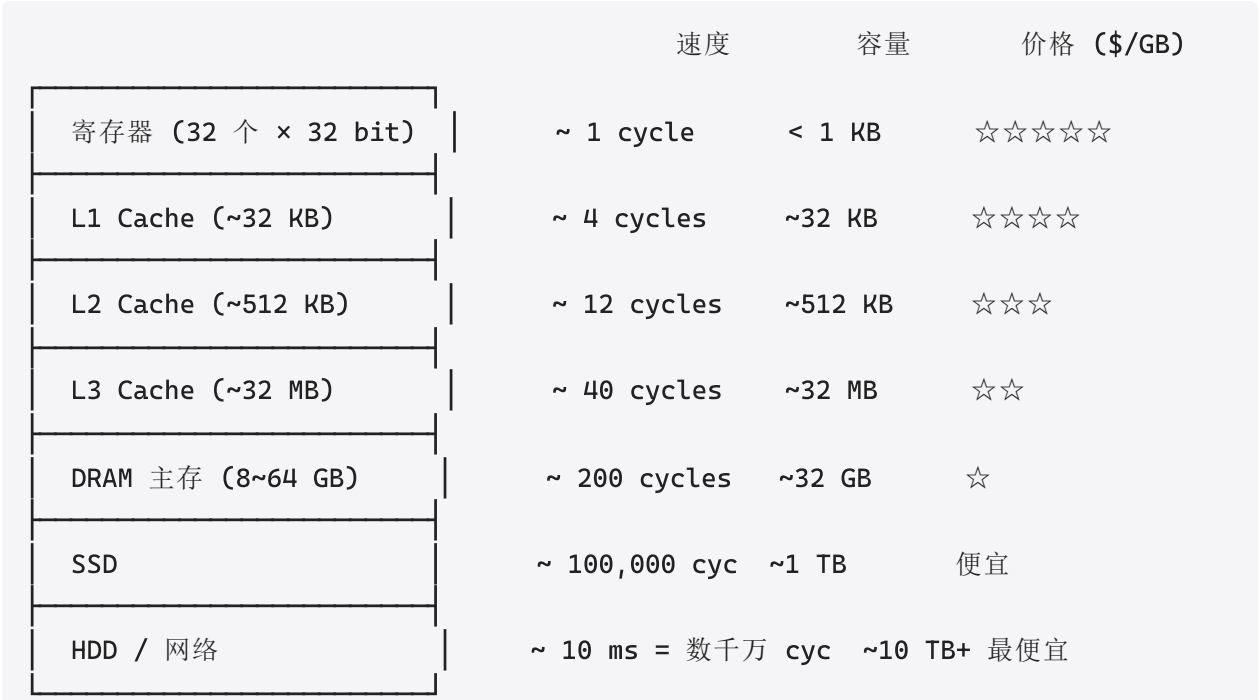
过去 40 年:

- CPU 速度提升 ~10000 倍。
- DRAM 速度提升 ~10 倍。

差距越拉越大,**今天 CPU 等一次主存访问,要"虚度"100~300 个时钟周期。**如果不解决这个问题,流水线再快也是空转。

> 💡 这是体系结构里的核心矛盾:**容量大** ↔ **速度快** ↔ **成本低**,三者只能选两个。

13.2 存储层次:把这三个矛盾分层处理



关键的精神洁癖:每一层只是下一层的 Cache。CPU 永远以为自己在跟最快那层(L1)打交道,但当 L1 缺数据,它会从 L2 拿;L2 缺数据,从 L3 拿……最终从硬盘/网络拿。

13.3 局部性原理(Locality)——Cache 能成立的根本

CPU 访问内存的模式不是随机的,而是高度**局部化**的:

- **时间局部性**:刚访问的地址,马上还会再访问(循环变量、函数参数)。
- **空间局部性**:刚访问的地址附近,马上会被访问(数组、结构体、代码顺序执行)。

> 没有局部性,Cache 就毫无意义——任何 Cache 都会一直 miss,等同于直接访问主存。
> **现代软件性能优化的根本,就是利用局部性。**

13.4 Cache 的基本组织

Cache 把内存地址按"行(Line)"组织,通常一行 64 字节(L1)。

每个 Cache Line 包含:

- **Tag**:地址的高位,用来识别这是哪一块。
- **Data**:64 字节的真实数据。
- **Valid bit**:这一行是否有效。
- **Dirty bit**(写回式):被修改过没有?

按"一个内存地址能映射到 Cache 哪个位置",Cache 分三种:

类型	映射	命中查找
直接映射	地址 % 行数 → 唯一一个槽	简单,但易冲突
全相联	任意位置	灵活,但比较器代价高
组相联	先取模找一组,组内任意位置	折中,**主流方案**。常见 4-way / 8-way / 16-way。

13.5 Cache 缺失的三种类型(3C)

- **Compulsory(强制)**:第一次访问,Cache 里没有 —— 不可避免。
- **Capacity(容量)**:数据集太大,Cache 装不下。
- **Conflict(冲突)**:在组相联中,几个数据映射到同一组,把彼此挤出去。

软件优化的常见目标就是**减少 Capacity / Conflict miss**——例如循环展开、分块、对齐分配。

13.6 一个能"看到"的局部性例子

```
// 例 1: 行优先访问(快)
for (int i = 0; i < N; i++)
    for (int j = 0; j < N; j++)
        sum += A[i][j];

// 例 2: 列优先访问(慢得多!)
for (int j = 0; j < N; j++)
    for (int i = 0; i < N; i++)
        sum += A[i][j];
```

C 语言数组按**行主序**存放,例 1 每次读连续 64 字节里的 16 个 int,**16 次访问只 1 次 miss**;例 2 每次跨 N×4 字节,**几乎每次都 miss。**

> 在 N = 4096 时,这两个版本可以差 **5~20 倍**性能。这是软件工程师必须懂硬件的最经典证据。

13.7 写策略

CPU 写一个变量时,Cache 怎么办?

- **Write-through**:同时写 Cache 和下级内存。简单,但慢。
- **Write-back**:只写 Cache,标 Dirty;直到这一行被替换时才写回主存。**主流方案。**

替换策略(组满了怎么办): - LRU(Least Recently Used)、LFU、Random、PLRU(伪 LRU)……

13.8 虚拟内存:再加一层 Cache

操作系统给每个进程一个独立的"虚拟地址空间",4 GB / 256 TB…… 但物理 DRAM 只有十几 GB。**虚拟到物理的翻译表**叫 **Page Table。**

为了不让每次访问都查页表(那也得访存),CPU 加了一个专用 Cache:

- **TLB(Translation Lookaside Buffer)**:虚拟页号 → 物理页号的小 Cache,通常几十~几百项,1~2 周期。

这一切都对程序员透明——你以为你在访问 `0x7fffd1234000`,实际上 CPU 正在做翻译、查 TLB、找 Cache、必要时读 DRAM、必要时换页到磁盘。

13.9 串联到 Part V:从硬件到 ISA

到这里 Part IV 结束:

- **数据通路**:让一条指令能流过五个阶段。
- **控制器**:让 CPU 能"读懂"指令。
- **流水线**:让多条指令并行。
- **存储层次**:让内存访问不拖累流水线。

但所有这一切,都建立在一个共同的"契约"上:CPU 究竟支持哪些指令?每条指令长什么样?

这个契约,就是 **ISA(指令集架构)** —— 软件与硬件的边界,Part V 的主角。

🔍 本章没讲什么

- 多核 Cache 一致性协议(MESI、MOESI)。
- NUMA 架构与跨 socket 访问。
- 持久内存(Persistent Memory)、CXL、HBM。

📚 想深入读什么

- Hennessy & Patterson, *Computer Architecture* — 第 2 章存储层次。
- Bryant & O'Hallaron, *CSAPP* — 第 6 章,程序员视角看 Cache。

← 上一章:流水线 · → 下一章:ISA

第 14 章 · 指令集架构 (ISA):软硬件契约

> 🌉 **桥 3** —— 这是软件与硬件之间唯一一份"合同"。 ISA 决定了一边的 CPU 设计师怎么造芯片,另一边的编译器作者怎么生成代码。

14.1 ISA 是什么?为什么是「契约」?

ISA(Instruction Set Architecture,指令集架构)= 一套规则,定义"这颗 CPU 能识别哪些指令、它们长什么样、寄存器有几个、内存怎么寻址"。

软硬件之间的关系:

```
软件层(编译器、操作系统、汇编程序员)
──────────────── 契约: ISA ────────────────
硬件层(微架构、流水线、Cache、布局布线)
```

ISA 同时是**两组人不见面合作的协议**:

- 软件方:**只要我生成的指令在 ISA 范围内,我不在乎你怎么实现它**(用 5 级流水线还是 OoO?用 7nm 还是 28nm?随你)。
- 硬件方:**只要我能正确执行这套 ISA,我可以随便折腾内部**(分支预测器多猛、Cache 多大,自由发挥)。

> 🔑 **这份契约,是过去 70 年计算机产业繁荣的根基。** 它让软件和硬件可以独立演化:操作系统升级时不需要换 CPU,CPU 升级时不需要重写软件。

14.2 ISA 的几大要素

任何 ISA 都包含:

1. **数据类型**:整数(8/16/32/64 位)、浮点(单/双精度)、向量。

2. **寄存器**:数量、宽度、用途约定(general-purpose、stack pointer、return addr...)。
3. **寻址方式**:立即数、寄存器、寄存器+偏移、间接.....
4. **指令编码**:每条指令多少 bit,字段怎么分。
5. **指令类别**:算术、逻辑、访存、分支、系统调用、特权.....
6. **存储器模型**:大端 / 小端、内存一致性、原子性。
7. **异常 / 中断**:CPU 怎么响应外部事件。
8. **特权级别**:用户态、内核态、虚拟化扩展。

14.3 RISC vs CISC:两种哲学

	RISC	CISC
代表	RISC-V、ARM、MIPS、SPARC	x86、x86-64、原生 z/Architecture
指令长度	固定(32 bit)	可变(1~15 字节)
指令数量	几十~一百多	上千
操作复杂度	简单(load/store + ALU 分离)	复杂(`rep movsb` 一条 ≈ 一段循环)
编译器友好	✓✓	✗(汇编更友好)
实现难度	低	高,但内部也"RISC化"

> 💡 **现代 x86 内部其实也是 RISC**:CPU 把每条 x86 指令解码成多个 μ-op(类似 RISC 的微指令)再执行。这是 CISC 表壳 + RISC 内核。

14.4 RISC-V:本书的范例 ISA

我们选 RISC-V 作为讲解 ISA 的工具,因为:

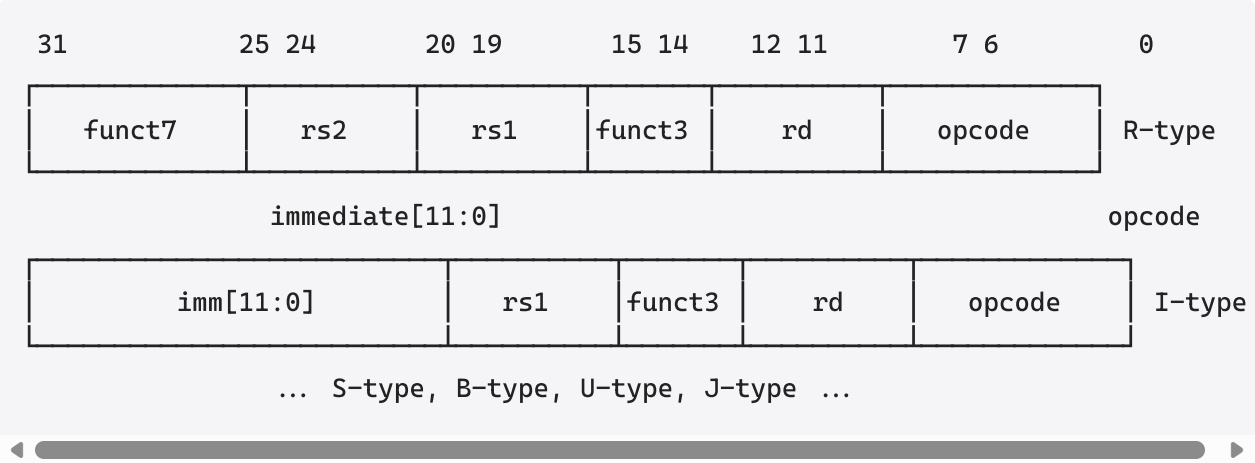
- **完全开源**,不用付授权费,任何人能造 RISC-V CPU。
- **设计干净**,没有历史包袱(x86 那些 segment 地址、DOS 兼容指令都没有)。
- **模块化**:RV32I 是基础整数集,扩展 M(乘除)、A(原子)、F(浮点)、C(压缩)等按需加。

RISC-V RV32I 寄存器

寄存器	别名	用途
x0	zero	**永远是 0**
x1	ra	返回地址(Return Address)
x2	sp	栈指针(Stack Pointer)
x3	gp	全局指针
x4	tp	线程指针
x5~x7	t0~t2	临时寄存器(caller-saved)
x8	s0/fp	保存寄存器 / 帧指针
x9	s1	保存寄存器(callee-saved)
x10~x17	a0~a7	**函数参数 / 返回值**
x18~x27	s2~s11	保存寄存器
x28~x31	t3~t6	临时寄存器

外加一个不属于通用寄存器堆的 **PC(程序计数器)。**

指令格式(RV32I 共 6 种)



> 💡 **所有 RISC-V 指令都是 32 bit**(若启用 C 扩展可以 16 bit)。这种规整性让取指与译码都极简单——**这正是 RISC 哲学的体现。**

14.5 一条指令的"三种身份"

同一条 RISC-V 加法指令,在不同视角下有不同形态:

视角	形态
程序员/编译器	`add x3, x1, x2` (汇编助记符)
二进制(ELF 内)	`0x002080B3` (机器码)
CPU 内部	一组控制信号: `RegWrite=1, ALUOp=ADD, RegSrc1=x1, …`

它们彼此一一对应。**汇编 → 机器码 由「汇编器」做;机器码 → 控制信号 由「CPU 译码器」做**(就是上一章数据通路里的控制器)。

下一章我们会专门讲汇编 → 机器码 → 控制信号这条链路。

14.6 ISA 的"非功能"重要性

ISA 决定的远不止"能做什么":

- **性能**:寄存器数量影响 spilling;指令稠密度影响 i-cache 命中率。
- **能耗**:简单 ISA 解码功耗低,适合移动设备(为什么 ARM 统治了手机)。
- **安全性**:特权指令、内存保护、Pointer Authentication(ARMv8.3)、CFI 扩展。
- **可移植性**:同一 ISA 的不同 CPU 可以跑同一个二进制(x86 30 年兼容)。

14.7 串联到下一章

ISA 是"规格说明书",但**程序员不会直接写机器码。**我们写**汇编**,然后交给汇编器变成机器码;或者写**高级语言**,交给编译器变成汇编。

下一章,我们就近距离看汇编语言:**它的语法、它和机器码的对应、它在 ELF 文件中怎么存放。**

🔍 本章没讲什么

- x86-64 的完整寻址、SIMD 指令(SSE/AVX)。
- ARM AArch64 的 ABI 与 RISC-V 的差异。
- 内存一致性模型(weak vs strong、TSO vs RC)。
- 量子计算 ISA、神经网络加速器 ISA。

📚 想深入读什么

- *RISC-V Specification* —— riscv.org 免费下载。
- Patterson & Waterman, *The RISC-V Reader*.
- Intel SDM(Software Developer Manual)—— 看一眼就知道 CISC 的"重量"。

← 上一章:存储层次 · → 下一章:汇编语言

第 15 章 · 机器码与汇编语言

> 机器码是 CPU 唯一真正"读懂"的东西。 汇编只是给人类看的"机器码别名"。

15.1 从二进制到助记符

CPU 实际执行的是 32 位(或 64 位)的二进制串。例如 RISC-V 的 `add x3, x1, x2`:

```
机器码 (二进制):
  0000000  00010   00001   000   00011   0110011
   funct7   rs2     rs1   funct3   rd     opcode

机器码 (16 进制): 0x002080B3
```

人脑直接读这些 0/1 难以承受,所以发明了**汇编(Assembly)**——一种**与机器码一一对应**的文本表示。

```
汇编:    add x3, x1, x2
机器码:  0x002080B3
```

汇编**没有"编译"成机器码,而是「翻译」**——通常每一行汇编对应一条机器码(伪指令除外)。这个翻译工作由 **汇编器(Assembler)** 完成,例如 GNU `as`、`riscv64-unknown-elf-as`。

15.2 RISC-V 常见指令(RV32I 子集)

算术与逻辑

```
add  x3, x1, x2      # x3 = x1 + x2
sub  x3, x1, x2      # x3 = x1 - x2
and  x3, x1, x2      # x3 = x1 & x2
or   x3, x1, x2      # x3 = x1 | x2
xor  x3, x1, x2      # x3 = x1 ^ x2
sll  x3, x1, x2      # x3 = x1 << x2  (左移)
srl  x3, x1, x2      # 逻辑右移
sra  x3, x1, x2      # 算术右移
slt  x3, x1, x2      # x3 = (x1 < x2) ? 1 : 0
```

立即数版本(I-type)

```
addi x3, x1, 100     # x3 = x1 + 100
andi x3, x1, 0xFF    # x3 = x1 & 0xFF
slti x3, x1, 5       # x3 = (x1 < 5)
```

访存(Load / Store)

```
lw   x3, 0(x1)       # x3 = mem[x1 + 0]   (load word, 4 字节)
lh   x3, 4(x1)       # 半字 (2 字节,有符号扩展)
lb   x3, 7(x1)       # 字节 (1 字节)
lbu  x3, 7(x1)       # 字节 (无符号扩展)
sw   x2, 8(x1)       # mem[x1 + 8] = x2
```

> 💡 **RISC 的 Load/Store 哲学:只有 load/store 能访问内存**,所有算术指令必须先把数据从内存搬到寄存器。这让流水线设计大大简化(因为 EX 阶段不会"突然访存")。

分支与跳转

```
beq  x1, x2, label    # if (x1 == x2) goto label
bne  x1, x2, label    # if (x1 ≠ x2) goto label
blt  x1, x2, label    # if (x1 < x2) goto label   (有符号)
bge  x1, x2, label    # if (x1 ≥ x2) goto label
bltu x1, x2, label    # 无符号小于
jal  x1, label        # x1 = PC + 4; goto label    (跳转并保存返回地址)
jalr x1, x2, 0        # x1 = PC + 4; goto x2 + 0   (间接跳转)
```

系统调用

```
ecall                 # 触发系统调用,陷入内核
ebreak                # 调试断点
```

15.3 一段 C → 一段汇编

输入:

```
int sum(int n) {
    int s = 0;
    for (int i = 1; i ≤ n; i++)
        s += i;
    return s;
}
```

GCC 编译(简化的 RISC-V 汇编):

```
sum:
    li      a1, 0           # s = 0    (a1 当 s)
    li      t0, 1           # i = 1    (t0 当 i)
.L_loop:
    bgt     t0, a0, .L_end  # if (i > n) goto end  ;  a0 是参数 n
    add     a1, a1, t0      # s += i
    addi    t0, t0, 1       # i++
    j       .L_loop
.L_end:
    mv      a0, a1          # 返回值放 a0
    ret                     # = jalr x0, ra, 0
```

每一行有清晰的硬件含义。**这就是高级语言「真正在硬件上发生的事」**。下一章(Ch 16)会展示这段代码怎么被打包成 ELF 文件、被链接、被加载。

15.4 汇编伪指令(Pseudo-Instructions)

为了写汇编更舒服,汇编器支持一些"伪指令"——它们本身不存在于 ISA,会被汇编器展开成真实指令:

伪指令	实际展开
`li x3, 100`	`addi x3, x0, 100`
`mv x3, x1`	`addi x3, x1, 0`
`nop`	`addi x0, x0, 0`
`j label`	`jal x0, label`
`ret`	`jalr x0, ra, 0`
`not x3, x1`	`xori x3, x1, -1`

> 💡 **CPU 的"指令数量"和"汇编程序员能写的助记符数量"不一定相等**——伪指令是中间的甜头。

15.5 字节序(Endianness):0x12345678 怎么存?

32 位整数 `0x12345678` 存到地址 0x1000 开始的 4 字节:

	0x1000	0x1001	0x1002	0x1003
Big-Endian	0x12	0x34	0x56	0x78
Little-Endian	**0x78**	**0x56**	**0x34**	**0x12**

x86、x86-64、RISC-V、ARM(默认)都是 **小端**。MIPS、PowerPC 早期是大端,网络协议(如 TCP/IP)用大端(俗称"网络字节序")。

> ⚠️ 跨平台传数据要用 `htonl()` / `ntohl()` 转换,否则可能出现"数字莫名变了"的诡异 bug。

15.6 调用约定(Calling Convention):函数怎么"沟通"

ISA 不规定,**ABI(Application Binary Interface)** 才规定:

- 参数怎么传?**RISC-V**:前 8 个用 `a0-a7`,多余的进栈。
- 返回值放哪?**RISC-V**: `a0`(双字 `a0+a1`)。
- 谁负责保存哪些寄存器?
- **Caller-saved (`t0-t6, a0-a7`)**:调用前自己存,调用后自己恢复。
- **Callee-saved (`s0-s11, sp, ra`)**:被调用函数如果用了,必须先存进栈,返回前恢复。

如果不遵守 ABI,**两个用不同规则的函数互相调用会立刻崩。**这就是为什么所有编译器、汇编手写代码、动态链接库**必须使用同一份 ABI。**

15.7 串联到下一章

我们已经会写汇编、读机器码。但一段汇编要变成可以双击运行的 `.exe` / ELF,还差几步:

- **汇编器** 把多个 `.s` 翻译成 `.o` 目标文件。
- **链接器** 把 `.o`、库、运行时拼到一个可执行文件。
- **加载器** 把可执行文件读进内存、设置好寄存器、跳到入口。

下一章我们打开 ELF 文件,看里面到底装了什么。

🔍 本章没讲什么

- x86-64 的 SSE/AVX 向量汇编。
- ARM AArch64 的 LDP/STP 配对指令。
- 位置无关代码(PIC)、GOT、PLT。

📚 想深入读什么

- Patterson & Waterman, *The RISC-V Reader* — 简洁。
- Bryant & O'Hallaron, *CSAPP* 第 3 章 — x86-64 视角的精华。

← 上一章:ISA · → 下一章:链接、加载、ELF

第 16 章 · 链接、加载、ELF 文件结构

> 一段汇编 → 一个能跑的程序,中间还有"打包、整合、上货"三步。

16.1 编译流程的四个阶段

输入一个 `hello.c`,GCC 默默做了 4 步:

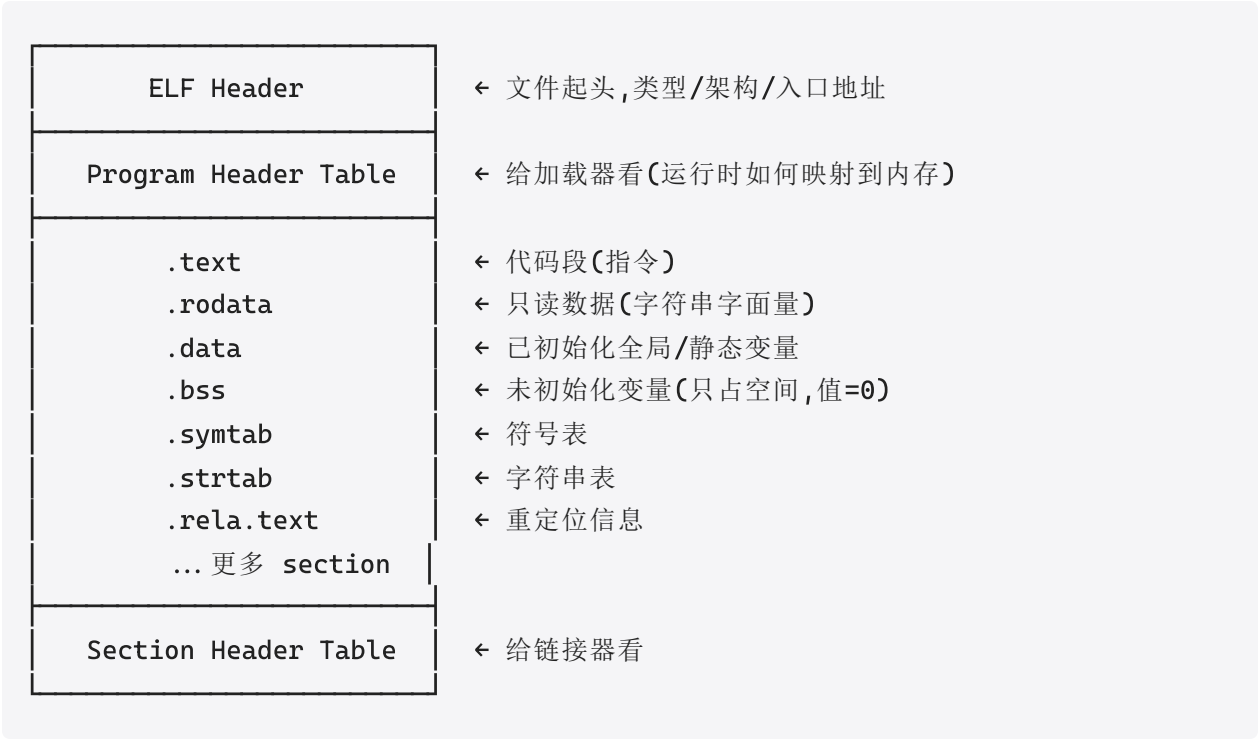
```
hello.c   ──[预处理]──► hello.i      (把 #include 展开,宏替换)
hello.i   ──[ 编译  ]──► hello.s      (变成汇编)
hello.s   ──[ 汇编  ]──► hello.o      (变成 ELF 目标文件)
hello.o   ──[ 链接  ]──► hello        (链接库,生成可执行文件)
```

可以让 GCC 给你看每一步:

```
gcc -E hello.c -o hello.i      # 只预处理
gcc -S hello.c -o hello.s      # 编译到汇编
gcc -c hello.c -o hello.o      # 汇编到目标文件
gcc hello.o -o hello           # 链接
```

16.2 ELF:Linux/Unix 的可执行文件格式

ELF (Executable and Linkable Format) 文件结构:



可以用 `readelf` 探索:

```
readelf -h hello            # ELF Header
readelf -l hello            # Program Headers
readelf -S hello            # Section Headers
objdump -d hello            # 反汇编 .text
nm hello                     # 看符号表
```

> 💡 **ELF 同时是「目标文件」和「可执行文件」的格式。** `.o` 文件给链接器, `hello` 给加载器,但底层格式同源。Windows 用 PE,macOS 用 Mach-O,各家略有不同,但思想一致。

16.3 链接器干什么?——「拼图」

如果一个程序由 `main.c`、`util.c`、`math.c` 组成,各自编译出 `main.o`、`util.o`、`math.o`。每个 `.o` 都引用了别的文件里的函数,但地址都是"占位的"。

链接器(`ld`)做两件事:

① 符号解析(Symbol Resolution)

在 `main.o` 里看到一个 `call sum`,但 `sum` 不在 `main.o`。链接器到 `util.o` 找,发现 `sum` 在那里,记录下"`main.o` 里 0x14 处的 call 应当指向 `util.o` 里 sum 的位置"。

如果找不到,你就遇到经典报错:**`undefined reference to 'sum'`**。

② 重定位(Relocation)

把所有 `.o` 的代码段拼到一起,按拼接后的最终地址,**修正所有"占位的地址"**。

```
main.o 里:
   call <0x00000000>      # 占位:这里要调 sum

链接后(假设 sum 最终地址是 0x4001A0):
   call <0x004001A0>
```

每个 `.o` 都带有 `.rela.text` section,告诉链接器**哪些地方需要重定位、用哪个符号、怎么算**。

16.4 静态链接 vs 动态链接

静态链接

把库 `libc.a` 整个(或所需部分)**直接拷进可执行文件**。

- ✅ 不依赖运行环境的库版本。
- ✗ 可执行文件大;每个程序都包含一份 libc 副本,内存浪费。

动态链接(主流)

可执行文件只记录"需要 libc.so",真正的库代码在运行时加载、多个进程共享。

- ✅ 文件小,内存高效,libc 升级所有程序受益。
- ✗ 依赖运行环境(臭名昭著的 "DLL Hell" / "shared library mismatch")。

实现机制:**PLT(Procedure Linkage Table)** + **GOT(Global Offset Table)** —— 一种"延迟绑定"技巧,函数第一次被调用时才解析地址。

16.5 加载器:把 ELF 搬到内存

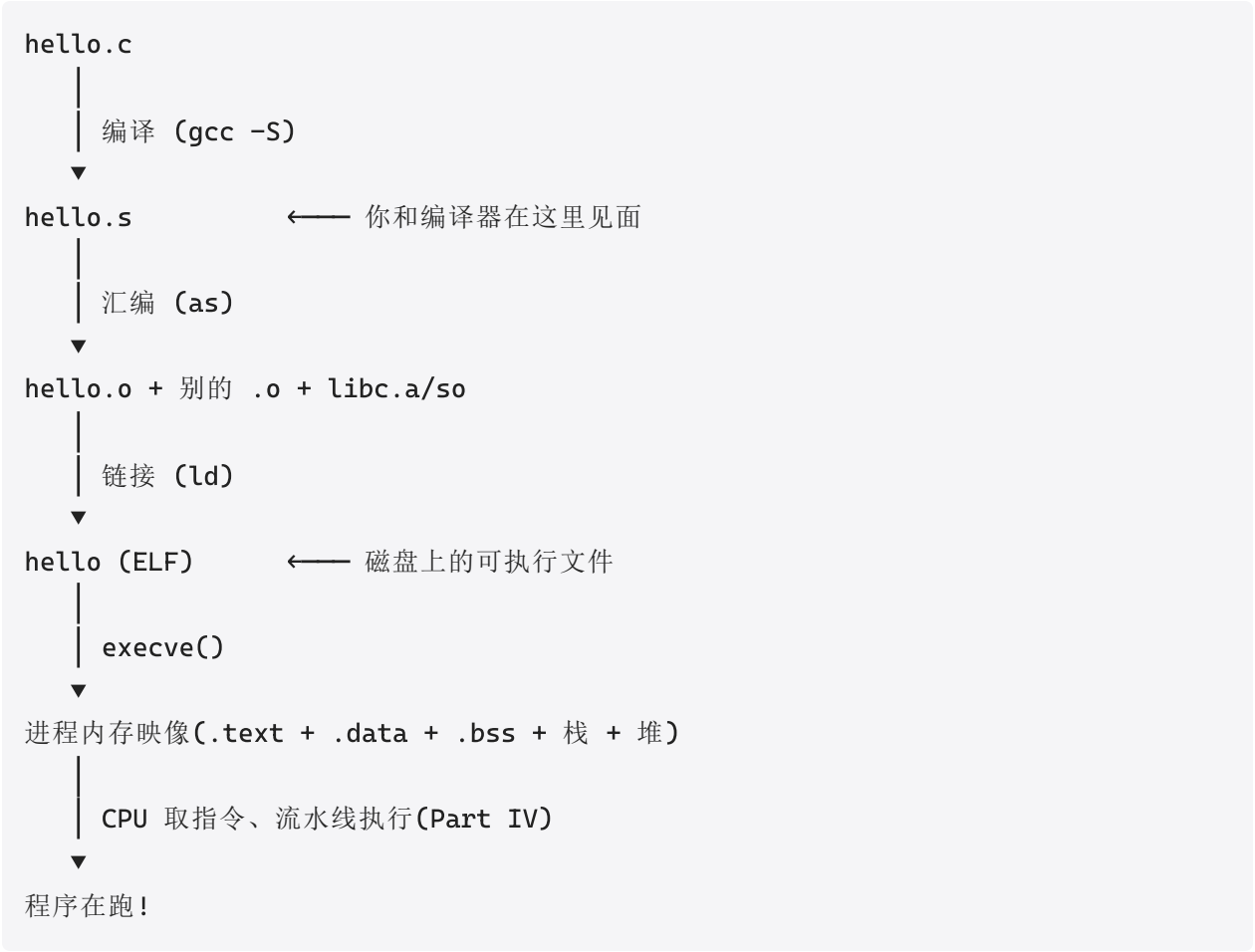
当你 `./hello` 时,内核 `execve` 系统调用做这些事:

1. 读 ELF Header,确认架构、可执行。
2. 按 Program Header,**把各 segment 映射到进程虚拟地址空间**(`mmap`): - `.text` → 只读 + 可执行段 - `.data` → 可读写段 - `.bss` → 全 0 页
3. 设置好栈、堆、命令行参数 `argv` 、环境变量 `envp` 。
4. 如果是动态链接,先调 `ld-linux.so` 加载共享库、解析符号。
5. 把 PC 跳到 ELF 的 **入口地址**(通常是 `_start` —— libc 提供,会调你的 `main`)。

之后,CPU 就开始一个时钟周期一个时钟周期地从 `_start` 取指令、执行——上一章 Part IV 讲的所有事情。

> 🌟 **这就是「双击运行」背后发生的事。**一切看似"瞬间打开"的应用,内部都跑过这套流程。

16.6 一个完整链路图(汇编 → 跑起来)



16.7 串联到 Part VI:为什么程序员主要写 C 而不是汇编?

汇编功能强,但有几个致命问题:

1. **完全不可移植**:换 ISA 就要重写。
2. **生产率极低**:写一个简单循环要十几条指令。
3. **难以维护**:寄存器分配等细节让人崩溃。
4. **抽象能力弱**:无法很好地表达数据结构、类型、模块。

于是 1972 年 Dennis Ritchie 发明了 **C 语言**——一种"可移植的汇编"。它既保持了对硬件的直接控制(指针、内存布局),又提供了抽象(函数、结构体、类型),还能被编译到任何架构。

下一部分(Part VI)我们就深入 C —— 它如何成为系统级软件的"通用语",以及它和上面汇编的对应关系。

🔍 本章没讲什么

- ELF 32-bit / 64-bit 的字段差异。
- Mac Mach-O / Windows PE 的对比。
- LTO(Link-Time Optimization)、IPO。

📚 想深入读什么

- 俞甲子等, *程序员的自我修养:链接、装载与库* —— 中文最佳。
- Levine, *Linkers and Loaders* — 英文经典。
- *CSAPP* 第 7 章。

← 上一章:汇编 · → 下一章:C 语言

第 17 章 · C 语言:可移植的汇编

> 「C 是高级语言中最低级的,也是低级语言中最高级的。」 它是连接软件世界和硬件世界的中枢。

17.1 为什么 C 这么特殊?

1972 年 Dennis Ritchie 在 Bell Labs 设计 C 时,目标极其朴素:**写 Unix 操作系统**。所以 C 必须:

- **能直接操作硬件**(指针、位运算、内存布局)。
- **可移植**(同一份代码能编译到 PDP-11、IBM、x86...)。
- **运行时极小**(操作系统不能依赖一个庞大的运行时)。
- **生成代码贴近汇编**(没有藏起来的"魔法")。

50 年过去了,C 至今仍是:

- 操作系统内核(Linux、Windows NT、macOS XNU、FreeBSD)的主语言。
- 嵌入式与单片机的事实标准。
- 数据库引擎、解释器(CPython、V8 早期)、JVM、虚拟机的实现语言。
- 几乎所有其他语言的最初引导(bootstrap)语言。

> 💡 **理解 C,你就理解了"软件如何变成硬件"**。它的每一个特性都能直接映射到上一章的汇编/ISA。

17.2 C 的核心抽象:类型 + 表达式 + 控制流 + 函数 + 指针

数据类型(直接对应硬件)

C 类型	通常宽度	对应汇编 load/store
`char`	1 字节	`lb` / `sb`
`short`	2 字节	`lh` / `sh`
`int`	4 字节	`lw` / `sw`
`long` (64 位平台)	8 字节	`ld` / `sd`
`float`	4 字节	`flw` / `fsw`
`double`	8 字节	`fld` / `fsd`
`T*` (指针)	与字长相同	同 long

> 💡 **C 的类型系统就是"对硬件类型的直接命名"**——这是它低级的体现。

控制流(都能直接翻译成分支/跳转)

```
if (x > 0) y = 1; else y = -1;      // → blt + jal
while (i < n) i++;                   // → cmp + bge + j
for (i = 0; i < n; i++) ...;         // → 同上
switch (x) { case 1: ... }           // → 跳转表 / 多个 beq
```

每一种 C 控制结构都对应一组分支指令,Ch 19 会逐一拆解。

函数(对应 jal + ret + ABI)

```
int sum(int n) { ... }
int total = sum(10);    // 编译器:把 10 装进 a0,jal sum,把返回值从 a0 取出
```

17.3 C 的核心武器:指针

指针就是地址 + 类型。

```
int x = 42;
int *p = &x;        // p 持有 x 的地址
*p = 100;           // 通过 p 写 x
```

这两行翻译成 RISC-V:

```
# int x = 42;  存在栈上 [sp + 12]
li   t0, 42
sw   t0, 12(sp)

# int *p = &x;  p 自己也在栈,值就是 sp+12 这个地址
addi t0, sp, 12
sw   t0, 8(sp)

# *p = 100;
li   t1, 100
lw   t0, 8(sp)          # 取出 p
sw   t1, 0(t0)          # 写 *p
```

> 🔑 **指针不是"特殊的变量",它就是一个"内容是地址的整数"。**这是 C 与所有"高层语言"最大的区别——它把"内存地址"当成一等公民。

17.4 数组、字符串其实都是指针

```
int  a[10];          // 数组名 a 等价于 &a[0]
a[3]  ≡  *(a + 3)    // 完全等价
"abc"                // 类型是 const char*,指向只读区
```

> ⚠️ **C 字符串是用 `\0` 结尾的字节串。**这是数十年内存安全 bug(buffer overflow、off-by-one)的来源。 `strlen("abc") = 3` ,但其实占 4 字节。

17.5 结构体:把多个变量打包

```
struct Point {
    int x;
    int y;
};

struct Point p = {1, 2};
p.x = 10;            // 等价于 *(int*)((char*)&p + 0)  = 10
p.y = 20;            // 等价于 *(int*)((char*)&p + 4)  = 20
```

结构体就是"按字段偏移读写的连续内存块"。每个字段在编译时确定一个**偏移量**,运行时**没有任何元数据**——这正是 C 的极简哲学。

> 💡 这与 Python/Java 的对象不同——后者通常带元数据(类型、引用计数、虚函数表)。

17.6 C 的极简运行时

C 程序运行时几乎"什么都没有":

- **没有垃圾回收**:`malloc` / `free` 自己管。
- **没有动态类型检查**:类型只在编译期检查,运行时是裸内存。
- **没有边界检查**:数组越界 = 你访问到不该访问的内存(undefined behavior)。
- **没有异常**:错误用返回值表示,或者 `setjmp/longjmp`。
- **没有内置字符串/容器**:都是库(`string.h`,或自己写)。

这是 C 的"诅咒"也是"祝福"。**诅咒**是程序员要为内存、错误、边界负全责;**祝福**是 C 程序运行起来"就是汇编",没有任何隐藏开销。

17.7 一段 C 与一段汇编的对应

```
int add(int a, int b) {
    return a + b;
}
```

GCC `-O2`:

```
add:
    add  a0, a0, a1     # 参数 a 在 a0、b 在 a1;返回值放回 a0
    ret
```

两条指令,而且没有压栈、没有 prolog —— GCC 看出函数极简,直接搞定。**这种"看穿编译结果"的能力,就是 C 程序员的力量来源。**

17.8 C 与现代衍生

语言	关键添加	抽象代价
C++	类、模板、RAII、异常	不为不用的特性付费(zero-overhead),但复杂度激增
Objective-C	消息传递、运行时反射	中等开销
Rust	所有权 + 借用检查器	编译期付出,运行期与 C 同档
Go	GC、goroutine、channel	GC 开销,但很多 C 复杂性消失
Zig	显式分配、comptime	接近 C 的简洁,加上现代特性

它们都试图改进 C 的某些痛点,但**没有任何一种能完全取代 C 在系统层的位置。**

17.9 串联到下一章

C 给我们提供了"控制硬件"的所有工具,但要真正用好,需要理解一件事:

> **C 程序的内存模型 —— 栈在哪、堆在哪、全局变量在哪、指针指向哪。**

下一章我们专门解剖 C 程序运行时的**进程地址空间**:这是连接"代码"与"硬件"的关键拼图。

🔍 本章没讲什么

- C 的完整语法(完整本可参考 K&R 或 ISO C 标准)。
- C99 / C11 / C23 新增特性。
- Undefined Behavior 完整列表。

📚 想深入读什么

- Kernighan & Ritchie, *The C Programming Language* (K&R) — 最经典,薄而深。
- Bryant & O'Hallaron, *CSAPP* — 把 C 与体系结构联在一起。
- Robert Seacord, *Effective C*, *Secure Coding in C and C++*。

← 上一章:链接 · → 下一章:内存模型

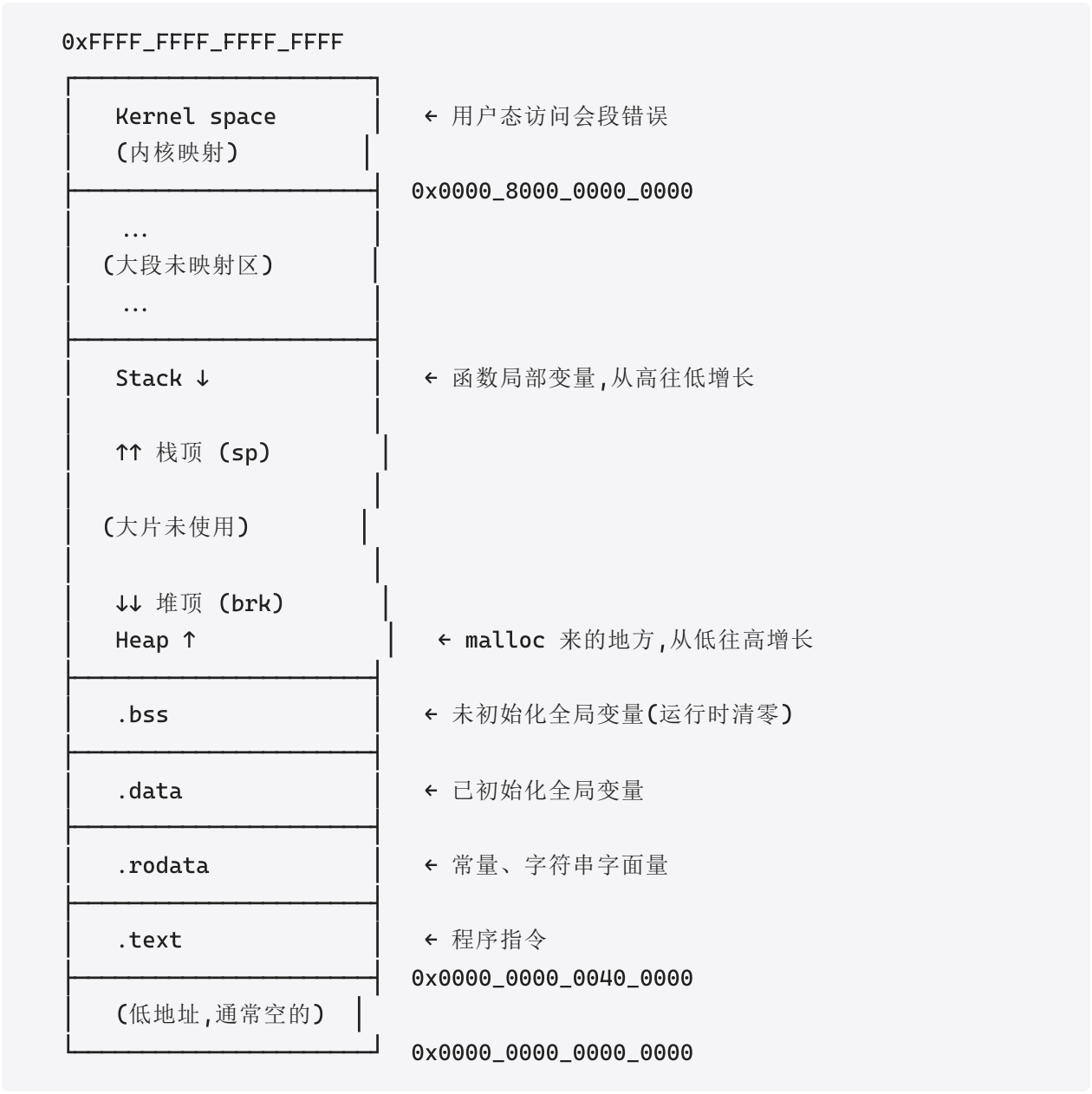
第 18 章 · 内存模型:栈、堆、指针、地址空间

> 一个进程的内存,是它的"私人宇宙"。 弄清这个宇宙的地图,是写好 C(以及调试任何程序)的基础。

18.1 进程的虚拟地址空间

每个 Linux 进程启动时,内核给它一份**4 GB(32 位)/ 256 TB(64 位)** 的虚拟地址空间——是的,**每个进程都以为整片地址空间都属于自己。**

典型布局(64 位 Linux,从高地址往低排):



关键观察:

- **栈和堆相向生长**,中间留出大量空间,以应对增长。
- `.text` 是只读 + 可执行,你写它会段错误。
- `.rodata` 只读,**写字符串字面量也会段错误**(`*"hello"='H'` 是 UB)。
- 堆栈共享 4 GB 虚拟空间,但物理上只用到的部分会被映射(按页 4KB / 2MB / 1GB)。

18.2 栈(Stack):自动管理的"局部记忆"

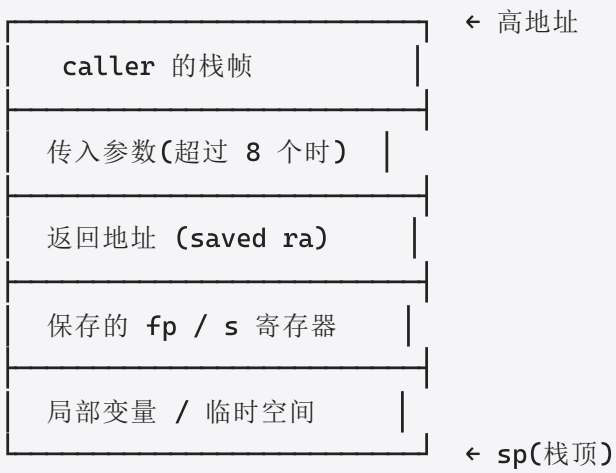
栈是函数调用的"草稿纸":

- 每次函数调用,**push 一个栈帧(stack frame)**,放它的局部变量、参数、返回地址。

- 函数返回时,**pop 栈帧,内存自动回收。**
- 速度极快(只是 sp 的加减)。
- 大小有限(通常 8 MB,可改),递归太深会**stack overflow。**

```
void foo(int x) {
    int  arr[1000];       // 4000 字节,直接在栈上
    char buf[256];        // 256 字节,在栈上
    bar(arr);
}                         // foo 返回时,这 4256 字节自动消失
```

栈帧典型结构(从高到低):



每次 `addi sp, sp, -32`(开栈)、`addi sp, sp, 32`(关栈)就是在这里发生的。

18.3 堆(Heap):程序员手动管理的"长期记忆"

```
int *p = malloc(sizeof(int) * 1000);    // 4000 字节,从堆上分配
p[0] = 42;
free(p);                                 // 必须手动释放
```

- `malloc / free` 由 C 标准库提供(实际上向 OS 要内存: `brk` / `mmap`)。
- 堆速度比栈慢得多(分配器要查找空闲块,释放要合并)。
- **生命周期由程序员决定**——能跨函数、跨线程使用。
- 忘记 `free` → **内存泄漏(memory leak)**; `free` 后还用 → **use-after-free**; `free` 两次 → **double free。**

> 💡 **这就是为什么人们想要 GC、想要 Rust 的所有权系统**——手动堆管理是 C 程序最大的 bug 来源之一。

一个 malloc 实现的简化原理

```
struct block {
    size_t size;
    int    free;
    struct block *next;
};
```

分配器维护一个空闲块链表(free list),`malloc` 找一个够大的、`free` 把块标 free 并合并相邻空闲块。 工业级实现(glibc 的 `ptmalloc`、Google 的 `tcmalloc`、Facebook 的 `jemalloc`)用了 size class、bin、线程缓存等高级技巧。

18.4 数据段:全局/静态变量

```
int g1 = 10;              // 在 .data:已初始化
int g2;                   // 在 .bss:未初始化(但 C 标准保证读到 0)
static int s = 42;        // 在 .data
```

它们的生命周期 = 整个程序。地址在编译/链接期就基本确定。

18.5 .text 段:代码,但也是"数据"

代码段在内存里**也是字节,只不过 CPU 把它当指令读。**这就是为什么:

- 函数指针 `void (*fp)() = some_function;` 是合法的——它就是一个代码段地址。
- JIT 编译器(JVM、V8)可以**动态生成机器码,然后跳过去执行**(需要把内存标 RWX)。
- 同样,**Buffer Overflow 攻击**可以把恶意机器码写到栈上,然后劫持执行流跳过去。
- 现代 OS 用 **DEP(Data Execution Prevention)** 阻止数据段执行,但攻击者用 **ROP(Return-Oriented Programming)** 绕过。

18.6 段错误(Segmentation Fault)的本质

- 访问没映射的地址 → 段错误。
- 写只读区(.text、.rodata) → 段错误。
- 用户态访问内核地址 → 段错误。

CPU 通过 **MMU + 页表** 查每一次访问,如果页表里没有映射,触发**缺页异常**,跳到内核的处理函数,通常就是给你发 SIGSEGV → 进程死。

> 🔑 **「段错误」不是一种神秘 bug,而是 CPU 在保护我们**——避免一个野指针毁掉整个系统。

18.7 一个完整的 C 内存观察

```
#include <stdio.h>
#include <stdlib.h>

int g_init = 1;          // .data
int g_uninit;            // .bss

int main(int argc, char **argv) {
    int local = 2;       // 栈
    static int s = 3;    // .data
    int *p = malloc(8);  // 堆
    const char *str = "hi"; // 指针在栈,内容在 .rodata

    printf("g_init   @ %p\n", &g_init);
    printf("g_uninit @ %p\n", &g_uninit);
    printf("s        @ %p\n", &s);
    printf("local    @ %p\n", &local);
    printf("p        @ %p (heap data @ %p)\n", &p, p);
    printf("str      @ %p (text @ %p)\n", &str, str);
    printf("main     @ %p\n", main);

    free(p);
    return 0;
}
```

运行后你会看到一个清晰的地址分层,**和 18.1 那张图完全对应。**

18.8 串联到下一章

我们已经有了完整的 C 程序运行时画面。下一章我们会做一个特别有教育意义的练习:

> **拿一段 C 代码,逐行看它对应的汇编**—— `if` / `while` / `for` / `struct` / `*p` / `a[i]` 各自变成什么。

这一步会把"C → 硬件"的链路彻底打通,完成本部分的最后一块拼图。

🔍 本章没讲什么

- TLS(线程局部存储)布局。
- ASLR(地址空间随机化)。
- 共享内存 / mmap 文件映射的特殊段。

📚 想深入读什么

- *CSAPP* 第 9 章 — 虚拟内存的最佳教学。
- *Linux Programming Interface* by Kerrisk — 进程内存的细节圣经。

← 上一章:C 语言 · → 下一章:从 C 到汇编

第 19 章 · 从 C 到汇编:每一行代码的代价

> 让我们把 C 的每一种结构,亲手"翻译"一遍。 看完这一章,你写 C 时眼前会自动浮现汇编。

19.1 工具:GCC + objdump 看汇编

随时可以这样查看一段 C 的汇编(以 RISC-V 为例,x86 类似):

```
gcc -O0 -S example.c -o example.s     # 生成汇编(无优化)
gcc -O2 -S example.c -o example.s     # 优化后的汇编

# 或者直接看二进制反汇编
gcc -O0 example.c -o example
objdump -d example | less
```

学 C 不看汇编,等于学武功不照镜子——你不知道每个动作做对了没。

19.2 if-else 的汇编

```
int abs(int x) {
    if (x < 0)
        return -x;
    else
        return x;
}
```

`-O0` 输出(注释为伪码):

```
abs:
    addi sp, sp, -16
    sw   a0, 12(sp)         # 保存 x
    lw   t0, 12(sp)
    bge  t0, x0, .L_else    # if (x ≥ 0) goto else
    lw   t0, 12(sp)
    sub  a0, x0, t0         # return -x
    j    .L_end
.L_else:
    lw   a0, 12(sp)         # return x
.L_end:
    addi sp, sp, 16
    ret
```

关键观察: `if` 的本质是**比较 + 条件分支**。 `bge / blt / beq` 等指令直接对应 `if`。

19.3 while / for 的汇编

```
int sum(int n) {
    int s = 0;
    for (int i = 1; i ≤ n; i++)
        s += i;
    return s;
}
```

汇编(伪码注释):

```
sum:
    li   t0, 0          # s = 0
    li   t1, 1          # i = 1
.L_loop:
    bgt  t1, a0, .L_end # if (i > n) goto end
    add  t0, t0, t1     # s += i
    addi t1, t1, 1      # i++
    j    .L_loop
.L_end:
    mv   a0, t0
    ret
```

循环 = 反向跳转。任何循环最终都会展开成"标签 + 条件分支 + 无条件跳回"。

19.4 switch-case 的汇编(跳转表)

```
int f(int x) {
    switch (x) {
        case 0: return 10;
        case 1: return 20;
        case 2: return 30;
        case 3: return 40;
        default: return -1;
    }
}
```

GCC 高优化时,会生成一个**跳转表**(Jump Table):

```
    # 假设 x 在 a0
    li   t0, 4
    bgeu a0, t0, .L_default
    la   t1, .L_jt          # 跳转表基址
    slli t2, a0, 2          # x * 4
    add  t1, t1, t2
    lw   t1, 0(t1)
    jr   t1                 # 间接跳转

.L_jt:
    .word .L0, .L1, .L2, .L3
.L0: li a0, 10; ret
.L1: li a0, 20; ret
.L2: li a0, 30; ret
.L3: li a0, 40; ret
.L_default: li a0, -1; ret
```

跳转表把 N 路 if-else 变成 1 次间接跳转——这是为什么 `switch` 比串联 `if-else if` 快得多。

19.5 函数调用 + 栈帧

```
int add(int a, int b) { return a + b; }

int main() {
    int x = add(3, 4);
    return x;
}
```

```
add:
    add  a0, a0, a1
    ret

main:
    addi sp, sp, -16
    sw   ra, 12(sp)          # 保存返回地址
    li   a0, 3               # 第一个参数
    li   a1, 4               # 第二个参数
    jal  ra, add             # 调用 add,返回地址存 ra
    sw   a0, 8(sp)           # 把返回值存到 x(局部变量)
    lw   ra, 12(sp)          # 恢复 ra
    addi sp, sp, 16
    ret
```

函数调用的成本: 1. 在栈上开辟空间(`addi sp, sp, -N`)。 2. 保存 `ra` 和需要保留的寄存器。 3. 准备参数到 `a0-a7` 。 4. `jal` 跳转。 5. 返回后恢复寄存器、回收栈空间。

> 💡 这就是为什么 **`inline` 关键字能加速**——内联消除了上述全部开销。当然,代价是代码膨胀。

19.6 数组访问

```
int arr[100];
int x = arr[i];
```

```
    la    t0, arr           # t0 = &arr
    slli  t1, a0, 2         # i * sizeof(int) = i * 4
    add   t0, t0, t1
    lw    a0, 0(t0)         # x = *(arr + i*4)
```

`arr[i]` = 「基址 + 偏移 × 元素大小」的 load,恰好对应 RISC-V 的 `lw rd, offset(rs1)`。

19.7 结构体访问

```
struct Point { int x, y; };
struct Point p;
int v = p.y;
```

GCC 在编译时已知 `y` 的偏移是 4:

```
    la   t0, p
    lw   a0, 4(t0)          # 直接读 p+4
```

结构体字段访问是零成本的——只是一个固定偏移的 load。

19.8 指针的间接访问

```
int *p = &arr[5];
int v = *p;
```

```
    la   t0, arr
    addi t0, t0, 20         # &arr[5]
    sw   t0, 8(sp)          # p 存到栈
    lw   t1, 8(sp)          # 读 p
    lw   a0, 0(t1)          # *p
```

`*p` = 一次 load,本身没什么开销。但**间接访问可能 cache miss**,这才是性能瓶颈所在(见 Ch 13)。

19.9 优化的力量(`-O2` 之后)

把 `sum` 用 `-O2` 编译,GCC 会做一件神奇的事:

```
sum:
    # GCC 发现这是 1+2+...+n,直接套求和公式!
    addi  a1, a0, 1
    mul   a0, a0, a1
    srli  a0, a0, 1
    ret
```

—— **整个循环消失了**!这就是现代编译器的力量。它能做的优化:

- **常量折叠**: `3 + 4` 直接变成 `7`。
- **死代码消除**:写了从不被读的变量被删掉。
- **循环展开 / 向量化**:把 4 次循环变成 1 条 SIMD 指令。
- **公共子表达式消除**: `a*b` 算两次只算一次。
- **内联**:小函数被嵌进调用处。
- **尾递归优化**:递归变循环。

> 💡 **写性能关键代码,要看 -O2 的汇编,而不是 -O0。**前者代表真实运行的形态。

19.10 这一部分的总结

我们打通了:

```
C 源码 → C 抽象 → 内存模型 → 汇编 → 机器码 → CPU → 硅
```

至此 **从硅到 C 的完整链路打通。**任何 C 程序的执行,都可以追溯到 NAND 门的翻转。

接下来 Part VII 我们要往**更高的抽象**走:**变量、对象、闭包、泛型**——这些"高级语言特性",如何被翻译成 C 一样能被编译/解释的形式。

🔍 本章没讲什么

- 浮点/SIMD 指令对应。
- C++ 的虚函数调用对应(vtable + indirect call)。
- 异常机制(setjmp/longjmp、零成本异常)的汇编展开。

📚 想深入读什么

- *CSAPP* 第 3 章 — x86-64 视角的 C → 汇编。
- Compiler Explorer (godbolt.org) — 在线看任意编译器/优化级别的汇编。**强烈推荐做实验。**

← 上一章:内存模型 · → 下一章:变量与类型

第 20 章 · 变量、表达式、类型系统

> 高级语言把"内存里的字节"装扮成"有名字、有类型的东西"。 这一章拆穿这套装扮。

20.1 变量到底是什么?

不同抽象层次的视角:

视角	「变量 x」是什么
Python 程序员	一个名字,绑定到一个对象
C 程序员	一片有名字的内存(几字节)
汇编程序员	一个寄存器,或一个栈偏移
硬件工程师	一组 D 触发器 + 译码逻辑

所有上层抽象,都建立在底层"几个比特保存某个数值"这个事实上。变量名只是给人看的——它在编译/链接后通常就消失了。

20.2 静态变量 vs 动态变量

静态:绑定在编译期

```
int x;          // x 的地址、大小、类型,编译期已知
```

动态:绑定在运行期

```
x = 1
x = "hello"     # 同一个名字,先指向 int,后指向 str
```

Python 的"变量"是字典里的 key:

```
namespace = {
    "x": <ref to int 1>      # 后变成 <ref to str "hello">
}
```

每次 `x = ...` 实际是修改这个字典。**这就是动态语言慢的根源之一**——简单的赋值要查表、做引用计数。

20.3 类型系统的三个维度

类型系统有三个独立的维度,**它们经常被混淆**:

维度	选项	典型语言
静态 vs 动态	类型在编译期 vs 运行期决定	C / Java vs Python / JS
强 vs 弱	是否允许隐式转换	Python(强) vs JS(弱)
显式 vs 推导	是否必须写出类型	Java vs Haskell / Rust

例如:

- **C**:静态、弱、显式(`int` → `char` 自动转,但要写出来)。
- **Python**:动态、强、推导(类型不写,但 `1 + "abc"` 直接报错)。
- **JS**:动态、弱、推导(`1 + "1"` = `"11"`,默默转字符串)。
- **Haskell**:静态、强、推导(完整 Hindley-Milner)。
- **Rust**:静态、强、推导(局部范围内推断,函数签名要显式)。

20.4 类型在底层是怎么编码的?

C / Rust(静态语言)

类型只在编译期存在。运行时**纯字节**——一个 `int` 就是 4 字节,没有"类型标签"。

Python(动态语言)

每个对象在内存里都带一个**类型指针**:

```
struct PyObject {
    Py_ssize_t  ob_refcnt;      // 引用计数
    PyTypeObject *ob_type;      // ← 类型指针
    /* 类型相关数据 */
};
```

每次操作 `a + b`,解释器:

1. 查 a.ob_type 找到加法函数指针 `nb_add`。
2. 查 b.ob_type 同样找。
3. 选其中一个,调用。

—— 这就是为什么 CPython 比 C 慢 50-100 倍。**所有数据访问都多一次间接、多一次类型分派。**

20.5 表达式求值:还是树!

`a + b * c` 不是从左到右算的,而是按运算符优先级构成一棵**抽象语法树**(AST):

```
      +
     / \
    a   *
       / \
      b   c
```

求值方式 = 后序遍历这棵树:

1. 求 `b` (读内存)
2. 求 `c` (读内存)
3. 算 `b * c`
4. 求 `a`
5. 算 `a + (b*c)`

所有编程语言的"表达式"本质上都是这种树。这棵树的构建,就是编译/解释的第一步——见 Part VIII。

20.6 强类型不是「不能转」,是「不会偷偷转」

```
# Python(强)
1 + "abc"         # TypeError!
int("1") + 1      # OK,但你显式调了 int()

# JavaScript(弱)
1 + "1"           # "11" —— 静默把 1 变成 "1"
[] + {}           # 谁知道呢
```

强类型的好处是**bug 不会偷偷溜过去**,坏处是要显式转换、写起来啰嗦。

20.7 类型推导:写得少,但还是静态的

```
let x = 5;           // Rust 自动推出 x: i32
let v = vec![1, 2]; // 推出 Vec<i32>
```

Rust / Haskell 的类型系统强大到能从用法**反推**类型。这对程序员是甜头(少打字)、对编译器是任务(算法是 Hindley-Milner 的变种)。

20.8 类型作为合同与文档

```
fn parse_csv(input: &str) → Result<Vec<Record>, ParseError>
```

这一个签名告诉你:

- 不会修改输入。
- 可能成功(返回 `Vec<Record>`)或失败(返回 `ParseError`)。
- **失败一定要被处理**(Rust 强制 `Result` 的接收方处理)。

强静态类型系统就是**机器可检的文档**。这是为什么大型项目从动态语言迁移到 TypeScript / Rust / Go 的根本动机。

20.9 串联到下一章

我们看清了"变量"和"类型"。下一章看**控制流**——`if`、`for`、函数、递归——它们在不同语言里有什么共性,以及怎么落到 CPU 的分支指令。

最后会看到一个特别有趣的事实:**所有控制流,本质上都能归结为「条件分支 + 跳转」**——也就是 CPU 提供的两种基本能力。

🔍 本章没讲什么

- 完整的类型理论(代数数据类型、依赖类型、线性类型)。
- 多态(parametric / ad-hoc / subtype)。
- 类型擦除(Java、TypeScript)。

📚 想深入读什么

- *Types and Programming Languages* (TAPL) by Pierce —— 类型理论圣经。
- *Programming Language Pragmatics* by Scott —— 各种语言机制的横向对比。

← 上一章:从 C 到汇编 · → 下一章:控制流

第 21 章 · 控制流:条件、循环、函数、递归

> 所有控制流,本质上都是「条件分支 + 跳转 + 栈」的组合。 这一章把 if/for/while/递归 全部"还原"。

21.1 一切控制流的两块基石

CPU 只提供两种"控制流"原语:

1. **条件分支(Branch)**: `if cond goto label`
2. **无条件跳转(Jump)**: `goto label` (包括函数调用 `jal` 和返回 `ret`)

加上**栈**(用于函数调用返回),**任何高级语言的控制流,都能由这三样组合而来。**

21.2 条件:if-else / 三元 / 模式匹配

```
# Python
if x > 0: y = 1
elif x == 0: y = 0
else: y = -1
```

```
// Rust 的模式匹配
let y = match x.cmp(&0) {
    Ordering::Greater ⇒ 1,
    Ordering::Equal   ⇒ 0,
    Ordering::Less    ⇒ -1,
};
```

```
-- Haskell
y = case compare x 0 of
        GT → 1
        EQ → 0
        LT → -1
```

不同语言的语法五花八门,但**底层全部展开成"几次比较 + 几次条件跳转"**——上一章我们看到的 `bge / blt / beq`。

21.3 循环:while / for / do-while / 迭代器

C 风格的 for

```
for (int i = 0; i < n; i++) body(i);
```

等价于:

```
int i = 0;
while (i < n) {
    body(i);
    i++;
}
```

Python 的 for(本质是迭代器协议)

```
for x in iterable:
    body(x)
```

展开:

```
it = iter(iterable)
while True:
    try:
        x = next(it)
    except StopIteration:
        break
    body(x)
```

—— Python 的 `for` 不是 C 的 for,它是**遍历迭代器**。

函数式语言:不用循环,用递归 / 高阶函数

```
sum [1..10]                    -- 直接求和
map (*2) [1, 2, 3]             -- [2, 4, 6]
foldl (+) 0 [1, 2, 3]          -- 6
```

但**底层依然是 CPU 的循环或递归**——只是被语言的运行时藏起来了。

21.4 函数 = 跳转 + 栈帧 + 调用约定

任何函数调用,**机器层面发生的事**:

1. 把参数放到约定位置(寄存器 / 栈)。
2. 把返回地址保存到 `ra`。
3. `jal` 跳到函数入口。
4. 函数干活;若调别的函数,要保存 `ra` 到栈。
5. 把返回值放到约定位置。
6. `ret` (= 跳到 `ra` 指的地方)。

这就是 ABI 的本质——所有人都遵守这个协议,不同语言写的函数才能互相调用(C ↔ Rust ↔ Python C 扩展……)。

21.5 递归:用栈代替循环

```
int fact(int n) {
    if (n ≤ 1) return 1;
    return n * fact(n - 1);
}
```

`fact(5)` 时栈的样子(从底往上):

```
main 帧
fact(5) 帧:  n=5,等 fact(4)
fact(4) 帧:  n=4,等 fact(3)
fact(3) 帧:  n=3,等 fact(2)
fact(2) 帧:  n=2,等 fact(1)
fact(1) 帧:  n=1,返回 1
```

每次返回时,栈帧弹出,上一帧拿到结果继续算。这就是**递归用栈的本质**。

> ⚠️ 递归太深 = stack overflow。深度依赖输入时,要么手动转循环,要么开大栈,要么用尾递归(下一节)。

21.6 尾递归优化(TCO):递归不耗栈的魔法

```
// 尾递归版本
int fact_tail(int n, int acc) {
    if (n ≤ 1) return acc;
    return fact_tail(n - 1, n * acc);     // 递归调用是函数最后一个动作
}
```

如果递归调用是**函数最后一件事**,编译器可以把它**重写成跳转**——不开新栈帧!这就是**尾调用优化**:

```
fact_tail:
.L_loop:
    li   t0, 1
    bge  t0, a0, .L_done    # if (n ≤ 1) return acc
    mul  a1, a0, a1         # acc = n * acc
    addi a0, a0, -1         # n = n - 1
    j    .L_loop            # 不是 jal,只是 j
.L_done:
    mv   a0, a1
    ret
```

—— **尾递归被自动展平成循环**!这就是 Lisp / Scheme 鼓励"递归代替循环"的底气。

21.7 异常 / try-catch:非局部控制流

```
try:
    risky()
except ValueError as e:
    handle(e)
```

异常的本质是**非局部跳转**——可以从函数内部直接跳到调用栈中某个 `catch`,跳过中间所有栈帧。

实现方式:

- **C 中的 `setjmp / longjmp`**:手动版本,慢但通用。
- **零成本异常表(Zero-Cost Exceptions)**:C++ / Rust 用的方案。正常路径无开销;只有抛异常时才查"栈展开表"找 catch。
- **Java 异常**:JVM 维护栈帧,可以遍历找 handler。

> 💡 "零成本"是指不抛异常时无开销;真抛起来其实**比手动错误码还慢**——这就是为什么 Rust 偏爱 `Result<T, E>`。

21.8 协程 / async-await:用户态的"轻量栈帧"

```
async def fetch(url):
    data = await http_get(url)
    return parse(data)
```

`await` 本质是:

1. 暂停当前函数,**保存它的"局部状态"**(变量、PC)到一个对象里。
2. 把控制权返回到调度器。
3. 当 IO 就绪,调度器**恢复**这个状态,继续执行。

实现方式有两种:

- **栈式协程**(Goroutine、Lua coroutine):每个协程有自己的小栈(通常几 KB),靠汇编做"栈切换"。
- **无栈协程 / state-machine**(Rust / C# / Python async):编译器把 async 函数**重写成一个状态机**,`await` 切换的是"状态字段",根本没有栈切换。

无论哪种,**底层依然是"保存现场、跳走、恢复现场"**——和 OS 切换进程是同一思路,只是粒度更轻。

21.9 串联到下一章

控制流之上,还有**抽象机制**:作用域、闭包、对象、泛型。它们让程序员能把控制流和数据"打包"成可重用的单元。下一章我们就拆这些——**面向对象、函数式、泛型**,在底层都是怎么实现的。

🔍 本章没讲什么

- Continuation Passing Style (CPS)。
- 协程的具体调度算法(Go 的 GMP、Tokio 的 work-stealing)。
- 长跳转、错误处理 vs 异常的对比。

📚 想深入读什么

- *Structure and Interpretation of Computer Programs* (SICP) — 控制流哲学的奠基书。
- Russ Cox, *libtask* / 各种 coroutine 实现源码 —— 看实战。

← 上一章:变量与类型 · → 下一章:抽象机制

第 22 章 · 抽象机制:作用域、闭包、对象、泛型

> 高级语言之所以"高级",因为它能把代码 + 数据,打包成可复用的单元。 这一章拆穿四种最常见的抽象——它们底层都是 C 能做的事,只是被语言运行时优雅地隐藏了。

22.1 作用域:名字解析的规则

```
x = 1
def foo():
    x = 2           # 这是局部变量,不是改外面的 x
    print(x)        # 2
foo()
print(x)            # 1
```

每个名字 `x` 在语言眼里都对应一个**绑定**(binding),作用域规则决定你看到哪个绑定:

- **词法作用域(Lexical Scope)**:看你的代码"嵌套层次",大多数现代语言都是。
- **动态作用域**:看运行时调用栈(古老的 Lisp、bash 变量等)。

实现:编译器/解释器维护一个**符号表(Symbol Table)栈**——每进入一个块就 push 一层、每离开就 pop。查名字时从栈顶往下找。

22.2 闭包:把"环境"装进函数对象

```
def make_counter():
    count = 0
    def step():
        nonlocal count
        count += 1
        return count
    return step

c = make_counter()
c(), c(), c()    # 1, 2, 3
```

`step` 函数已经返回了它所在的 `make_counter` 函数,但**它仍然能访问 `count`**——这种"函数 + 它所捕获的环境",叫**闭包**。

闭包的底层实现

闭包不是魔法,本质上就是:

```
struct Closure {
    void (*func)(struct Closure *env, ...);    // 函数指针
    /* 捕获的变量(被提升到堆上) */
    int *captured_count;
};
```

调用闭包时,把 `Closure*` 当成隐含的第一个参数传进函数。Rust、C++ Lambda、Go 都用类似机制:

```
// Rust 闭包
let mut count = 0;
let step = || { count += 1; count };    // 编译成一个匿名 struct + impl FnMut
```

> 🔑 **关键洞察**:闭包之所以变量不会随栈帧销毁而消失,是因为编译器/运行时**把它们提升到堆**。这就是为什么 Python / JS 创建闭包比 C 慢——多一次堆分配。

22.3 面向对象:数据 + 函数指针 + 继承

对象的本质

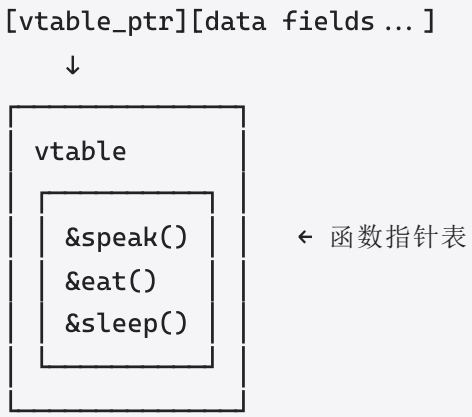
C 风格的"对象":

```
struct Animal {
    int    age;
    char  *name;
    void (*speak)(struct Animal *self);    // 函数指针
};
```

这就够了。**所谓"对象方法",其实是"第一个参数是 self 的函数"**。Python 的 `self`、C++ 的 `this`、Rust 的 `&self` 全部都是这一回事。

虚函数 / 多态

C++ 类的对象在内存里通常长这样:



调用 `obj→speak()` 时: 1. 读 obj 的第一个字段(vtable_ptr)。 2. 读 vtable 的第 0 项(speak 的函数指针)。 3. 调用,把 obj 当 this 传进去。

这种"两次间接 + 间接 call"在 CPU 上被叫做**间接分支(indirect branch),比直接调用慢**(分支预测器很难猜中目标)。这就是 OOP 的运行时代价。

继承

子类"继承"父类,本质是**子类的内存布局把父类的字段放在最前**,并提供一份新的 vtable(可以覆盖某些项):

```
Animal:    [vtable*][age][name]
Dog:       [vtable*][age][name][breed]
             ↑                    ↑
            指向 Dog 自己的 vtable   Dog 加的字段
```

向上转型(`Dog* → Animal*`)是免费的(地址不变);向下转型需要 RTTI 检查。

22.4 接口 / Trait / Protocol:统一的"行为合同"

不依赖继承,只看"能做什么":

```
trait Speak {
    fn speak(&self);
}

impl Speak for Dog { fn speak(&self) { println!("woof"); } }
impl Speak for Cat { fn speak(&self) { println!("meow"); } }

fn shout(x: &dyn Speak) {        // 任何实现了 Speak 的类型
    x.speak();
}
```

底层实现:**胖指针(Fat Pointer)**——一个普通指针 + 一个 vtable 指针:

```
&dyn Speak  =  (data_ptr, vtable_ptr)
```

调用 `x.speak()` 时,从 vtable 找到 speak 函数,把 data_ptr 当 self 传进去。

> 💡 这与 C++ 虚函数的差别在于:**vtable 不是嵌在对象里,而是单独的指针对**。这让对象可以同时"实现多个 trait",而不必把 vtable 都塞进对象。

22.5 泛型:同一份代码,多种类型

```
fn max<T: Ord>(a: T, b: T) -> T {
    if a > b { a } else { b }
}

max::<i32>(3, 5);
max::<String>("hello".to_string(), "world".to_string());
```

实现策略:

策略	谁用	怎么做
Monomorphization(单态化)	Rust / C++ 模板	编译期为每种 T 生成一份机器码
类型擦除(Type Erasure)	Java / TypeScript	编译时检查,运行时全是 `Object` (JVM) 或啥都没有(TS)
运行时分发	Python / Smalltalk	没有泛型,反正运行时类型决定一切

单态化 性能最好(零开销),但代码膨胀;**擦除** 文件小、灵活,但有装箱/拆箱开销。

22.6 元编程:让代码生成代码

```
// C 宏:文本替换
#define MAX(a, b) ((a) > (b) ? (a) : (b))
```

```
// Rust 宏:语法树级
macro_rules! max {
    ($a:expr, $b:expr) => { if $a > $b { $a } else { $b } };
}
```

```
# Python 装饰器:运行时元编程
@cache
def fib(n): ...
```

元编程的本质是:**程序在编译/运行期"操作另一段程序"**。这种能力越强,语言就越灵活,但也越难调试。

22.7 Part VII 总结:抽象的代价

抽象	运行时代价
静态类型	几乎为零(编译期完成)
动态类型	类型分派 + 类型对象间接,~10x
单态化泛型	零(编译时展开)
类型擦除泛型	装箱/拆箱开销
闭包	一次堆分配 + 一次间接调用
OOP 虚函数	两次间接 + 难以预测的分支
异常	不抛时近零;真抛时很慢
GC	周期性卡顿,平均 5-20%

所有"高级"特性,都用一些运行时代价换取了开发效率。设计语言就是在这个空间里取舍。

22.8 串联到 Part VIII

我们已经看清了"高级语言里有哪些抽象",但还没回答最关键的问题:

> **这些抽象,是怎么从源代码,变成 CPU 能执行的指令的?**

答案有三种:

- **编译(Compile)**:像 C/Rust/Go,把源码翻译成机器码,然后直接执行。
- **解释(Interpret)**:像 Python/Ruby,逐句翻译并执行,从不生成机器码文件。
- **混合**:像 Java/C#,先编译成字节码,再用 JIT 在运行时编译成机器码。

下一部分(Part VIII)我们就深入这个最神奇的领域——**编译器与解释器。**

🔍 本章没讲什么

- 高阶类型(HKT)、依赖类型、AC tactics。
- 反射 API 的运行时实现。
- ABI 兼容性问题(C++ name mangling、Rust 不稳定 ABI)。

📚 想深入读什么

- *SICP* —— 用 Scheme 实现 OOP / 闭包,极具启发。
- *Crafting Interpreters* —— 自己写一门小语言,所有抽象一目了然。

← 上一章:控制流 · → 下一章:编译原理总览

第 23 章 · 编译原理总览:词法 → 语法 → 语义 → IR → 目标代码

> 🌉 **桥 4** —— 这是从"人类思想"到"电子运动"的最后一座桥。 编译器是计算机科学的"皇冠明珠"。

23.1 编译器的"五段式流水线"

任何严肃的编译器都长这样:

```
源代码 (一段文本)
    │
    ▼
① 词法分析 (Lexer)          → Tokens
    │
    ▼
② 语法分析 (Parser)         → AST (抽象语法树)
    │
    ▼
③ 语义分析 (Sema)           → 检查类型、作用域,丰富 AST
    │
    ▼
④ 中间代码生成 (IR Gen)     → IR (Intermediate Representation)
    │
    ▼
⑤ 优化 (Optimization)       → 优化后的 IR
    │
    ▼
⑥ 目标代码生成 (Codegen)    → 汇编 / 机器码
```

> 💡 **是不是觉得眼熟?** 这和 Ch 9 的 EDA 流程完全对偶! RTL → 通用门 → 工艺映射 ↔ AST → IR → 机器码。 **编译器和综合器,是同一个抽象的两种应用。**

23.2 ① 词法分析:把字符流变成 Token

输入:

```
int x = 42 + y;
```

Lexer 输出 Token 序列:

```
[KEYWORD "int"] [IDENT "x"] [PUNCT "="] [NUM 42] [PUNCT "+"] [IDENT "y"] [PUNCT
```

实现:**正则** → **DFA**(确定有限自动机)。每个 Token 类别对应一个正则,Lexer 跑一个状态机匹配。

工具: `flex` (C)、 `logos` (Rust)、Python 的 `re` 自己写。

23.3 ② 语法分析:把 Token 序列变成 AST

输入:上一步的 Token 流。 输出:**抽象语法树(AST)。**

```
        Decl(int)
        /       \
       x         BinOp(+)
                /        \
              42          y
```

实现方法:

- **递归下降(Recursive Descent)**:每个语法规则对应一个函数。最直观,实战中最常用。
- **LL(1) / LL(k)** 表驱动:工具 ANTLR、PLY。
- **LR / LALR**:工具 yacc/bison。功能强,但调试痛苦。

> ⚠️ **写编译器最容易栽在 parser**——优先级、结合性、二义性都是坑。但只要 AST 对了,后面就顺利了。

一个简单的递归下降片段(Python)

```
def parse_expr():
    left = parse_term()
    while peek() in ('+', '-'):
        op = consume()
        right = parse_term()
        left = BinOp(op, left, right)
    return left
```

23.4 ③ 语义分析:类型与作用域

到这里 AST 在结构上是对的,但**还可能违反语言规则**:

- 用了未声明的变量?
- 类型不匹配(`int + str`)?
- 函数调用参数个数不对?
- 在 `const` 变量上写?

Sema 阶段做这些检查,**同时给 AST 节点打上类型标注**,为后续 codegen 做准备。

实现:**遍历 AST + 维护符号表(scope chain)。**

> 💡 在静态语言里,**Sema 就是最严苛的"类型守门员"**——它确保运行时不会出现"类型不匹配"。Rust 的 borrow checker 就发生在这一阶段。

23.5 ④ 中间代码(IR):语言无关、平台无关的中间形式

为什么要 IR?——**为了不写 N×M 个编译器。**

```
不用 IR:    每加一种语言、每加一种 CPU,都要写一个新组合
            (Python on x86, Python on ARM, Rust on x86, Rust on ARM, ...)

用 IR:     N 个前端 ── IR ── M 个后端
            (新语言只写一个前端,新 CPU 只写一个后端)
```

LLVM 是这种架构的"经典实现":Clang、Swift、Rust 编译器、Julia 都用它。

LLVM IR 长这样(看上去就像"高级版的汇编"):

```
define i32 @add(i32 %a, i32 %b) {
entry:
  %sum = add i32 %a, %b
  ret i32 %sum
}
```

特征: - **SSA(Static Single Assignment)**:每个变量只赋值一次,优化算法极易写。 - **强类型**:每个 IR 指令都标注类型。 - **平台无关**:没有具体寄存器,只有"虚拟寄存器" `%a` 、 `%b` 、 `%sum` 。

23.6 ⑤ 优化:在 IR 上"打磨"

经典优化 pass(LLVM 上百种,常见的几个):

优化	干啥
Constant Folding	`3+4 → 7`
Dead Code Elimination	删掉无用语句
Common Subexpression Elimination	重复表达式只算一次
Loop Invariant Code Motion	把循环里"不变的"提到外面
Inlining	小函数嵌进调用处
Vectorization	普通循环 → SIMD
Tail Call Optimization	尾递归 → 循环

每个 pass 读取 IR、改写 IR,再交给下一个 pass。LLVM 的 `opt -O3` 会跑几十个 pass。

> 🔑 **大部分编译器优化的力量都在这一阶段。**-O0 和 -O3 的差距,不是前端的不同,而是这里跑了多少个 pass。

23.7 ⑥ 目标代码生成:IR → 汇编

最后一步:把 IR 翻译成具体 ISA 的汇编。涉及:

- **指令选择**:这条 IR 要用哪几条机器指令(常用 BURG/IBURG 树重写)。
- **寄存器分配**:无限的虚拟寄存器 → 有限的物理寄存器(经典算法:**图着色 / 线性扫描**)。
- **指令调度**:在不改变结果的前提下,重排指令以适应流水线 / 减少 stall。

23.8 完整链路:Hello World

```
int main() { puts("Hello"); return 0; }
```

每个阶段后的产物:

```
源码 (5 行 C)
   ↓ Lexer
Tokens: int, main, (, ), {, puts, (, "Hello", ), ;, return, 0, ;, }
   ↓ Parser
AST: FuncDecl(main, [...]) → CallExpr(puts, "Hello") + Return(0)
   ↓ Sema
带类型的 AST: int main(void) → call(int(*)(const char*) puts, ...) ...
   ↓ IR Gen
LLVM IR:
   define i32 @main() {
     %0 = call i32 @puts(i8* @.str)
     ret i32 0
   }
   ↓ Optimization
(此例无优化空间)
   ↓ Codegen
RISC-V 汇编:
   main:
     addi sp, sp, -16
     sw   ra, 12(sp)
     la   a0, .LC0
     call puts
     li   a0, 0
     lw   ra, 12(sp)
     addi sp, sp, 16
     ret
```

—— **这就是一段 C 源码到机器汇编的完整旅程。**链接器再把它和 libc 拼起来,就有了 ELF 可执行文件;加载器再把 ELF 装进内存;CPU 再开始一个时钟周期一个时钟周期地执行。

整本书的主线在这里闭环了。

23.9 串联到下一章

理论说够了。下一章我们**亲手写一个迷你编译器**——把"算术表达式"编译成一种简单的栈式字节码,自己实现 Lexer / Parser / Codegen。

整个项目几百行 Python,但**它有真编译器的所有"骨架"。**这个练习能让你彻底理解编译。

🔍 本章没讲什么

- 自下而上 vs 自上而下 parser 的细节。
- LLVM 的 PassManager / IR DSL。
- MLIR(多层 IR)的现代趋势。
- JIT 编译器的细节(Ch 25 会讲)。

📚 想深入读什么

- *Engineering a Compiler* by Cooper & Torczon — 现代编译器最好教材之一。
- *Compilers: Principles, Techniques, and Tools*(龙书)— 经典,但偏理论。
- Bob Nystrom, *Crafting Interpreters* — 免费在线,**最好的入门书。**

← 上一章:抽象机制 · → 下一章:迷你编译器

第 24 章 · 编译器实战:写一个迷你编译器

> 几百行 Python,把上一章的理论变成能跑的真东西。 完整代码在 `code/part8-compiler/mini_compiler.py`。

24.1 我们的目标语言:Tiny

支持:

- 整数字面量 + 算术表达式 `+ - * /`
- 变量赋值与读取: `x = 1 + 2 * 3`
- `print` 语句: `print(x)`
- 单行 `;` 分隔多语句

例子:

```
x = 3 + 4 * 2
y = x - 5
print(y)
```

预期输出: `6`。

24.2 目标后端:一个栈式 VM

我们不直接生成 RISC-V 汇编(那要处理寄存器分配),而是生成一种**简单的栈式字节码**,然后写一个 ~20 行的 VM 跑它。这就是 JVM、Python(CPython)、Lua、WebAssembly 的核心思路。

字节码指令集:

指令	操作
`PUSH n`	压一个常量 n 到栈顶
`LOAD name`	从变量表取值压栈
`STORE name`	弹栈顶,存到变量 name
`ADD/SUB/MUL/DIV`	弹两个,运算,压结果
`PRINT`	弹栈顶,打印
`HALT`	停

24.3 完整实现(Python)

完整可运行版本在 `code/part8-compiler/mini_compiler.py`,关键片段如下:

Lexer(词法分析器)

```
import re
TOKEN_SPEC = [
    ("NUM",    r"\d+"),
    ("ID",     r"[A-Za-z_]\w*"),
    ("OP",     r"[+\-*/]"),
    ("ASSIGN", r"="),
    ("LP",     r"\("), ("RP", r"\)"),
    ("SEMI",   r";"),
    ("WS",     r"\s+"),
]
TOKEN_RE = re.compile("|".join(f"(?P<{k}>{p})" for k, p in TOKEN_SPEC))

def tokenize(src):
    pos = 0
    while pos < len(src):
        m = TOKEN_RE.match(src, pos)
        if not m:
            raise SyntaxError(f"unexpected: {src[pos]!r}")
        kind, text = m.lastgroup, m.group()
        pos = m.end()
        if kind ≠ "WS":
            yield (kind, text)
    yield ("EOF", "")
```

Parser(递归下降)+ AST

```
from dataclasses import dataclass

@dataclass
class Num:    value: int
@dataclass
class Var:    name: str
@dataclass
class BinOp:  op: str; left: object; right: object
@dataclass
class Assign: name: str; value: object
@dataclass
class Print:  expr: object

class Parser:
    def __init__(self, toks): self.toks, self.i = list(toks), 0
    def peek(self, k=0): return self.toks[self.i + k]
    def eat(self, kind=None):
        t = self.toks[self.i]
        if kind and t[0] ≠ kind:
            raise SyntaxError(f"expected {kind}, got {t}")
        self.i += 1
        return t

    def parse_program(self):
        stmts = []
        while self.peek()[0] ≠ "EOF":
            stmts.append(self.parse_stmt())
            if self.peek()[0] == "SEMI": self.eat("SEMI")
        return stmts

    def parse_stmt(self):
        t = self.peek()
        if t[0] == "ID" and t[1] == "print":
            self.eat("ID"); self.eat("LP")
            e = self.parse_expr(); self.eat("RP")
            return Print(e)
        if t[0] == "ID" and self.peek(1)[0] == "ASSIGN":
            name = self.eat("ID")[1]; self.eat("ASSIGN")
            return Assign(name, self.parse_expr())
        return self.parse_expr()

    def parse_expr(self):
```

```
        left = self.parse_term()
        while self.peek()[0] == "OP" and self.peek()[1] in "+-":
            op = self.eat("OP")[1]; right = self.parse_term()
            left = BinOp(op, left, right)
        return left

    def parse_term(self):
        left = self.parse_factor()
        while self.peek()[0] == "OP" and self.peek()[1] in "*/":
            op = self.eat("OP")[1]; right = self.parse_factor()
            left = BinOp(op, left, right)
        return left

    def parse_factor(self):
        t = self.peek()
        if t[0] == "NUM": return Num(int(self.eat("NUM")[1]))
        if t[0] == "ID":  return Var(self.eat("ID")[1])
        if t[0] == "LP":
            self.eat("LP"); e = self.parse_expr(); self.eat("RP")
            return e
        raise SyntaxError(f"unexpected {t}")
```

Codegen(AST → 字节码)

```
def compile_node(node, code):
    if isinstance(node, Num):
        code.append(("PUSH", node.value))
    elif isinstance(node, Var):
        code.append(("LOAD", node.name))
    elif isinstance(node, BinOp):
        compile_node(node.left, code)
        compile_node(node.right, code)
        code.append({"+": "ADD","-": "SUB","*": "MUL","/": "DIV"}[node.op])
    elif isinstance(node, Assign):
        compile_node(node.value, code)
        code.append(("STORE", node.name))
    elif isinstance(node, Print):
        compile_node(node.expr, code)
        code.append("PRINT")

def compile_program(stmts):
    code = []
    for s in stmts: compile_node(s, code)
    code.append("HALT")
    return code
```

VM(栈式虚拟机)

```
def run(bytecode):
    stack, env, pc = [], {}, 0
    while True:
        instr = bytecode[pc]; pc += 1
        if instr == "HALT": return
        op = instr[0] if isinstance(instr, tuple) else instr
        if   op == "PUSH":  stack.append(instr[1])
        elif op == "LOAD":  stack.append(env[instr[1]])
        elif op == "STORE": env[instr[1]] = stack.pop()
        elif op == "ADD":   b=stack.pop(); a=stack.pop(); stack.append(a+b)
        elif op == "SUB":   b=stack.pop(); a=stack.pop(); stack.append(a-b)
        elif op == "MUL":   b=stack.pop(); a=stack.pop(); stack.append(a*b)
        elif op == "DIV":   b=stack.pop(); a=stack.pop(); stack.append(a//b)
        elif op == "PRINT": print(stack.pop())
```

Driver

```
src = "x = 3 + 4 * 2; y = x - 5; print(y)"
ast = Parser(tokenize(src)).parse_program()
bc  = compile_program(ast)
for ins in bc: print(" ", ins)
run(bc)        # → 输出 6
```

字节码长这样:

```
  ('PUSH', 3)
  ('PUSH', 4)
  ('PUSH', 2)
  MUL
  ADD
  ('STORE', 'x')
  ('LOAD', 'x')
  ('PUSH', 5)
  SUB
  ('STORE', 'y')
  ('LOAD', 'y')
  PRINT
  HALT
```

24.4 这 200 行里的一切

它包含**真实编译器的所有阶段**:

阶段	在哪
Lexer	`tokenize()`
Parser(递归下降)	`Parser` 类
AST(结构化语法树)	`@dataclass` 节点
Codegen(树形 → 线性)	`compile_node()`
优化	(本例没做,可以加常量折叠)
VM 执行	`run()`

把它当骨架,你可以扩展:

1. 加 `if`、`while` —— 引入跳转字节码 `JMP`、`JZ`。
2. 加函数 —— 引入 `CALL`、`RET`、调用栈。
3. 加局部变量 / 作用域 —— codegen 维护栈偏移。
4. 加类型检查 —— codegen 前加一遍 Sema。
5. 加优化 pass —— 在字节码上做 dead code、常量折叠。
6. 替换后端 —— 输出真正的 RISC-V 汇编而非字节码。

24.5 真实工业编译器的"放大版"

你刚写的这个迷你编译器,**和 GCC、Clang、Rustc 在抽象架构上是一回事**,只是规模和工程化水平不同:

你的迷你编译器	工业编译器
200 行 Python	几百万行 C++
一种字节码	LLVM IR、机器指令、各种 backend
5 种 AST 节点	几百种 IR/AST 节点
没优化	几十~几百个 pass
单文件	千万级模块的并行编译

核心思想完全一致 —— 这就是为什么读一遍 *Crafting Interpreters* 几乎能让你看懂任何编译器的源码结构。

24.6 串联到下一章

我们刚做了:**解析源码 → AST → 字节码 → VM 执行。**

注意我们**没有生成机器码**,而是写了一个 VM 来执行字节码。这正是 **解释器(Interpreter)** 的核心思路。下一章我们要把这个话题展开:

- **解释器** vs **编译器** 的本质对比。
- **JIT(Just-In-Time)编译**:运行时把"热点"字节码变成机器码,获得接近 native 的速度。

- 看 **CPython、JVM、V8** 各自的设计哲学。

🔍 本章没讲什么

- LL/LR parser 表的构造。
- LLVM IR 的真实写法。
- 寄存器分配的具体算法。

📚 想深入读什么

- Bob Nystrom, *Crafting Interpreters* — 同类项目的"豪华版",从 tree-walking 到 bytecode VM。**强烈推荐免费在线版。**
- Andrew Appel, *Modern Compiler Implementation in ML/Java/C* — 大学教材级。

← 上一章:编译原理 · → 下一章:解释器、虚拟机、JIT

第 25 章 · 解释器、虚拟机与 JIT

> 编译 vs 解释,不是非黑即白的对立——现代语言通常都是混合的。 这一章看 CPython、JVM、V8 三种代表方案的设计哲学。

25.1 解释器 vs 编译器:本质区别

先明确两个英文缩写,后面会反复用:

- **AOT** = **Ahead-Of-Time**(提前编译):**运行之前**就把源码翻译成机器码,生成可执行文件。Ch 23 讲的 GCC/Clang/Rustc,就是 AOT 编译器的代表。
- **JIT** = **Just-In-Time**(即时编译):**运行过程中**,根据运行情况把"热点代码"动态翻译成机器码。Java、JavaScript 引擎是经典 JIT。

直觉上的对比:

- **AOT**:像把菜**提前全部做好装盒**——启动飞快,但口味是固定的。
- **JIT**:像**现点现做** + 老练的厨师**根据你点菜的频率不断调整**——启动慢,但跑得越久越快。
- **解释**:像每次都从头念一遍菜谱再做——简单,但每次都不长记性。

回到三者的本质区别:

	编译器 (AOT)	解释器
何时翻译?	一次,提前	每次,运行时
输出	机器码文件	没有文件,直接执行
启动	慢(要先编译)	快
运行速度	快(原生指令)	慢(逐句解释)
灵活性	低	高(可 eval、可热更新)

但**实际上很多语言两者并存**:

- **C / Rust / Go**:纯 AOT。
- **Python / Ruby**:解释(实际上先编成字节码,再解释字节码)。
- **Java / C#**:先 AOT 编译成字节码,再用 JIT 在运行时把热点编译成机器码。
- **JavaScript V8**:类似 Java,但更激进地分层。

25.2 三种解释器的演进

① Tree-Walking 解释器

最朴素:**直接遍历 AST 求值**。我们 Ch 24 的 mini_compiler 如果不生成字节码,直接对 AST 递归求值,就是这种。

- ✅ 实现极简单。
- ✗ 慢——每次都要走树、做类型分派。

② Bytecode VM(字节码虚拟机)

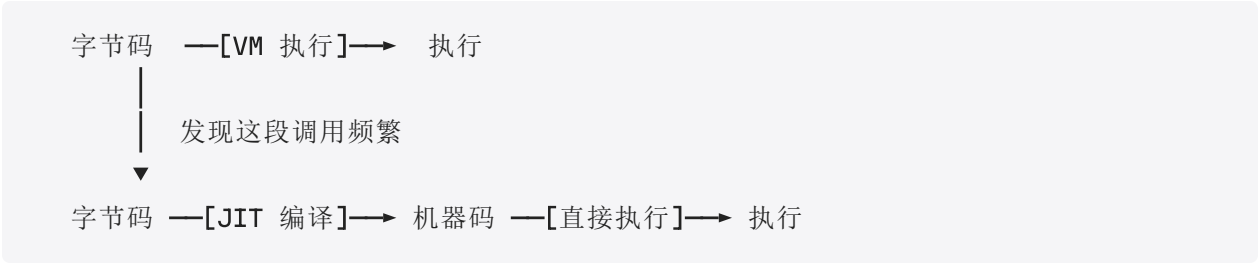
把 AST 一次性编译成线性的字节码,然后用一个紧凑的循环执行。**这就是 Ch 24 我们做的**,也是 CPython、Lua、Ruby、Erlang 的核心。

```
while True:
    op = bytecode[pc]; pc += 1
    if   op == ADD:  ...
    elif op == LOAD:  ...
     ...
```

工业级实现的优化: - **Switch dispatch → Direct Threading / Computed Goto**(用标签数组而非 switch,减少分支预测失败)。 - **Inline Caches**:记下"上次这条指令操作的类型",下次直接跳过类型查找。 - **Superinstructions**:把常见的 2-3 条字节码合并成一条。

③ JIT(Just-In-Time 编译)

最激进的解释器,本质是个运行时编译器。运行字节码时,**统计哪些函数 / 循环跑得多**(称为"热点 / hot spot"),然后把这些热点动态翻译成原生机器码,直接执行。



JIT 能做什么、为什么有时**比 AOT 编译还快**?

- **看到运行时类型才优化**:Python 的 `def f(x): return x + 1`,JIT 看到 99% 时 x 是 int,就生成 int 加法的机器码,加上一个"如果 x 不是 int 就退回解释器"的检查。这种 **Speculative Optimization** 是 AOT 做不到的。
- **inline 跨语言边界的 hot path**。
- **基于 profile 重新优化**(adaptive recompilation)。

25.3 案例 1:CPython

Python 的"标准实现",是 **bytecode VM(没有 JIT)**。

```
def add(a, b): return a + b
```

```
$ python -m dis mymodule.py
   ...   LOAD_FAST    a
         LOAD_FAST    b
         BINARY_ADD
         RETURN_VALUE
```

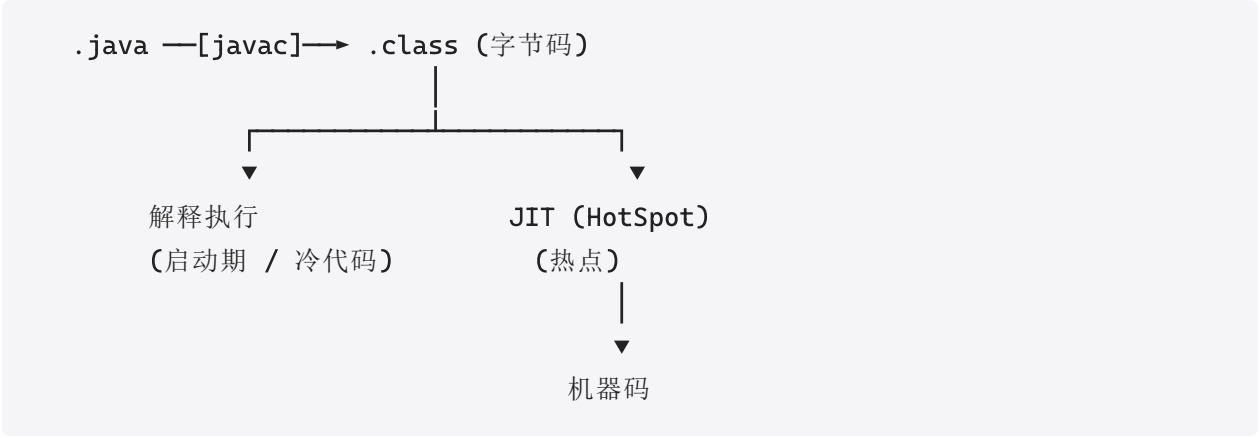
—— 整个就是一个栈式字节码,和我们 Ch 24 那个一模一样,只是规模大得多(几百种 opcode)。

为什么 Python 慢? 1. 字节码本身就比机器码慢 5-20 倍。 2. 每次 `BINARY_ADD` 都要做类型分派(int? float? str? 自定义 `__add__`?)。 3. 引用计数到处加锁(GIL)。

PyPy、Cinder、Pyston 等替代实现都加了 JIT,通常比 CPython 快 5-50 倍。Python 3.13 也开始引入实验性 JIT。

25.4 案例 2:JVM

Java 的设计从一开始就是**编译 + 解释 + JIT 三段式**:



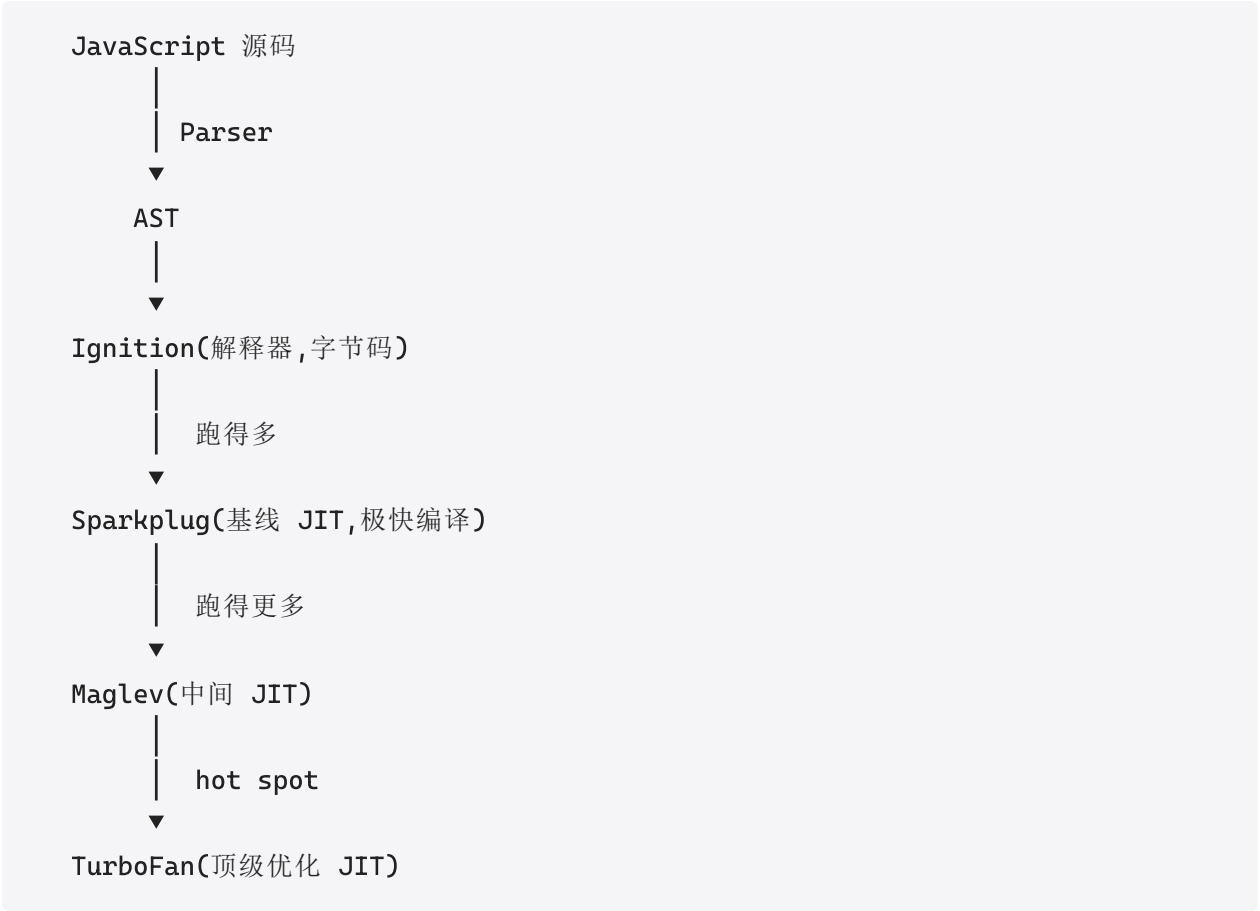
HotSpot JVM 有**两个 JIT 层级**:

- **C1(client compiler)**:快速编译,中等优化。
- **C2(server compiler)**:慢慢编译,极致优化。

代码先解释执行 → 跑得多了 → C1 编译 → 还跑得多 → C2 编译。**冷代码不浪费 CPU,热代码极致优化。**

25.5 案例 3:V8(JavaScript)

Google V8 是当代最复杂的 JIT 之一,**4 层执行**:



每一层都更慢编译、更快执行。如果 V8 猜错了类型,会**Deoptimize**(去优化)回到解释器,重新收集 profile 再编译。

> 💡 这种"编译 - 去优化"的循环,使得现代 JS 接近 C 速度,但也带来了 **Spectre/Meltdown** 类侧信道攻击的复杂安全风险。

25.6 何时该编译,何时该解释?

场景	推荐方案	为什么
系统底层(内核、驱动)	AOT 编译	启动快、可预测、不依赖运行时
嵌入式	AOT	内存有限,装不下 JIT
命令行工具	AOT 或 解释	启动时间敏感
服务器(长跑)	JIT	启动慢一点没关系,长期收益大
浏览器	JIT	没法 AOT(代码每次不同)
数据科学 / REPL	解释 + 选择性编译(如 Numba)	需要交互性

没有银弹,只有场景。

25.7 一个有趣的对偶:JIT ≈ CPU 内部的"自适应硬件"

现代 CPU 内部本身就是个"硬件 JIT": - 把 x86 指令解码成 μ-op 缓存。 - 分支预测器学习历史。 - 乱序执行调度。

而软件 JIT 在更高的抽象层做类似的事: - 把字节码翻译成机器码缓存。 - 类型剖析("这个变量永远是 int")。 - 内联函数边界。

> 🌟 **本质上,所有"高性能系统"都在做同一件事:观察输入分布,把高频路径变快。**CPU 在硬件层面做,JIT 在软件层面做,Cache 在存储层面做,数据库索引在数据层面做。这就是体系结构里的「Common Case Fast」原则。

25.8 「Common Case Fast」:一条贯穿全书的原则

> *Make the common case fast (and the rare case correct).* — David Patterson & John Hennessy

这是 Hennessy & Patterson 在《Computer Architecture》开篇就反复强调的设计哲学,也可能是整个计算机系统中**最重要的一条工程原则。**它的逻辑极其朴素:

> 与其让所有情况都"还不错",不如**把常见的情况做到飞快,代价是少见的情况慢一点甚至复杂一点。**

因为根据**Amdahl 定律**,你优化某段代码 10 倍,整个程序提速多少,**完全取决于这段代码占总时间的比例。**优化一段只占 1% 的代码到无穷快,程序也只快 1%;反过来,把占 90% 时间的部分快 2 倍,整体就快了大约 1.8 倍。

> 💡 因此,**任何严肃的性能优化,第一步都是「测量,找出真正的高频路径」**,而不是凭直觉猜。

这条原则在每一层都出现过

让我们沿着本书的抽象阶梯,看看「Common Case Fast」是怎么以**完全不同的形式**反复出现的:

抽象层	「常见情况」是什么	怎么把它做快	代价
CMOS / 电路 (Ch 4)	数字电路大部分时间不翻转	CMOS 静态功耗几乎为零;**只在翻转瞬间耗电**	翻转时短路电流(动态功耗)
CPU 流水线 (Ch 12)	大多数指令是顺序的,不跳转	**预测分支不跳**,继续顺序取指	预测错时冲刷流水线
分支预测器 (Ch 12)	同一条分支历史规律性强	用 BHT/TAGE 学习,准确率 ~95%+	预测器本身占面积、功耗
乱序执行 (Ch 12)	大多数指令彼此独立	一旦数据就绪就执行,不等前面	重排序缓冲区(ROB)硬件巨复杂
Cache (Ch 13)	访问有时间/空间局部性	L1/L2/L3 缓存常用数据	Cache 很贵、占大量芯片面积
TLB (Ch 13)	同一程序常访问少数几个页	缓存最近的虚→物地址翻译	TLB miss 时几十周期惩罚
虚拟内存 (Ch 13/26)	进程不会同时用满全部地址空间	按需分配物理页,空闲就不分配	缺页时要陷入内核
EDA 综合 (Ch 9)	芯片中大部分路径时序很松	只在关键路径上用大驱动单元	关键路径单元面积/功耗高
系统调用 (Ch 26)	大多数代码是用户态计算	用户态执行不陷入内核	真要 syscall 时一次几百周期
GC (Ch 22)	大多数对象短命、本来就该被回收	分代 GC,只扫年轻代	老年代偶尔大停顿(Major GC)
JIT (本章)	程序 80% 时间花在 20% 代码	只把这 20% 编成机器码	编译花时间;猜错类型要去优化
数据库索引	查询常按某些列过滤	B+ 树 / 哈希索引让 O(N)→O(log N)	索引占空间,写入时要维护
CDN / Cache 层	互联网请求高度集中	把热门内容推到离用户近的节点	一致性、过期、回源逻辑
HTTP/2 头部压缩	HTTP 头大量重复	用 HPACK 字典编码	状态机复杂度上升

抽象层	「常见情况」是什么	怎么把它做快	代价
CPU 内部 μ-op cache	同一段指令被反复执行	解码一次,μ-op 反复用	失效时重新解码慢
Branch Target Buffer	间接跳转的目标也有规律	缓存上次跳到哪	误预测时罚周期

—— **每一行都是同一个公式:猜一个高频假设 → 给这种情况设一条快速路径 → 给少见情况留一条慢但正确的退路。**

为什么它如此普遍?——一个深层的统计学事实

之所以这条原则到处适用,是因为**真实世界的几乎所有数据分布都是高度倾斜的**(俗称 80/20、长尾、Zipf 分布):

- 程序 80% 时间在 20% 代码里(Pareto 法则)。
- 网站 80% 访问集中在 20% 页面。
- 用户 80% 查询命中 20% 数据。
- 数学库的 80% 调用是 sin / cos / exp / log,而不是贝塞尔函数。

如果输入是**均匀分布**的,「Common Case Fast」就完全没意义——所有情况都一样常见,你没法挑一个去优化。但**真实世界从来不是均匀的**,所以这个原则才如此万能。

实施它的三步配方

无论你在哪一层工作,实施「Common Case Fast」永远是同样三步:

1. **测量(Profile)**:用 `perf` / `gprof` / Cache 分析器 / SQL `EXPLAIN`,找出真正的高频路径。**绝对不要凭直觉猜**——人类对热点的直觉通常是错的。
2. **隔离(Specialize)**:为高频情况写一条快速路径,可能用更少的检查、更紧凑的数据结构、内联展开。
3. **退路(Fallback)**:其余情况走一条慢但**正确**的通用路径。"快但偶尔出错"在工程上是不可接受的——这条原则强调"快"和"正确"是分开的轴。

> ⚠️ **常见误区**: - 「过早优化」:在没测量的时候优化错地方。 - 「过度专门化」:为不存在的高频路径写一堆代码,反而拖慢。 - 「忘了退路」:快速路径假设失效时,慢路径不正确,导致 bug。Spectre 漏洞就是 CPU 在分支预测错误时**没充分回滚**所有可观察状态导致的(Ch 12)。

「Common Case Fast」与本书的"抽象阶梯"

回到本书的主线——**计算机系统每一层的抽象,本质上都是「Common Case Fast」的产物**:

- **冯·诺依曼架构**之所以胜出而不是数据流机:**串行控制流是大多数程序的常态**,我们为它优化,代价是并行硬一点。
- **缓存层级**之所以是金字塔:**越快越贵,所以只放最常用的数据。**
- **进程隔离**之所以默认开:**大多数代码不需要互相干扰**,共享反而是少数情况(显式 mmap)。
- **垃圾回收**之所以可行:**对象寿命极不均匀**,允许我们专门为短命对象优化。

> 🌟 一旦你在脑子里装上这条原则,你看任何复杂系统(操作系统、JVM、数据库、TCP/IP、CDN)时都会**反复发现同一个套路**:它们用各自层面的"高频路径优化"换来了对外简单的接口和惊人的性能。
>
> **这是计算机科学和工程实践之间最深刻的桥梁之一。**

25.9 串联到下一部分

我们已经把"代码 → 字节码 → 机器码"全部讲清了。但程序不是孤立运行的——它运行在**操作系统**上,和文件、网络、显示器、其他进程打交道。

最后一部分(Part IX)我们要把所有层连起来,看一段简单的程序如何在硅、CPU、OS、应用之间走完一个完整的循环。

🔍 本章没讲什么

- 具体 JIT 的优化算法(GVN、scalar replacement)。
- WebAssembly 与浏览器之外的应用。
- AOT 与 JIT 的混合方案(GraalVM、CRaC)。

📚 想深入读什么

- Aleksey Shipilëv 的 JVM 性能博客 — 工程师视角。
- *Crafting Interpreters* 第 14-30 章 — 从零写一个 bytecode VM。
- V8 Blog 与 Maglev / TurboFan 论文。

← 上一章:迷你编译器 · → 下一章:操作系统

第 26 章 · 操作系统:管理硬件资源的软件

> CPU 只会一条一条执行指令。是操作系统让"多个程序、多个用户、多个设备"和谐共存。

26.1 操作系统是什么?

简短答案:**操作系统(Operating System, OS)是一段特殊的软件,它管理硬件,并向应用程序提供服务。**

它有四个核心职责:

1. **进程管理**:让多个程序"同时"在一个 CPU 上跑(其实是分时切换)。
2. **内存管理**:每个进程一个虚拟地址空间(Ch 18)。
3. **文件系统**:把硬盘上的字节流抽象成"文件" / "目录"。
4. **设备驱动**:统一管理键盘、网卡、显示器等。

外加一个超级重要的能力:**保护**——防止一个程序搞挂别的程序、或者搞挂操作系统本身。

26.2 用户态 vs 内核态:CPU 的两种"特权级"

现代 CPU 至少支持 2 个特权级(x86 有 4 个,但实际只用 2 个;RISC-V 标准 3 个:U/S/M):

- **用户态(User Mode)**:跑普通程序。**不能执行特权指令**(如关中断、改页表、操作 I/O 端口)。
- **内核态(Kernel Mode)**:跑操作系统。**所有指令都能执行。**

> 🔑 **这是计算机安全的硬件基石。**没有特权级,任何应用都能毁掉整个系统。

26.3 系统调用:用户态请求内核服务的桥

应用程序需要做"特权操作"(读文件、发包、创建进程)时,**不能直接执行内核代码**——必须通过**系统调用(syscall)**。

机制:

```
[ 用户态 ]  →  执行特殊指令 (RISC-V 是 ecall, x86 是 syscall)
                        │
                        │  CPU 自动切换到内核态,跳到预设的入口
                        ▼
[ 内核态 ]  →  内核根据系统调用号,分发到对应函数
                        │
                        │  完成后,执行 sret/sysret
                        ▼
[ 用户态 ]  →  继续执行下一条指令
```

C 程序员看到的是 `read()` / `write()` / `mmap()` / `fork()`,这些都是 libc 包装的系统调用。

26.4 进程与线程

- **进程(Process)**:一个独立的程序实例,有自己的地址空间、文件描述符、PID。
- **线程(Thread)**:进程内的"执行流",**多个线程共享地址空间**,各有自己的栈和寄存器。

进程间隔离 → 安全;线程间共享 → 高效但容易互相打架(数据竞争)。

创建进程的"奇怪 API":fork + exec

Unix 风格(本身已经是 50 多岁的老古董,但仍是事实标准):

```
pid_t pid = fork();         // 几乎"复制"当前进程
if (pid == 0) {             // 子进程
    execve("/bin/ls", ...);
}
// 父进程在这里
```

`fork` 把当前进程整个克隆一份,然后 `exec` 用一个新程序替换掉子进程的内存映像。这种"分两步"的设计虽然看似冗余,但提供了极大的灵活性(中间可以重定向 I/O、改环境变

量)。

26.5 调度器:谁先跑?

CPU 只有一个(或几个),但进程可能有几百个。**调度器(Scheduler)**决定哪个进程在哪个时刻在哪个 CPU 上跑。

经典策略:

- **轮转(Round-Robin)**:每人 10ms,轮流来。
- **优先级**:更重要的先跑。
- **公平调度(CFS,Linux)**:按"已用 CPU 时间"挑最少的那个。
- **实时调度**:硬实时系统的需求(航空、车控)。

每次切换叫**上下文切换(Context Switch)**:保存当前进程的所有寄存器到内存,加载下一个进程的寄存器。**通常需要 1~10 μs**——不算大,但频繁切换会拖性能。

26.6 文件系统:把硬盘字节当"目录树"

硬盘原本是一片连续的扇区(几亿块,每块 4KB)。文件系统是**一种数据结构 + 一组操作**,让你能:

- 创建/打开/读/写文件。
- 组织成树形目录。
- 管理元数据(权限、所有者、修改时间)。

常见文件系统:ext4(Linux 默认)、NTFS(Windows)、APFS(Mac)、ZFS、Btrfs。

应用程序看到的是 `open() / read() / write() / close()`,这些都是 syscall。

26.7 中断与异常:让 CPU 「停下来」

CPU 默认一条一条执行指令。但外部世界(网卡来包、键盘按下、定时器到点)需要 CPU 注意。机制:

- **中断(Interrupt)**:外部硬件触发,跳到内核中断处理函数。

- **异常(Exception)**:CPU 自己产生(除零、缺页、非法指令)。
- **陷阱(Trap)**:程序主动触发(就是上面的 syscall)。

中断处理流程(简化):

1. 硬件信号到达 → CPU 暂停当前指令。
2. CPU 把 PC、状态寄存器存到栈,切换到内核态。
3. 跳到中断向量表中对应的处理函数。
4. 处理(可能很快、可能调度别的进程)。
5. 恢复现场,从被打断的地方继续。

> 💡 **中断让 OS 不必"轮询"**。CPU 大部分时间可以执行用户代码,只在事件发生时被"叫醒"。

26.8 虚拟内存(再访)

- 每个进程一个独立 4GB(或 256TB)虚拟地址空间。
- 内核维护**页表**,把虚拟页 → 物理页。
- TLB 加速翻译(Ch 13)。
- 物理内存不够时,把不常用的页**换出到磁盘**(swap)。
- 多进程共享物理页(共享库、Copy-on-Write)。

虚拟内存是 50 年来 OS 最重要的发明之一——它解决了**隔离、保护、共享、超额分配**四件事。

26.9 串联到下一章

我们终于具备了讲清楚 **「`print("Hello, World!")` 完整旅程」** 的所有零件:

- 硅上的 CMOS、门、CPU、Cache、TLB(Part I-IV)
- ISA、汇编、ELF(Part V)
- C、Python(Part VI-VII)
- 编译器、解释器(Part VIII)
- 操作系统(本章)

下一章我们就把这些线索串起来,**沿着一个 Hello World,走遍所有抽象层**。这是本书的"压轴"。

🔍 本章没讲什么

- 进程间通信(pipe、socket、共享内存、消息队列)。
- 锁、信号量、原子操作、内存序。
- Linux 内核架构(VFS、netfilter、cgroups、namespaces)。
- 微内核 vs 宏内核;容器 vs 虚拟机。

📚 想深入读什么

- *Operating Systems: Three Easy Pieces*(OSTEP)— 免费在线,清晰易读。
- *Linux Programming Interface* by Kerrisk — Linux 系统编程圣经。
- *xv6* (MIT) 源码 — 真实但简化的教学 OS,几千行代码可以读完。

← 上一章:解释器、JIT · → 下一章:Hello World 完整旅程

第 27 章 · 一行 `print("Hello")` 的完整旅程

> 这是我们在前言里许下的承诺: *从硅原子里电子的运动,讲到屏幕上 `Hello` 的字符。*
> 现在,我们终于具备了所有零件——把它一气讲完。

27.1 起点:一行代码

```
# hello.py
print("Hello, World!")
```

你按下 Enter,屏幕上出现 `Hello, World!`。这一秒钟里,**计算机要走完 11 个层级**。我们一层一层往下走。

27.2 第 11 层 · Python 解释器(CPython)

CPython 是一个用 C 写的解释器(Ch 25)。它执行你的 hello.py 时:

1. **Lexer**(Ch 23):把源码切成 tokens `[NAME "print", LPAR, STRING "Hello ... ", RPAR]`。
2. **Parser**:构造 AST。
3. **Compile**:把 AST 编成 Python **字节码**: `LOAD_NAME 'print' LOAD_CONST 'Hello, World!' CALL 1 POP_TOP`
4. **VM 执行字节码**: - `LOAD_NAME 'print'` → 在内置命名空间里找到 print 函数对象。 - `CALL 1` → 调用 print。

27.3 第 10 层 · Python 标准库的 `print` 函数

`print` 是用 C 实现的内置函数(`bltinmodule.c`),它最终调用:

```
PyObject_CallMethod(file, "write", "O", text);
```

也就是 `sys.stdout.write("Hello, World!\n")`。`sys.stdout` 在交互式 shell 里是一个 `io.TextIOWrapper` 对象,内部调:

```
fwrite(buf, 1, n, fp);
```

—— 我们终于离开 Python,进入 **C 标准库(libc)**。

27.4 第 9 层 · libc(`fwrite` → `write`)

`fwrite` 是 stdio 的"带缓冲"接口。它把数据攒进缓冲区,在合适的时机调底层的:

```
ssize_t write(int fd, const void *buf, size_t count);
```

`write` 是 **glibc 对系统调用的薄封装**,它做:

1. 把 fd、buf、count 装进寄存器(RISC-V 的 a0/a1/a2)。
2. 把 syscall 编号(Linux x86-64 上 `write` 是 1)装进 a7。
3. 执行 `ecall` 指令(Ch 14)。

27.5 第 8 层 · ISA / CPU 取指

`ecall` 这条指令的二进制是:

```
0x00000073   →   00000000_00000000_00000000_01110011
```

CPU 内部:

1. **IF**(Ch 11):把 PC 送给 i-cache(Ch 13),拿回这条指令。
2. **ID**:控制器看到 opcode = 1110011 → 这是 ecall。
3. **EX**:CPU 注意到这是特权指令,**触发陷阱(trap)**: - 把当前 PC 存到 mepc 寄存器。 - 把 trap 原因写入 mcause。 - **切到内核态(Ch 26)**,跳到 stvec 里登记的内核陷阱入口。

27.6 第 7 层 · 内核态:系统调用分发

Linux 内核的 entry 点(`do_syscall_64` for x86,RISC-V 类似):

1. 保存所有用户态寄存器到内核栈。
2. 读 a7 知道是 syscall 1(write)。
3. 在 `sys_call_table` 里找到 `sys_write` 函数指针。
4. 调用它。

27.7 第 6 层 · `sys_write` → tty 子系统

stdout 通常连到一个**伪终端(pty)** 或 **真终端(tty)**:

1. `sys_write` 找到 fd 对应的 `struct file`。
2. 调它的 `f_op→write_iter`,这是 tty 子系统的 write。
3. tty 把数据 push 到一个**行缓冲(line buffer)**。
4. 最终交给**终端驱动**。

27.8 第 5 层 · 终端驱动 → 帧缓冲(framebuffer)

如果你在桌面环境(GNOME/KDE)的终端里:

1. 数据其实经过一个**伪终端 master**,被 GNOME Terminal(用户态进程)读到。
2. GNOME Terminal 调 GUI 库(GTK/Qt)的 draw_text。
3. GUI 库把字符 → 字形 → 像素位图,写到一个**绘图 surface**。
4. 通过 X11/Wayland 协议交给**合成器(Compositor)**,合成器再把整个屏幕的像素**写到帧缓冲(framebuffer)**。

27.9 第 4 层 · GPU / 显示控制器 → 物理像素

1. 显卡(GPU)读帧缓冲,通过 HDMI/DisplayPort 协议,**逐像素扫描**送到显示器。
2. 显示器内部的**时序控制器(TCON)** 把像素值变成对每个**子像素(R、G、B)** 的电压驱动信号。
3. LCD 像素的液晶分子受电场影响转向,改变背光透过率;OLED 像素直接因电流发光。

27.10 第 3 层 · 数字逻辑

GPU、CPU、显示器、键盘控制器,所有这些芯片内部都是:

- 时钟边沿驱动的 **D 触发器**(Ch 7)
- **组合逻辑**(加法器、MUX、译码器,Ch 6)
- 由 **NAND/NOR 门** 搭出来(Ch 5)

每条 ecall 指令、每个像素值,都对应着上百万个 NAND 门的瞬时翻转。

27.11 第 2 层 · 物理:CMOS 晶体管

每个 NAND 门是 4 个 CMOS 晶体管(Ch 4)。整颗 CPU 大约几十亿到上千亿个。

每次门翻转的物理过程: 1. 上一级输出电压跨过 PMOS/NMOS 的阈值。 2. 沟道形成或消失,带动下一级节点电容充放电。 3. 电压稳定,逻辑值"传播"到下一级。

整个过程不到 1 ns,**功耗几乎全部发生在翻转的瞬间。**

27.12 第 1 层 · 半导体物理:电子的运动

每次 CMOS 翻转,沟道里**几千~几万个电子**在电场作用下漂移。这些电子来自掺杂的硅(Ch 2):

- N 型硅多余的电子。
- P 型硅多余的空穴。
- 它们通过 PN 结的电场被精确地控制(Ch 3)。
- 硅原子的能带结构(Ch 2)决定了这一切在室温下"恰好"可控。

27.13 完整链路图(从底到顶)

```
你看到屏幕上的 'H' 字符
      ▲
像素 (R=255, G=255, B=255)
      ▲
GPU 帧缓冲 → 显示器扫描
      ▲
终端 / GUI 合成器 (用户态)
      ▲
tty 驱动 / pty (内核)
      ▲
sys_write (内核函数)
      ▲
ecall 指令陷入内核 (CPU 切换特权级)
      ▲
libc write() / Python C 层
      ▲
CPython 字节码 VM 解释 print
      ▲
你的 hello.py
      ▼
──────── 抽象界面分割线 ────────
      ▲
每条机器指令 → 五级流水线 → 几亿次 D 触发器翻转
      ▲
每个 D 触发器 → ~6 个 CMOS 晶体管
      ▲
每个 CMOS → 几千个电子在硅沟道里漂移
      ▲
硅原子的能带 (1.12 eV 禁带)
```

这就是「从硅到软件」的完整链路。从下往上每一层,都为上一层"隐藏复杂度,提供新能力"。

27.14 抽象层为什么如此重要

如果**没有这套层级**,会发生什么?

- 写一个网页要懂半导体物理。
- 换一颗 CPU 要重写所有软件。

- Python 程序员要管理寄存器分配。
- 操作系统每次升级都要 IC 设计师配合。

这套抽象 = 文明级别的"分工"。每一层让上一层的人专注自己的问题,不用懂下一层的细节。

> 但**抽象不是免费的。**下一章我们看抽象层带来的代价 —— 性能、调试、安全方面的"账本"。

🔍 本章没讲什么

- macOS / Windows 的对应链路(细节不同,精神一致)。
- 网络 IO(`print` 改成发个 HTTP 请求,会再涉及 TCP/IP 协议栈)。

📚 想深入读什么

- *CSAPP* — 全书都在串这个链路,值得整本读。
- Linux source code(尤其 `fs/read_write.c` 、 `drivers/tty/tty_io.c`)。

← 上一章:操作系统 · → 下一章:抽象层的代价与意义

第 28 章 · 全栈视角:抽象层的代价与意义

> 抽象不是免费的午餐。 但没有它,我们也吃不上饭。 这是本书的最后一章,我们退一步,看整张地图。

28.1 抽象的双面性

整本书从硅讲到软件,**核心叙事就是「抽象」**:

- CMOS 抽象掉了电子和电场,只露出 0/1。
- 逻辑门抽象掉了 CMOS 拓扑,只露出布尔代数。
- Verilog 抽象掉了门电路,只露出 RTL。
- ISA 抽象掉了微架构,只露出指令集。
- C 抽象掉了 ISA,只露出"可移植的汇编"。
- 高级语言抽象掉了内存模型,只露出对象与函数。
- 编译器把这些抽象一层层翻译回硬件能跑的形式。

每一层都是一个胜利,但每一层也都有代价。

28.2 代价 1:性能开销

抽象层	典型开销
高级语言(动态类型)	比 C 慢 10~100x
GC 内存管理	平均 5~20% 吞吐损失,周期性卡顿
虚函数 / 接口分发	单次调用慢 2-5x,且阻碍内联
异常处理	不抛时近零;真抛时极慢
Cache 抽象	错误访问模式可让性能跌 5-20x
系统调用	一次 ~100ns~1μs(几百~几千个时钟周期)
进程切换	1-10 μs
网络栈	一次包处理 ~10μs

经验法则:每跨一层抽象,大约 10x 慢一档。所以"从 Python 到 CPU"差几个数量级是合理的。

28.3 代价 2:调试难度

抽象层越多,bug 出现时**离根本原因越远**:

- Python 程序很慢 → 是算法?是 Python?是 GIL?是 Cache miss?是 GPU 显存带宽?
- C 程序段错误 → 是空指针?是栈溢出?是 use-after-free?是别人改了我?

每一层都可能"骗"你。例如 CPU 的乱序执行让你看到反直觉的内存读写顺序;编译器优化可能让"看起来正确的代码"产生错误结果(如 strict aliasing 违例);JIT 在分层切换时 bug 极难重现。

> 💡 这就是为什么"打通整个抽象栈"的工程师特别值钱——他们能往下找到问题真正发生的层。

28.4 代价 3:安全的攻击面

每一层都是潜在的攻击面:

- **CPU 层**:Spectre、Meltdown、Rowhammer。
- **OS 层**:权限提升、缓冲区溢出、TOCTOU。
- **运行时层**:JIT spraying、反序列化漏洞。
- **应用层**:SQL 注入、XSS、内存安全 bug。
- **生态层**:供应链攻击(npm、pip、容器镜像)。

抽象隐藏了复杂度,但攻击者会去找隐藏处的破绽。这是为什么现代安全研究既要懂 web,也要懂操作系统,也要懂硬件。

28.5 但抽象的"红利"远超代价

让我们也算一笔正收益的账:

没有抽象	有了抽象
每个程序员要懂半导体	70% 的程序员一辈子不懂硬件,也能创造价值
换 CPU = 重写所有软件	x86 程序在 ARM 下二进制级别都能跑(模拟器/翻译层)
一个 OS 服务于一种硬件	Linux 跑在从手表到超级计算机的所有平台
协作要求知识完全重叠	一个 IC 工程师 + 一个 Web 前端 + 一个 ML 研究员能合作
一个 bug 要改整个栈	大多数 bug 限制在一层内

计算机文明的整个生态——开源、互联网、移动应用、AI——**都建立在这套抽象层之上。**

28.6 选择正确的抽象层

回到你的工作——**怎么选?**

你的目标	应该工作在哪一层
设计新芯片	Verilog / 网表 / 物理设计
写操作系统 / 嵌入式	C / 汇编
写数据库 / 游戏引擎 / 浏览器	C++ / Rust
写高性能服务	Go / Rust / Java(JIT)
业务系统 / Web 后端	Python / Java / TypeScript
数据科学 / ML 应用	Python(底下接 CUDA / NumPy)
快速原型 / 脚本	Python / Bash

没有最好的语言,只有最合适当前问题的抽象层。

28.7 当抽象漏水时(Leaky Abstractions)

Joel Spolsky 有句名言:**所有不平凡的抽象,在某种程度上都会"漏水"。**

例子:

- **TCP** 应该是"可靠流",但你必须懂超时、丢包、Nagle 算法,才能写出快的网络程序。
- `file` **抽象** 让你以为读硬盘和读网络挂载文件一样,但延迟差几个数量级。
- **GC** 让你"不用关心内存",但内存泄漏(意外的引用持有)和性能问题逼你去看堆 dump。
- **CPU 抽象** 让你以为指令是顺序执行的,但乱序、Cache、分支预测可能让两段"等价代码"性能差 100x。

> **能识别抽象在何时漏水,是高级工程师的核心能力之一。**这就是为什么完整理解整张地图(也就是这本书想给你的)如此重要。

28.8 从这本书出发,你可以去哪?

不同方向的"下一站":

🔧 EE / 芯片方向

- 模拟集成电路设计:Razavi 系列。
- 数字 VLSI 设计:Weste & Harris。
- 计算机体系结构:Hennessy & Patterson。
- 实操:学一个 EDA 工具(Synopsys DC + ICC,或开源 Yosys + OpenROAD)。
- 项目:用 Verilog 自己实现一个 RV32I CPU,Yosys + nextpnr 综合到 FPGA。

💻 CS / 系统方向

- 操作系统:OSTEP、xv6 源码。
- 编译器:*Crafting Interpreters* → LLVM。
- 数据库内核:CMU 15-445。
- 分布式系统:MIT 6.824(Raft)。
- 项目:用 Rust 写一个 mini OS / DB / 浏览器。

🧠 跨界方向

- 高性能计算:CUDA / GPU 体系结构。
- 编译器在硬件加速器上:MLIR、TVM、TensorRT。
- 形式化验证:TLA+、Coq。
- 安全:CTF、漏洞研究。
- AI 编译器和芯片:这是 EE + CS 完美融合的当代前沿。

28.9 最后一句话

计算机不是魔法,只是大量简单原则的庞大堆叠。

读完这本书,你不一定记得每一个细节。但你应该记住:

> 当你写一行代码,你是在指挥几十亿个开关。 当你按下回车,你是在调动从硅原子到云端服务器的整个文明的产物。

这个事实本身,就足够让人对这门学科保持终生的敬畏与好奇。

> **🎯 全书总图回顾**
>
> Part 0:沙子 → Part I:晶体管 → Part II:门 → Part III:RTL → Part IV:CPU → Part V:ISA → Part VI:C → Part VII:高级语言 → Part VIII:编译/解释 → Part IX:OS / 应用
>
> 这条线就是这本书的全部。 现在它已经在你的脑子里了。

← 上一章:Hello World 旅程 · → 附录 A:推荐阅读

附录 A · 推荐阅读与课程

> 这本书是一张地图。每个 Part 都标了"想深入读什么"。这里再做一个集中索引,加上一些**横跨多层的好资源**。

A.1 一站式跨层教材(强烈推荐)

这些书本身就有"跨层"的雄心,和本书定位最接近:

- **Patrick Henry Winston, *The Elements of Computing Systems* (NAND2Tetris)** 🌟
- 配套有 **MOOC**:nand2tetris.org
- **从一个 NAND 门开始,亲手搭出 CPU、汇编器、VM、编译器、操作系统、Tetris 游戏。**
- 这是和本书**精神最接近**的一门课。强烈推荐学完本书后立刻动手。
- **Bryant & O'Hallaron, *Computer Systems: A Programmer's Perspective* (CSAPP)** 🌟
- **以 C 程序员视角串起体系结构、汇编、内存、链接、I/O、并发。**CMU 15-213 的教材。
- 配套实验(bomblab、attacklab、cachelab、shelllab、proxylab)非常硬核。
- **Patterson & Hennessy, *Computer Organization and Design* (RISC-V Edition)**
- 体系结构入门。从数字逻辑一路讲到 CPU、流水线、存储层次。

A.2 按层级深入

半导体器件(Part I)

- Neamen, *Semiconductor Physics and Devices* — 本科教材。

- Sze, *Physics of Semiconductor Devices* — 工业级深度。
- Razavi, *Design of Analog CMOS Integrated Circuits* — 模拟视角。

数字逻辑(Part II)

- Mano & Ciletti, *Digital Design* — 经典本科。
- Harris & Harris, *Digital Design and Computer Architecture* — 直接连到 CPU。

HDL 与 EDA(Part III)

- Samir Palnitkar, *Verilog HDL* — Verilog 入门。
- Stuart Sutherland, *SystemVerilog for Design* — 现代设计首选。
- Sherwani, *Algorithms for VLSI Physical Design Automation* — 后端经典。
- 在线练习:**HDLBits** ☀ — 免费 Verilog 题库。

体系结构(Part IV)

- Hennessy & Patterson, *Computer Architecture: A Quantitative Approach* — 圣经。
- Shen & Lipasti, *Modern Processor Design* — 超标量 / 乱序细节。

ISA & 系统编程(Part V-VI)

- *RISC-V Specification* — riscv.org 免费下载。
- Patterson & Waterman, *The RISC-V Reader*。
- Kernighan & Ritchie, *The C Programming Language* (K&R) — 薄而经典。
- 俞甲子等, 《程序员的自我修养:链接、装载与库》— 中文最佳。
- Robert Seacord, *Effective C / Secure Coding*。

高级语言(Part VII)

- Pierce, *Types and Programming Languages* (TAPL) — 类型理论圣经。
- Scott, *Programming Language Pragmatics* — 各种语言机制横向对比。

编译器与解释器(Part VIII)

- **Bob Nystrom, *Crafting Interpreters*** ☀ — 免费在线,最好的入门书。
- Cooper & Torczon, *Engineering a Compiler* — 现代编译器最佳教材之一。
- Aho 等, 《Compilers: Principles, Techniques, and Tools》(龙书)— 经典。
- Appel, *Modern Compiler Implementation in ML/Java/C* — 大学教材级。

操作系统(Part IX)

- **Remzi 夫妇, *Operating Systems: Three Easy Pieces* (OSTEP)** 🌟 — 免费在线,清晰易读。
- Kerrisk, *The Linux Programming Interface* — Linux 系统编程圣经。
- **xv6 (MIT)** — 真实但简化的教学 OS,几千行代码可读完。

A.3 重磅 MOOC / 课程

- **MIT 6.004** — 计算机体系结构 + Verilog,有公开视频。
- **CMU 15-213 / 15-410** — CSAPP / 操作系统。
- **MIT 6.S081** — 用 xv6 写自己的 OS。
- **Stanford CS143** — 编译器,从词法到代码生成。
- **MIT 6.824** — 分布式系统(Raft 共识)。
- **CMU 15-445** — 数据库内核。

A.4 工具与"实验环境"

- **Compiler Explorer** (godbolt.org) 🌟 — 在线看任意编译器/优化的汇编。**这本书读到 Ch 19 / Ch 23 时一定要打开它。**
- **Yosys + nextpnr** — 开源 EDA 工具链,可以从 Verilog 综合到真实 FPGA。
- **Spike / QEMU** — RISC-V 模拟器。
- **Verilator** — 把 Verilog 编译成 C++,跑得飞快。

A.5 跨界的好书

- 吴军, 《数学之美》/《浪潮之巅》— 信息技术史与数学根源。
- Petzold, *Code: The Hidden Language of Computer Hardware and Software* — 极其通俗的"从硅到软件"科普。
- Hofstadter, *Gödel, Escher, Bach* — 自指、抽象与计算的哲学讨论。

← 回到正文 第 28 章 · → 附录 B:术语表

附录 B · 术语表(中英对照)

> 横跨 EE 和 CS 的常用术语表。术语后面的 [Ch x] 是本书第一次出现的章节。

EE / 硬件 / IC 设计

中文	English	简释	章节
半导体	Semiconductor	导电性可控的材料	[Ch 2]
能带	Band	固体中电子能量的连续区间	[Ch 2]
禁带 / 带隙	Band Gap	价带与导带之间的能量空缺	[Ch 2]
PN 结	PN Junction	P 型与 N 型半导体的接触界面	[Ch 3]
耗尽区	Depletion Region	PN 结附近无自由载流子的区域	[Ch 3]
二极管	Diode	单向导电的两端器件	[Ch 3]
MOSFET	Metal-Oxide-Semiconductor FET	场效应晶体管	[Ch 4]
NMOS / PMOS	NMOS / PMOS	N 沟道 / P 沟道 MOS	[Ch 4]
CMOS	Complementary MOS	NMOS + PMOS 互补结构	[Ch 4]
阈值电压	Threshold Voltage (Vth)	MOS 开始导通的栅极电压	[Ch 4]
逻辑门	Logic Gate	AND/OR/NOT 等基本逻辑单元	[Ch 5]
组合逻辑	Combinational Logic	输出只取决于当前输入	[Ch 6]
时序逻辑	Sequential Logic	输出依赖历史(有状态)	[Ch 7]
触发器	Flip-Flop	边沿触发的 1 比特存储单元	[Ch 7]
锁存器	Latch	电平触发的 1 比特存储单元	[Ch 7]
寄存器	Register	多个触发器并联组成的存储单元	[Ch 7]
有限状态机	FSM (Finite State Machine)	有限状态的控制结构	[Ch 7]
HDL	Hardware Description Language	硬件描述语言(Verilog/VHDL)	[Ch 8]
RTL	Register Transfer Level	寄存器传输级抽象	[Ch 8]

中文	English	简释	章节
EDA	Electronic Design Automation	电子设计自动化	[Ch 9]
综合	Synthesis	RTL → 门级网表	[Ch 9]
网表	Netlist	门级电路连接清单	[Ch 10]
标准单元	Standard Cell	预设计的基本逻辑单元 (NAND2_X1 等)	[Ch 10]
布局 / 布线	Placement / Routing	物理设计的两个步骤	[Ch 9]
静态时序分析	STA (Static Timing Analysis)	验证时序约束	[Ch 9]
Setup / Hold	Setup / Hold	时钟边沿前后的数据稳定要求	[Ch 7]
时钟域跨越	CDC (Clock Domain Crossing)	跨时钟域信号传递	[Ch 7]
流片	Tapeout	把版图发送给晶圆厂	[Ch 9]

计算机体系结构

中文	English	简释	章节
ALU	Arithmetic Logic Unit	算术逻辑运算单元	[Ch 6]
数据通路	Datapath	CPU 中数据流过的路径	[Ch 11]
控制器	Control Unit	CPU 中产生控制信号的部分	[Ch 11]
程序计数器	PC (Program Counter)	下一条指令的地址	[Ch 11]
流水线	Pipeline	多阶段并行处理	[Ch 12]
冒险	Hazard	流水线中的依赖问题	[Ch 12]
数据冒险	Data Hazard	后指令依赖前指令未写回的数据	[Ch 12]
控制冒险	Control Hazard	分支指令导致的取指不确定	[Ch 12]
转发 / 旁路	Forwarding / Bypass	不等写回直接传值	[Ch 12]
分支预测	Branch Prediction	猜测分支去向	[Ch 12]
超标量	Superscalar	每周期发射多条指令	[Ch 12]
乱序执行	Out-of-Order (OoO)	指令执行顺序与程序顺序不同	[Ch 12]
Cache	Cache	高速缓存	[Ch 13]
TLB	Translation Lookaside Buffer	虚拟地址翻译缓存	[Ch 13]
局部性	Locality	时间 / 空间访问局部性	[Ch 13]
ISA	Instruction Set Architecture	指令集架构	[Ch 14]
RISC / CISC	RISC / CISC	精简 / 复杂指令集	[Ch 14]
微架构	Microarchitecture	ISA 的具体硬件实现	[Ch 14]

软件 / 操作系统 / 编译器

中文	English	简释	章节
机器码	Machine Code	CPU 直接执行的二进制指令	[Ch 15]
汇编	Assembly	机器码的人类可读形式	[Ch 15]
汇编器	Assembler	汇编 → 机器码	[Ch 15]
链接器	Linker	把多个 .o 拼成可执行文件	[Ch 16]
加载器	Loader	把可执行文件读入内存执行	[Ch 16]
ELF	Executable and Linkable Format	Linux/Unix 可执行格式	[Ch 16]
ABI	Application Binary Interface	二进制层面的接口规范	[Ch 15]
调用约定	Calling Convention	函数参数 / 返回值的约定	[Ch 15]
栈 / 堆	Stack / Heap	自动 / 手动管理的内存区	[Ch 18]
段错误	Segmentation Fault	非法内存访问	[Ch 18]
系统调用	System Call (syscall)	用户态请求内核服务	[Ch 26]
用户态 / 内核态	User Mode / Kernel Mode	CPU 的两种特权级	[Ch 26]
进程 / 线程	Process / Thread	独立 / 共享地址空间的执行单元	[Ch 26]

中文	English	简释	章节
上下文切换	Context Switch	切换执行流时保存 / 恢复状态	[Ch 26]
虚拟内存	Virtual Memory	进程独立的虚拟地址空间	[Ch 13]
缺页	Page Fault	访问未映射页时的异常	[Ch 18]

编程语言 / 编译器

中文	English	简释	章节
编译器	Compiler	把源码翻译成另一种形式	[Ch 23]
解释器	Interpreter	直接执行源码或字节码	[Ch 25]
词法分析	Lexical Analysis (Lexer)	字符流 → token 流	[Ch 23]
语法分析	Parsing	token 流 → AST	[Ch 23]
AST	Abstract Syntax Tree	抽象语法树	[Ch 23]
语义分析	Semantic Analysis	类型检查、作用域检查	[Ch 23]
IR	Intermediate Representation	中间表示	[Ch 23]
SSA	Static Single Assignment	每个变量只赋值一次的 IR 形式	[Ch 23]
寄存器分配	Register Allocation	虚拟 → 物理寄存器映射	[Ch 23]
AOT	Ahead-Of-Time Compilation	提前编译(运行前就生成机器码,如 GCC)	[Ch 25]
JIT	Just-In-Time Compilation	即时编译(运行时把热点编成机器码)	[Ch 25]
字节码	Bytecode	VM 执行的中间码	[Ch 24]
GC	Garbage Collection	垃圾回收	[Ch 22]

中文	English	简释	章节
闭包	Closure	函数 + 捕获的环境	[Ch 22]
多态	Polymorphism	同一接口多种实现	[Ch 22]
泛型	Generics	参数化类型	[Ch 22]

← 附录 A:推荐阅读 · → 附录 C:实验环境搭建

附录 C · 实验环境搭建

> 这本书强调"动手"。下面是各个 Part 推荐的免费工具,装上后所有例子都能跑。

C.1 通用基础

工具	用途	Windows	Linux/Mac
Git	版本管理	git-scm.com	`apt/brew install git`
VS Code	通用编辑器	code.visualstudio.com	同
Python 3.10+	跑本书示例	python.org	`apt/brew install python3`

C.2 Part I-II:数字逻辑仿真(可选)

如果想交互地玩 CMOS / 门电路:

- **Falstad Circuit Simulator** — falstad.com/circuit 🌟 — 浏览器里就能跑,极其直观。
- **LTspice** — 免费 SPICE 仿真器(模拟电路)。
- **Logisim Evolution** — 免费数字逻辑仿真,可拖拽搭电路。

C.3 Part III:Verilog 与 EDA(开源全栈)

商业工具(VCS、DC、ICC)需付费且通常需要公司账号。开源全套已足够本书使用:

```
# Linux / Mac
apt install iverilog gtkwave verilator yosys
brew install icarus-verilog gtkwave verilator yosys
```

工具	作用	对应章节
Icarus Verilog (iverilog)	Verilog 仿真器	Ch 8
Verilator	Verilog → C++ 高速仿真	Ch 8
GTKWave	看仿真波形(VCD)	Ch 8
Yosys	开源综合工具	Ch 9-10
OpenROAD / nextpnr	开源后端 / FPGA P&R	Ch 9

Hello-Verilog 验证步骤

```
# 1. 写 counter.v(本书 Ch 8 的 4 位计数器)
# 2. 写 tb_counter.v(testbench)
iverilog -o counter.vvp counter.v tb_counter.v
vvp counter.vvp                      # 跑仿真,生成 counter.vcd
gtkwave counter.vcd                   # 看波形

# 3. 综合到通用门级网表
yosys -p "read_verilog counter.v; synth; write_verilog -noattr counter_synth.v"
```

C.4 Part IV-V:RISC-V 工具链

```
# 装 RISC-V GCC 交叉编译器和模拟器
# Linux:
sudo apt install gcc-riscv64-unknown-elf qemu-system-misc

# Mac:
brew tap riscv/riscv
brew install riscv-tools
```

跑一个 hello.c on RISC-V:

```
riscv64-unknown-elf-gcc -O2 -S hello.c -o hello.s     # 看汇编
riscv64-unknown-elf-gcc hello.c -o hello              # 编出 ELF
qemu-riscv64 ./hello                                  # 模拟器跑
```

C.5 Part VI-VII:看 C 与高级语言的汇编

最方便的工具:**Compiler Explorer / godbolt.org** 🌟 — 浏览器里就能选编译器和优化级别,实时看汇编。

本地命令:

```
gcc -O2 -S example.c -o example.s              # x86 汇编
gcc -O2 -fno-asynchronous-unwind-tables -S \
    -masm=intel example.c -o example.s         # Intel 语法 + 干净

clang -O2 -emit-llvm -S example.c -o example.ll  # LLVM IR
```

C.6 Part VIII:迷你编译器

本书提供的迷你编译器 `code/part8-compiler/mini_compiler.py`,只要装了 Python 3.10+ 即可:

```
python code/part8-compiler/mini_compiler.py
```

预期输出:

```
== Bytecode ==
  ('PUSH', 3)
  ('PUSH', 4)
   ...
== Run ==
6
21
```

C.7 Part IX:操作系统实验

xv6 是 MIT 6.S081 的教学 OS,简化但完整:

```
git clone https://github.com/mit-pdos/xv6-riscv
cd xv6-riscv
make qemu                    # 在 QEMU 里跑 xv6,几秒就启动
```

C.8 Docker / 一站式环境(推荐新手)

如果不想分别装,可以拉一个 Docker:

```
docker run -it --rm -v $PWD:/work registry.gitlab.com/wallento/riscv-tools-insta
```

或者用 GitHub Codespaces / Gitpod —— 浏览器里直接打开本仓库,环境已配置好。

C.9 推荐目录结构

如果你要边读本书边动手,建议这样组织你的实验仓库:

```
my-experiments/
├── ch8-verilog/
│   ├── counter.v
│   └── tb_counter.v
├── ch15-asm/
│   ├── hello.c
│   └── hello.s
├── ch19-c-to-asm/
│   └── ...
├── ch24-mini-compiler/
│   └── extensions/   # 自己加 if/while/function
└── README.md
```

每读完一章,就在对应目录下做实验、做笔记。**这是把本书"知识"变成"能力"的关键。**

← 附录 B:术语表 · ↑ 回到首页